

微网与外部电网电力调控策略的优化模型

摘要

本文研究微网在电价、负载与光伏出力三重信息条件下的购电与储能调控问题,将四个问题统一归结为含储能的单母线能量流线性规划:以时段末储电量 E_t 、计划购电量 g_t 与净缺口 n_t 为核心量,按“确定信息—预测不确定—多阶段更新—电价波动”递进引入不确定性,全部模型均为线性规划,可用 HiGHS 精确求解。

针对问题 1,建立 144 个时段的确定性线性规划,约束含母线功率平衡、储能动态、容量与功率限制及日周期条件 $E_{144} = E_0$ 。由电量守恒导出全天购电量分解式 $\sum_t g_t = \text{净缺口} + \text{弃光} + (1/\eta_c - \eta_d)A$;由 KKT 条件证明最优策略为影子价格阈值策略 ($p_t < 0.9\lambda$ 充电、 $p_t > 1.111\lambda$ 放电,套利门槛价比 1.235)。求得全天购电费 **35101.57 元**、全天购电量 59482.70 kWh,较无储能基线节省 12950.48 元 (**27.0%**),0:00 与 24:00 储电量均为 6000 kWh。

针对问题 2,建立“日前预测—安全裕度—实际结算”的两阶段模型:经滚动精度评价与决策导向标定,负载取 7 日滑动平均 (RMSE 955.9 kW)、光伏取 EWMA(0.7,14) (MAE 150.1 kW);安全裕度以 1 月数据按总费用最小标定为 $(\alpha, \beta) = (0.15, 0)$;报童模型给出最优覆盖分位点 $c_u/(c_u + c_o) \in [0.80, 0.96]$,与标定结果互相印证。全年总费用 **1664.80 万元** (计划 1534.78、紧急 130.02),紧急购电量仅占负载的 **0.746%**,安全裕度节省 **496.37 万元 (22.97%)**。

针对问题 3,将“调低罚 0.5 倍、调高收 1.5 倍”的三档价格化简为对称罚金 $\Phi_t = p_t \tilde{g}_t + 0.5 p_t |\tilde{g}_t - \hat{g}_t|$,据此证明调整机制下的最优计划分位点为**中位数**,并给出更新价值分解 $V_\tau \approx 0.5 p \cdot \Delta \text{MAD} + \Delta(\text{紧急费})$ 。滚动时域重优化下题目方案 P3 的总费用为 **2136.95 万元**。实测 12:00 更新的边际价值为正 (5.25 万元),6:00 与 18:00 为负,逐小时更新亦无增益 (且 19:00–24:00 四个发布时刻的预报完全相同);信息价值分解显示完美光伏信息价值 200.25 万元,仅为完美负载信息价值 520.39 万元的 38%。故本题数据条件下不需要引入其他时刻的预报,制约收益的是负载侧不确定性。

针对问题 4,以 7 日滑动平均预测电价、按实际电价结算重算问题 2 与问题 3,总费用分别为 **2034.59 万元**与 **2224.44 万元** (较固定电价上升 22.2% 与 4.1%)。由于电价与负载的相关系数为 0.508,本文证明此时最优计划分位点**高于中位数**,与鲁棒模型“取价格上界”的结论一致。

模型检验方面,用独立线性规划 (母线侧变量布局) 与独立动态规划交叉验算确定性核心 (偏差 7×10^{-8} 与 0.276%),用独立结算公式复算全部费用并核对 5 个结果文件的结构与守恒关系,共 **40 项检查全部通过**;灵敏度分析表明充放电效率 (约 $\pm 4\%$) 与储电量上下限 (约 2.6%) 是主要影响参数,而日周期约束下初始电量对费用几乎无影响。全文以线性规划为统一内核,用影子价格、报童分位点与信息价值分解三条理论线索串联四问,使“策略为何如此”与“结果是多少”相互印证。

关键词: 微网购电优化; 储能套利; 安全裕度; 滚动时域优化; 信息价值; 线性规划

一、问题重述与分析

1.1问题背景

微网是为解决光伏等分布式能源本地消纳、减少并网问题而发展起来的小型发电系统。非孤岛式微网与外部电网（以下简称外网）相连：在外网电价较低或分布式能源有剩余电量的时段为储能设备充电，在外网电价较高的时段释放电能，从而在满足小区负载的前提下尽可能节省购电费用。因此，微网调控的核心是一类**带储能的时序能量调度问题**：利用小区负载、光伏发电量与储能设备当前储电量等数据，给出每个时段的计划购电量与储能充放电量。

附件 1-附件 4 给出了电价、小区负载、光伏发电预测与实际功率、光伏发电整点预报等数据；附录 1 给出了储能设备的容量、功率与效率参数。四个问题在**信息结构**上逐步升级：问题 1 为完全信息下的单日计划；问题 2 中负载与光伏需要自行预测，且缺电须按 5 倍电价紧急购电；问题 3 增加 6:00、12:00、18:00 三个时刻的光伏预报与付费调整机制；问题 4 中电价本身也实时波动，需在未知电价的条件下重算问题 2 与问题 3。

1.2四问的信息结构与要求

表 1 对比了四问的信息结构与决策要求。

表 1 四个问题的信息结构与决策要求对比

问题	决策时可获得的信息	决策时点	目标与关键约束
1	当日电价、小区负载、光伏预测功率全部已知（附件 1）	0:00 一次	全天购电费最小；供电不低于负载；储能 0:00 与 24:00 储电量相同
2	电价每天相同（附件 1）；负载与光伏仅有历史数据（附件 2），须自行预测	0:00 一次	计划购电费与紧急购电费之和的期望最小；缺电按交易时刻电价的 5 倍紧急购电
3	电价每天相同；负载须自预测；光伏有 0:00/6:00/12:00/18:00 的 24 小时整点预报（附件 3）	0:00、6:00、12:00、18:00	计划购电费、调整购电费、紧急购电费之和最小；调低部分按 0.5 倍、调高部分按 1.5 倍交易时刻电价计费
4	电价实时波动（附件 4），0:00 决策时未知	同问题 2、3	同问题 2、3，但全部费用按实际交易时刻电价结算

四个问题还要求按统一的表格格式输出：表 1 给出指定时段的购电量与全天购电量、购电费；表 2 给出储能设备在六个 4 小时时段的充放电量以及 0:00、24:00 的储电量；表 3 给出指定日期（2025-03-20、06-21、09-23、12-21）的紧急购电量。问题 3 另需分析**是否需要引入其他时刻的预报制定调整购电策略**。

1.3统一框架与核心量

通读四问可以看到，尽管信息条件不同，其物理过程、几何约束与优化目标完全一致：**同一根母线上，购电量、光伏出力与储能放电之和等于负载与储能充电之和；储能**

电量按效率递推；以购电费用为目标。据此本文建立全文统一框架：

本文将所有问题统一归结为含储能的单母线能量流线性规划；四问的差别仅是”进入模型的序列是确定值、预测量，还是带更新的预测量”以及”偏差如何计价”。

据此定义贯穿全文的三个核心量（表 4）：时段末储电量 E_t 、计划购电量 g_t 与净缺口 $n_t = (L_t - P_t)\Delta t$ 。四问递进关系为：确定信息下单日优化（问题 1）→ 预测替代真值并引入安全裕度（问题 2）→ 预报更新下的滚动重优化与偏差计价（问题 3）→ 电价亦不确定时按实际价格结算（问题 4）。

1.4 数据处理与关键统计

表 2 汇总了四个附件的内容与规模。

表 2 附件数据内容与规模

附件	内容	规模
1	某天的电价、小区负载、光伏发电预测功率（10 分钟粒度）	144×4
2	2025.1.1–12.31 每天的小区负载与光伏发电实际功率	$2 \times (365 \times 145)$
3	每天 0:00、6:00、12:00、18:00 发布的未来 24 小时整点光伏预报	1460×26
4	2025.1.1–12.31 每天的实时电价（10 分钟粒度）	365×145

数据处理上作了三项核对：其一，附件 2、附件 4 的时间标签为区间结束时刻（如 00:10 表示 $[0:00, 0:10]$ ，0:00+1 表示 $[23:50, 24:00]$ ），据此 144 个标签恰好完整覆盖 $[0:00, 24:00]$ ，与”0:00 制定全天计划””0:00 与 24:00 储电量相同”的表述自洽；其二，附件 3 的”预报 k 小时”是相对于发布时刻的第 k 小时（6:00 发布的”预报 1 小时”指当日 07:00），该对齐方式下各时段误差量级合理（见后文表 13），若误按”当日第 k 小时”对齐将产生 6–18 小时的系统性错位；其三，附件 2 存在格式差异（时间列为时间对象而非字符串），读取时统一转换为数值矩阵。

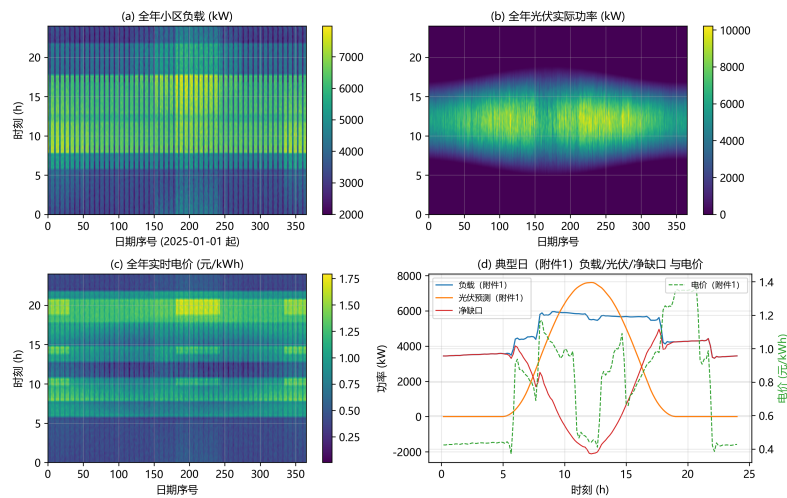


图 1 附件数据总览：(a) 全年小区负载热力图；(b) 全年光伏实际功率热力图；(c) 全年实时电价热力图；(d) 典型日（附件 1）负载、光伏、净缺口与电价

图 1 与 表 3 汇总了数据的主要特征：

表 3 关键统计量（实测）

对象	统计量	数值
附件 1 电价	最小值 / 最大值 / 均值	0.3713 / 1.3952 / 0.7662 元/kWh
附件 1 负载	最小值 / 最大值 / 均值 / 日电量	3309 / 5959 / 4626 kW / 111025 kWh
附件 1 光伏预测	峰值 / 日电量 / 占负载比例	7612 kW / 55483 kWh / 50.0%
附件 2 负载	全年最小 / 最大 / 均值	1996 / 7979 / 4626 kW
附件 2 光伏	日电量最小 / 均值 / 最大	34361 / 55483 / 75223 kWh
月度净缺口	12 月最大 / 5 月最小	76259 / 38284 kWh/日
附件 4 电价	最小 / 最大 / 均值	0.0076 / 1.7936 / 0.7662 元/kWh
附件 4 日内极差	均值 / 最大	1.1073 / 1.4995 元/kWh
相关性	电价-负载 / 电价-光伏	0.508 / 0.028

图 2 进一步给出月度特征。由图 1 and 2 and 表 3 可得三点对建模直接有用的认识：

（1）**附件 1 是全年典型日**。其负载与光伏曲线与附件 2 的全年均值曲线相关系数均为 1.0000，说明附件 1 并非任意一天，而是官方给定的”典型日”，可直接作为问题 2 的基准预测轮廓或与滑动平均预测作对照。

（2）**储能容量相对缺口偏小**。可用容量为 $10800 - 1200 = 9600$ kWh，仅占典型日净缺口（55542 kWh）的 17%、日均负载电量的 8.6%。因此储能的角色是时段间的价格搬运而非替代购电，全天购电量不可能低于净缺口。

（3）**季节差异显著**。12 月与 1 月的日均净缺口（76259、75278 kWh）约为 5 月（38284 kWh）的 2 倍，且 12 月电价最高（月均 0.8617 元/kWh）；光伏日电量在 12 月仅 36332 kWh、8 月可达 69984 kWh。这意味着预测误差与紧急购电风险在冬季最集中，安全裕度应随季节调整。

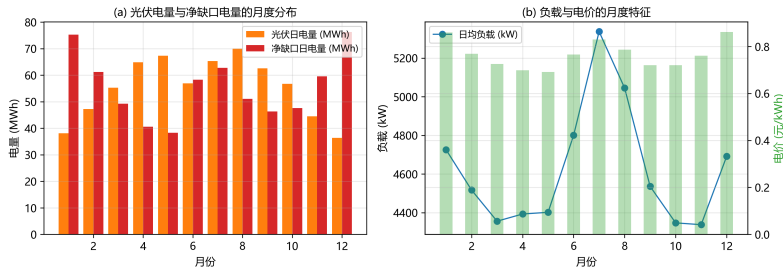


图 2 月度特征：(a) 光伏日电量与净缺口电量的月度分布；(b) 日均负载与月均电价的月度特征

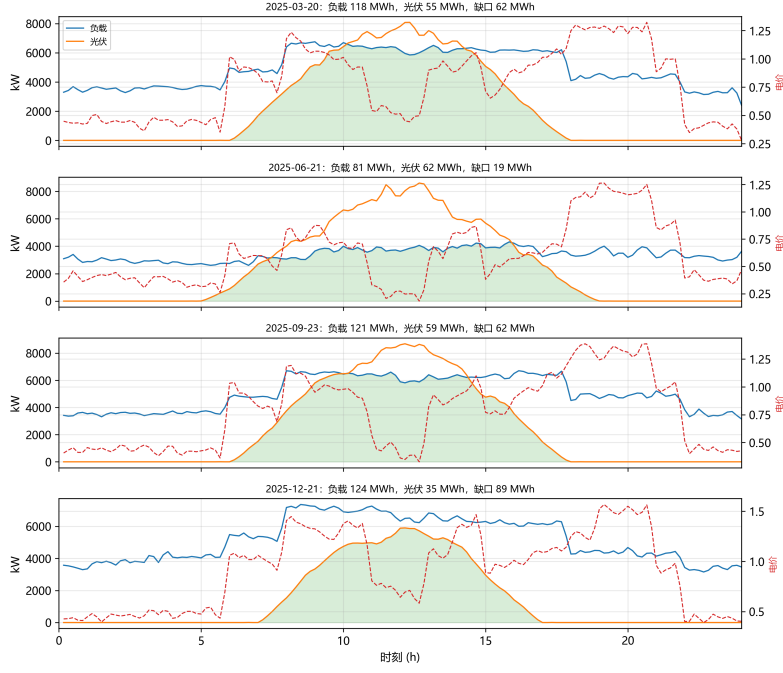


图 3 表 3 指定四天（2025-03-20、06-21、09-23、12-21）的负载、光伏与实时电价

图 3 给出了表 3 四个指定日期的数据画像：06-21（夏至）光伏日电量最高（62073 kWh）、净缺口最小（19335 kWh）；12-21（冬至）光伏仅 35339 kWh、净缺口达 89038 kWh 且电价均值最高（0.9101 元/kWh），是最严酷的调度场景。这四天恰好覆盖光伏出力的四季极端，后文各问均按此四天给出明细结果。

二、模型假设与符号说明

2.1 模型假设

根据微网运行的物理规律与题目给定条件，本文作如下假设，并标注其强度：[强] 表示由题目或物理规律直接支持、全文统一采用；[弱] 表示存在合理歧义、本文明确确定并在检验部分作敏感性讨论。

[强] 假设

- (1) 时间离散化：以 $\Delta t = 10/60 = 1/6$ h 为时段长度，全天划分为 $T = 144$ 个时段；时段内电价、负载与光伏功率视为恒定，能量等于功率与 Δt 之积。
- (2) 充放电效率各为 0.9，即充电时从母线取用的电能仅有 90% 存入电池，放电时电池需释放 $1/0.9$ 的电能才能向母线提供 1 kWh（往返效率 81%）。
- (3) 储能最大容量 12000 kWh，最大充放电功率 5000 kW，为保证不过充过放，储量始终限制在 $[1200, 10800]$ kWh 之间；功率上限按电池侧理解，即单时段电池侧充放电量不超过 $5000\Delta t = 833.33$ kWh。
- (4) 2025-01-01 0:00 储电量为 6000 kWh；问题 1 按题目要求取 $E_{144} = E_0$ ，问题 2-4 取日周期边界 $E_{144} = E_0$ （原因见 §4.6）。
- (5) 允许弃光（光伏盈余无法入库时直接舍弃），不允许向电网售电，购电量非负；外网容量充足，不构成约束。

- (6) 储能在日内严格按计划充放电；若实际光伏低于预测，由此产生的负载缺额按紧急购电处理。

[弱] 假设

- (7) 附件 2、附件 4 的时间标签为区间结束时刻；结果模板的 144 行（列）与数据按时序一一对应填充，模板行 $a-b$ 对应数据标签 b 。
- (8) 附件 3 的”预报 k 小时”为相对发布时刻的第 k 小时；从整点预报到 10 分钟粒度采用分段常数插值（整点值代表其后 1 小时）。
- (9) 问题 2 的 0:00 预测只使用该日之前的历史负载与光伏数据，不使用附件 3 的预报；问题 3、4 使用附件 3 的预报作为光伏输入，负载仍由历史数据预测。
- (10) 问题 3 的违约金以 0:00 计划为唯一基准一次结算；问题 4 中计划购电、违约与紧急购电的费用一律按附件 4 的实际电价结算，而决策阶段使用历史电价预测。

2.2 符号说明

表 4 主要符号与含义

符号	含义	符号	含义
$T, \Delta t$	时段数 144、时段长 1/6 h	η_c, η_d	充电、放电效率 (0.9)
L_t	时段 t 小区负载功率 (kW)	P_t	时段 t 光伏功率 (kW)
p_t	时段 t 外网电价 (元/kWh)	g_t	计划购电量 (kWh)
a_t, b_t	电池侧充电量、放电量 (kWh)	s_t	弃光电量 (kWh)
E_t	时段末储电量 (kWh)	e_t	紧急购电量 (kWh)
$\tilde{g}_t$	调整后购电量 (kWh)	$\hat{g}_t$	0:00 计划购电量 (kWh)
n_t	净缺口 $n_t = (L_t - P_t)\Delta t$ (kWh)	A	电池侧全天吞吐量 $\sum_t a_t$ (kWh)
$\bar{A}$	单时段电池侧最大充放电量	E_0	日初储电量 (kWh)
α, β	负载上浮、光伏下调的安全裕度	λ	储电量影子价格 (元/kWh)

2.3 共同物理模型

四问共用同一物理模型^[1]。设时段 $t = 1, \dots, T$ ，时段 t 表示区间 $((t-1)\Delta t, t\Delta t]$ 。

母线功率平衡：购电量、光伏可用量与储能放电量之和等于负载与储能充电量之和，未被利用的光伏以弃光形式计入：

$$g_t + \eta_d b_t + P_t \Delta t = L_t \Delta t + \frac{a_t}{\eta_c} + s_t, \quad t = 1, \dots, T, \quad (1)$$

其中 $s_t \geq 0$ 。由式 (1) 与 $s_t \geq 0$ 立即得到题目要求的”微网提供的电能不低于小区负载”： $g_t + \eta_d b_t + P_t \Delta t \geq L_t \Delta t$ ，故无需另设约束。

储能动态：电池侧充放电量与储量的递推关系为

$$E_t = E_{t-1} + a_t - b_t, \quad t = 1, \dots, T, \quad (2)$$

效率已体现在式 (1) 中（充电量 a_t 对应母线取用 a_t/η_c ，放电量 b_t 对应向母线提供 $\eta_d b_t$ ），因此式 (2) 不含效率系数，量纲为 kWh。

变量界与边界条件:

$$0 \leq a_t \leq \bar{A}, \quad 0 \leq b_t \leq \bar{A}, \quad g_t \geq 0, \quad s_t \geq 0, \quad E_{\min} \leq E_t \leq E_{\max}, \quad (3)$$

$$E_0 = 6000 \text{ kWh}, \quad E_T = E_0 \quad (\text{问题 1 为题目要求, 问题 2-4 为日周期口径}), \quad (4)$$

其中 $E_{\min} = 1200$ 、 $E_{\max} = 10800$ kWh, $\bar{A} = P_b \Delta t = 833.33$ kWh。

量纲核对: 式 (1) 中 $g_t, a_t/\eta_c, s_t$ 为 kWh, $P_t \Delta t$ 、 $L_t \Delta t$ 为 kW × h = kWh; 式 (2) 各项为 kWh; 功率上限经 $\bar{A} = P_b \Delta t$ 换算为能量上限, 量纲一致。目标函数中 $p_t g_t$ 的单位为元, 效率因子 η_c, η_d 无量纲。

2.4 共同模型的三条结构性质

这三条性质对四问均成立, 是后文分析的基础。

性质 1 (充放电互斥, 故模型保持线性规划): 若某时段 $a_t > 0$ 且 $b_t > 0$, 令 $a'_t = a_t - x$ 、 $b'_t = b_t - y$ 并取 $x/\eta_c = \eta_d y$, 则 式 (1) 不变, 而 E_t 的变化量为

$$-x + y = x \left(\frac{1}{\eta_c \eta_d} - 1 \right) > 0, \quad (5)$$

即储电量严格增加、费用与可行性均不劣。故存在最优解满足 $a_t b_t = 0$ 。这说明无需引入 0-1 变量即可用连续线性规划精确求解。

性质 2 (全天购电量分解式): 对 式 (2) 全天求和并利用 $E_T = E_0$ 得 $\sum_t a_t = \sum_t b_t \triangleq A$; 代入 式 (1) 全天求和得

$$\sum_{t=1}^T g_t = \underbrace{\sum_{t=1}^T (L_t - P_t) \Delta t}_{\text{净缺口能量}} + \sum_{t=1}^T s_t + \left(\frac{1}{\eta_c} - \eta_d \right) A = \text{净缺口} + \text{弃光} + \frac{19}{90} A, \quad (6)$$

式中 $A = \sum_t a_t$ 为电池侧全天吞吐量, 与母线侧充电量相差效率因子: $A = \eta_c \sum_t (a_t/\eta_c)$, 系数 $\frac{1}{\eta_c} - \eta_d = \frac{19}{90} = 0.2111\bar{1}$ 为精确值; 等价地, 损耗项也可写为 $(1 - \eta_c \eta_d) \times$ 母线侧充电量 $= 0.19 \sum_t (a_t/\eta_c)$ 。该式表明: 每在电池中搬运 1 kWh (电池侧), 全天需多购 $19/90 = 0.2111$ kWh; 换算为向负载多供 1 kWh 的代价是 $1/(\eta_c \eta_d) - 1 = 23.46\%$ 。储能不能“创造”电量, 其价值完全来自时段间的价格差搬运。式 (6) 同时是后文数值结果的守恒校验依据。

性质 3 (阈值策略): 设 式 (2) 的对偶变量 (储电量影子价格) 为 λ_t (元/kWh)。当购电为边际供能方式时, KKT 条件^[2]给出: 储电量在上下限内部且既不充也不放的时段 λ_t 保持不变; 充电时段 $p_t = \eta_c \lambda$, 放电时段 $p_t = \lambda/\eta_d$ 。因此存在阈值 λ , 最优策略为

$$\text{充电当 } p_t < 0.9\lambda, \quad \text{放电当 } p_t > 1.111\lambda, \quad \text{其余时段闲置}, \quad (7)$$

充放之间的价格比门槛为 $1/(\eta_c \eta_d) = 1.235$ 。 λ 由功率上限、储电量上下限与边界条件内生决定。式 (7) 说明储能的最优行为是“按影子价格筛选取价格低于门槛的时段充电、高于门槛的时段放电”, 是后文解释最优计划结构的理论依据。

三、问题 1：完全信息下的日计划购电模型

3.1 模型建立

问题 1 中电价、小区负载与光伏预测功率全天已知，故可在 0:00 一次求解全天最优计划。以式 (1)–式 (4) 为基础，建立如下线性规划：

$$\begin{aligned} \min_{\{g_t, a_t, b_t, s_t, E_t\}} F_1 &= \sum_{t=1}^T p_t g_t. \quad (8) \\ \text{s.t.} \quad &\begin{cases} g_t + \eta_d b_t + P_t \Delta t = L_t \Delta t + \frac{a_t}{\eta_c} + s_t, & \text{母线功率平衡} \\ E_t = E_{t-1} + a_t - b_t, & \text{储能动态} \\ E_{144} = E_0 = 6000, & \text{日周期条件} \\ 1200 \leq E_t \leq 10800, \quad 0 \leq a_t, b_t \leq 833.33, & \text{状态与功率约束} \\ g_t \geq 0, \quad s_t \geq 0. & \text{非负与弃光} \end{cases} \quad (9) \end{aligned}$$

模型为线性规划：决策变量 720 个（ g, a, b, s 各 144 个、 E 为 144 个），等式约束 289 条（平衡 144 条、动态 144 条、终端 1 条），状态与功率约束以变量界表达。采用 HiGHS 求解器^[8] 与 SciPy 科学计算库^[9] 求解，规模极小，可在毫秒级得到全局最优解。

3.2 模型性质与最优计划结构

(1) 全天购电量分解。由式 (6)，问题 1 的全天购电量满足“全天购电量 = 净缺口 + 弃光 + 0.2111A”。代入实测数据：净缺口 55541.97 kWh，弃光 0，电池侧吞吐量 $A = 18666.60$ kWh，理论购电量为 $55541.97 + \frac{19}{90} \times 18666.60 = 59482.70$ kWh，与求解结果一致（残差 2.2×10^{-11} ）；其中 $A = 18666.60$ kWh 为电池侧吞吐量，等于 $0.9 \times$ 母线侧充电量（ $0.9 \times 20740.67 = 18666.60$ kWh）。该恒等式说明储能套利的“代价”是 23.46% 的能量损耗，其收益必须来自足够大的时段价差。

(2) 套利判据。设电池侧电量 A 在时段 t_c 充入、在时段 t_d 放出，则费用变化为

$$\Delta F = A \left(\frac{p_{t_c}}{\eta_c} - \eta_d p_{t_d} \right), \quad (10)$$

当且仅当 $p_{t_d}/p_{t_c} > 1/(\eta_c \eta_d) = 1.235$ 时值得搬运。用附件 1 电价核对主要窗口：夜间 0.42 元/kWh 充电、晚峰 1.30 元/kWh 放电的价比为 3.10；午间 11 时 0.47 充电、晚峰 1.30 放电的价比为 2.77；清晨 4 时 0.44、早峰 8 时 1.08 的价比为 2.46。三者均远超 1.235，故最优解必然表现为“夜充、峰放、日末回充”的结构。

(3) 阈值策略验证。式 (7) 预测了“充电时段电价低于阈值、放电时段电价高于阈值”的分布特征。图 4 将 144 个时段的充放电状态标注在电价轴上：充电时段电价集中在 0.37–0.53 元/kWh（夜间与午间谷段），放电时段电价集中在 0.87–1.35 元/kWh（早晚峰），两个区间几乎无重叠，与理论阈值策略一致。这说明最优计划不是“逐时段贪心”，而是由储电量影子价格 λ 统一决定的套利行为。

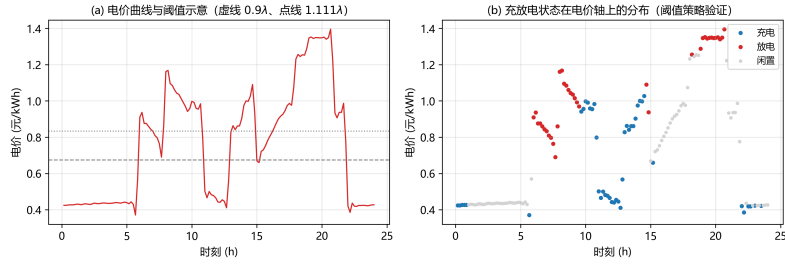


图 4 问题 1 的阈值策略验证：(a) 附件 1 电价曲线与阈值示意；(b) 充、放、闲置状态在电价轴上的分布

3.3 求解结果

求解得全天购电费 $F_1^* = 35101.57$ 元、全天购电量 59482.70 kWh；若不计储能（逐时段按净缺口购电），全天购电费为 48052.05 元，故储能带来 27.0% 的费用下降，即 12950.48 元。储电量轨迹为：0:00 与 24:00 均为 6000 kWh，最低 1200 kWh（20:40），最高 10800 kWh（05:40），全程满足 $[1200, 10800]$ kWh 的约束；电池侧单时段最大充电量达到上限 833.33 kWh（功率约束起作用），最大放电量为 795.17 kWh（电池侧；对应母线侧 $0.9 \times 795.17 = 715.65$ kWh）。

表 5 问题 1 指定时段的购电量与全天合计（表 1 格式）

时间段	购电量 (kWh)	时间段	购电量 (kWh)	时间段	购电量 (kWh)
10:00–10:10	0.00	12:00–12:10	573.01	14:00–14:10	0.00
16:00–16:10	445.43	18:00–18:10	531.89	20:00–20:10	0.00
全天购电量 / 全天购电费			59482.70 kWh / 35101.57 元		

表 6 问题 1 储能设备在指定时间段的充放电量与储电量（表 2 格式）

时间段	充电量 (kWh)	放电量 (kWh)
0:00–4:00	4407.41	0.00
4:00–8:00	925.93	6365.84
8:00–12:00	4417.59	1703.00
12:00–16:00	5656.41	91.10
16:00–20:00	0.00	5780.13
20:00–24:00	5333.33	2859.87
0:00 / 24:00 储电量	6000.00 kWh / 6000.00 kWh	

表 5 给出表 1 指定时段的购电量，表 6 给出储能充放电量。由表 6 and 图 5 可见，储能全天完成约 1.9 个充放循环：0:00–4:00 夜间低谷充电 4407.41 kWh，4:00–8:00 早峰放电 6365.84 kWh；8:00–12:00 与 12:00–16:00 在午间次低谷继续充电（合计 10074.00 kWh），16:00–20:00 晚峰放电 5780.13 kWh；20:00–24:00 电价回落，再充入 5333.33 kWh 以满足日周期条件。充电量按母线侧口径统计，故全天充电量（20740.67 kWh）大于放电量（16799.94 kWh），差额即往返效率损耗。

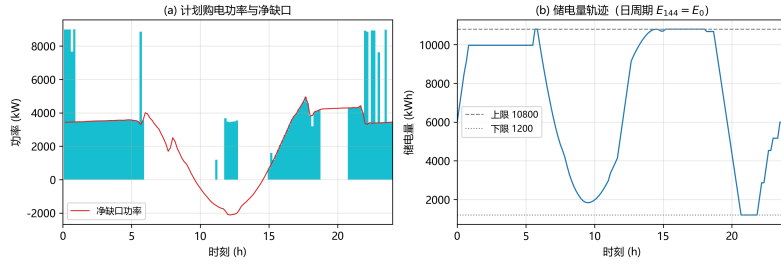


图 5 问题 1 的最优计划：(a) 计划购电功率与净缺口；(b) 储电量轨迹（日周期 $E_{144} = E_0$ ）

3.4 灵敏度分析

为检验结论的稳健性，用参数化线性规划对效率、功率上限、储电量上下限、初始电量与电价水平作单因素扰动，结果见图 6 与表 7。

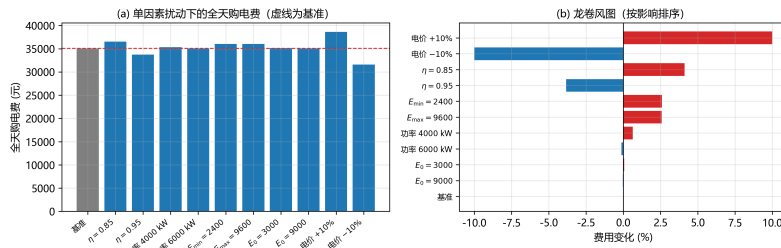


图 6 问题 1 的单因素灵敏度：(a) 各扰动情景下的全天购电费；(b) 龙卷风图（按影响大小排序）

表 7 问题 1 灵敏度分析结果

扰动情景	全天购电费 (元)	相对基准变化	电池侧吞吐量 (kWh)
基准	35101.57	—	18666.60
效率 $\eta = 0.85$	36548.22	+4.12%	19199.93
效率 $\eta = 0.95$	33756.16	-3.83%	18811.66
最大功率 4000 kW	35327.90	+0.64%	18510.81
最大功率 6000 kW	35056.72	-0.13%	18769.48
储量下限 2400 kWh	36014.15	+2.60%	16901.22
储量上限 9600 kWh	36008.47	+2.58%	16901.22
初始电量 3000 kWh	35125.20	+0.07%	18666.60
初始电量 9000 kWh	35093.43	-0.02%	18666.60
电价 +10%	38611.72	+10.00%	18666.60
电价 -10%	31591.41	-10.00%	18666.60

三点结论：其一，效率是最敏感的内部参数——往返效率由 81% 降至 72.25% ($\eta = 0.85$) 时费用上升 4.12%，且吞吐量反而增加（需搬运更多电量补偿损耗），说明效率直接决定套利的可行空间；其二，储电量上下限的影响（约 2.6%）大于功率上限（约 0.6%），因为可用容量仅为日净缺口的 17%，容量而非功率是套利的紧约束；其三，日周期条件下初始电量几乎无影响 ($\pm 0.07\%$)，因为 $E_{144} = E_0$ 要求全天自平衡，初始电量只改变循环起点而不改变搬运总量；电价整体升降则近似线性传递到费用（弹性约为 1）。

四、问题 2：预测不确定下的两阶段购电模型

4.1 信息结构与决策时序

问题 2 中电价每天相同（取附件 1 的 144 段电价），而小区负载与光伏发电每天不同，且 0:00 决策时只能获得历史数据。决策结构为两阶段：第一阶段在 0:00 用历史数据预测当日负载与光伏，并一次确定全天的计划购电量 g_t 与充放电计划 a_t, b_t ；第二阶段在运行中被动接受实际结果——若某时段的实际供给低于负载，则按交易时刻电价的 5 倍紧急购电。总费用为

$$F_2 = \sum_{t=1}^T p_t g_t + \sum_{t=1}^T 5p_t e_t, \quad e_t = (L_t \Delta t - g_t - \eta_d b_t - P_t \Delta t)^+, \quad (11)$$

其中 $(\cdot)^+ = \max(\cdot, 0)$ 。由于 L_t, P_t 在决策时未知， F_2 为随机变量，目标为期望费用最小，即 $\min \mathbb{E}[F_2]$ 。

需要强调的是，紧急电价是计划电价的 5 倍且随 p_t 缩放：晚峰时段（ $p_t \approx 1.35$ 元/kWh）缺电的代价高达 6.74 元/kWh，是夜间（ $0.43 \rightarrow 2.14$ 元/kWh）的 3 倍以上。因此”预留多少安全电量”成为本问的核心决策。

4.2 第一层：日前预测与决策导向评价

本文考察八类同时段预测器^[7]：昨日持续、3/7/14/30 日滑动平均、指数加权平均 EWMA(0.7, 14) 与 EWMA(0.5, 7)、同月历史均值。所有预测均严格使用预测日之前的数据（滚动评价，避免前视偏差），结果见表 8 与图 7。

表 8 预测器精度的滚动评价（2025-02-01 至 12-31，共 334 天，单位：kW）

预测器	负载 MAE	负载 RMSE	光伏 MAE	光伏 RMSE
昨日持续	650.0	1128.4	185.3	375.2
3 日滑动平均	945.1	1241.3	154.5	313.5
7 日滑动平均	781.2	955.9	153.4	303.9
14 日滑动平均	782.7	959.8	171.5	327.2
30 日滑动平均	789.8	973.5	225.6	400.1
EWMA(0.7,14)	845.0	1071.4	150.1	301.0
EWMA(0.5,7)	812.6	1110.8	154.1	312.6
同月历史均值	813.8	1029.3	178.1	334.7

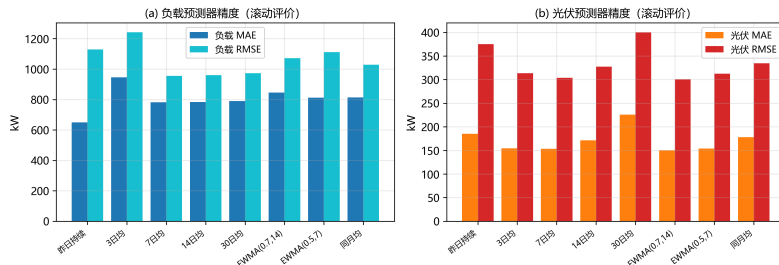


图 7 预测器精度的滚动评价：(a) 负载；(b) 光伏

表 8 显示了一个值得注意的现象：负载的 MAE 最优（昨日持续，650.0 kW）与 RMSE 最优（7 日滑动平均，955.9 kW）并不一致。由于紧急购电只由误差的”正

尾”触发，而 5 倍电价对大幅偏差的惩罚远重于小幅偏差，MAE 最小并不等于费用最低。因此本文不单纯以精度指标选型，而采用**决策导向评价**：在 1 月数据上，对候选预测器与安全裕度作联合标定，以”计划购电费 + 紧急购电费”的实测总费用为准则。最终选取负载采用 7 日滑动平均（RMSE 最优）、光伏采用 EWMA(0.7,14)（MAE 与 RMSE 双优）。

4.3 第二层：安全裕度模型与标定

由于 0:00 的预测存在误差，若按点预测制定计划，则实际光伏低于预测的时段必然触发高价紧急购电。为此引入**安全裕度**，用较”悲观”的输入编制计划：

$$\tilde{L}_t = (1 + \alpha)\hat{L}_t, \quad \tilde{P}_t = (1 - \beta)\hat{P}_t, \quad \alpha, \beta \geq 0, \quad (12)$$

其中 $\hat{L}_t, \hat{P}_t$ 为点预测， α 为负载上浮系数， β 为光伏下调系数。计划子问题为

$$\min_{\{g_t, a_t, b_t, s_t, E_t\}} \sum_{t=1}^T p_t g_t \quad \text{s.t.} \quad \begin{cases} g_t + \eta_d b_t + \tilde{P}_t \Delta t = \tilde{L}_t \Delta t + \frac{a_t}{\eta_c} + s_t, \\ E_t = E_{t-1} + a_t - b_t, \quad 1200 \leq E_t \leq 10800, \quad E_{144} = E_0, \\ 0 \leq a_t, b_t \leq 833.33, \quad g_t \geq 0, \quad s_t \geq 0. \end{cases} \quad (13)$$

裕度参数在 1 月 8-31 日（共 24 天，使用其之前的历史数据预测）上按总费用最小标定：

$$\min_{\alpha, \beta} \sum_{d \in \mathcal{D}_{\text{train}}} \left[\underbrace{\sum_t p_t g_t^{(d)}}_{\text{计划费}} + \underbrace{\sum_t 5p_t e_t^{(d)}}_{\text{紧急费}} \right]. \quad (14)$$

表 9 预测器与安全裕度的联合标定结果（前 5 名，训练集为 1 月 8-31 日）

排名	负载上浮 α	光伏下调 β	1 月总费用 (元)	说明
1	0.15	0.00	1524501.83	最优
2	0.12	0.06	1528941.47	
3	0.12	0.00	1529084.42	
4	0.15	0.06	1547144.12	
5	0.12	0.12	1555544.16	
—	0.00	0.00	1945353.62	无裕度基线（同预测器组合）

表 9 列出前 5 名标定结果。标定的最优裕度为 $\alpha = 0.15, \beta = 0$ ，1 月总费用由 1945353.62 元降至 1524501.83 元，**节省 420851.79 元（21.63%）**。最优 α 落在搜索区间内部（图 8(b) 呈单峰形态），说明”裕度越大越好”的判断并不成立：过度预留会把购电费推高，反而抵消紧急费的下降。

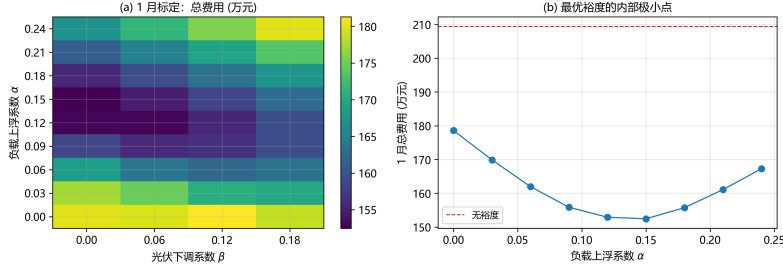


图 8 问题 2 的安全裕度标定：(a) (α, β) 平面上 1 月总费用的响应面；(b) 最优裕度的内部极小点

4.4 裕度的理论区间：报童模型解释

为解释标定得到的裕度是否合理，将单时段决策近似为报童问题。设该时段所需“计划供给量”为 q 、实际需求为随机量 D ：少计划 1 kWh 的代价是改为按 $5p_t$ 紧急购电，即缺货成本

$$c_u = 5p_t - p_t = 4p_t; \quad (15)$$

多计划 1 kWh 而未被使用的代价取决于能否入库留用：若储能尚有余量，多买的电只损失往返效率， $c_o = p_t(1 - \eta_c \eta_d) = 0.19p_t$ ；若储能已满或无余量，则该电量被直接浪费， $c_o = p_t$ 。由报童模型^[3]，最优覆盖分位点为

$$q^* = F_D^{-1}\left(\frac{c_u}{c_u + c_o}\right) \in \left[\frac{4}{4+1}, \frac{4}{4+0.19}\right] = [0.80, 0.96]. \quad (16)$$

即计划供给应覆盖净需求不确定性的 80%–96% 分位。该区间与数值标定的 $\alpha = 0.15$ 互相印证：在负载预测 RMSE 约 956 kW 的量级下，上浮 15%（约 694 kW，与 0.7 倍标准差相当）正落在该分位区间的合理范围内。同时， $c_u = 4p_t$ 与 c_o 均随 p_t 等比缩放，故各时段的最优分位点相同，但绝对代价不同——储能容量有限时，安全储备应优先分配给高价时段，这正是“晚峰前把电储足”的严格解释。

4.5 对照模型

(1) 两阶段随机规划（样本平均近似）^[4]。设情景集 $\{(\pi_s, L_t^s, P_t^s)\}_{s=1}^S$ 由历史日整条曲线抽样生成（保留时段间相关结构），则

$$\min \sum_t p_t g_t + \sum_s \pi_s \sum_t 5p_t e_t^s \quad \text{s.t.} \quad \begin{cases} g_t + \eta_d b_t + P_t^s \Delta t + e_t^s = L_t^s \Delta t + \frac{a_t}{\eta_c} + s_t^s, & \forall t, s \\ \text{储能动态、状态与功率约束同式 (13),} & \forall t \\ g_t, a_t, b_t, E_t \text{ 为第一阶段变量（不依赖情景）,} \\ e_t^s, s_t^s \geq 0 \text{ 为第二阶段追偿变量.} \end{cases} \quad (17)$$

由于全部约束线性、 e_t^s 的费用为正，最优解自动满足 $e_t^s = (L_t^s \Delta t - \text{供给})^+$ ；当情景数与分布趋近真实时，式 (17) 的最优值趋于 $\min \mathbb{E}[F_2]$ ，故可作为裕度法的理论参照。

(2) 鲁棒优化。取箱式不确定集^[5] $L_t \in [\hat{L}_t - \Gamma_L \sigma_t^L, \hat{L}_t + \Gamma_L \sigma_t^L]$ 、 $P_t \in [\hat{P}_t - \Gamma_P \sigma_t^P, \hat{P}_t + \Gamma_P \sigma_t^P]$ （ σ 由历史残差估计）。由于 e_t 关于 L_t 单调增、关于 P_t 单调减，最坏情形在负载取上界、光伏取下界处同时达到，故鲁棒对应模型恰好等价于把式 (12) 的裕度取为

$\alpha = \Gamma_L \sigma_t^L / \hat{L}_t$ 、 $\beta = \Gamma_P \sigma_t^P / \hat{P}_t$ 的确定性线性规划。这说明了裕度法与鲁棒优化之间的关系：裕度框架是”柔性的鲁棒—随机折中”，其参数由数据标定而非先验给定。为给出量化对照，表 10 在同一 334 天报告期上比较了五种模型：本文的预测 + 裕度模型总费用最低（除不可达的完美信息下界外），两阶段随机规划（10 情景）高出 2.00%，鲁棒模型因过度保守高出 24.06%，无裕度计划高出 29.82%。

表 10 问题 2 的模型对比（334 天，样本外结算）

模型	全年总费用 (元)	紧急购电量 (kWh)	相对主模型
本文：预测 + 安全裕度 ($\alpha = 0.15$)	16647985	275908	—
对照一：无裕度计划 ($\alpha = \beta = 0$)	21611681	2225337	+29.82%
对照二：两阶段随机规划 (SAA, $S = 10$)	16980919	94049	+2.00%
对照三：鲁棒优化 ($\Gamma = 1$, 箱式不确定集)	20653801	691072	+24.06%
对照四：完美信息下界	12229643	0	-26.54%

注：对照二的情景取该日之前 10 天的实际负载与光伏曲线，第一段计划在样本外与实际数据结算；对照三按 $\hat{L} + \sigma_L$ 、 $\hat{P} - \sigma_P$ 编制计划， σ 取前 10 天同时段标准差。

4.6 日周期边界与储能执行规则

题目仅对问题 1 明确要求”储能设备在 0:00 和 24:00 的储电量相同”，问题 2–4 未直接规定终端电量。但若完全不加终端条件，单日目标中”剩余电量”没有价值，最优解必然在当日最后一个高价时段把电量放到下限 1200 kWh，逐日如此即形成退化。因此必须补充终端口径。本文采用日周期条件 $E_{144} = E_0 = 6000$ kWh：其优点是题目给定的 2025-01-01 0:00 电量 6000 kWh 相容、每天的计划可重复执行、且不引入额外的未来信息。需要注意，问题 2、3 中电价每天完全相同，稳态下”今晚剩余 1 kWh 的价值”与”明晚同一时刻的价值”相等，故日周期解与跨日最优解的差异仅在二阶量级（来源于负载与光伏的逐日差异），不影响结论；问题 4 中电价逐日波动，日周期口径会略微低估跨日搬运的收益，本文在 §八 中对此作专门讨论。

储能执行规则为：储能在日内严格按 0:00 计划充放电；当实际光伏低于预测导致负载缺额时，按式 (11) 紧急购电；实际光伏高于预测产生的盈余按弃光处理。在该规则下实际储电量轨迹与计划一致，故逐日 0:00 与 24:00 储电量均可复核。

4.7 求解结果

在报告期（2025-02-01 至 12-31，共 334 天）上按上述模型求解，全年结果见表 11，月度分布见图 9。

表 11 问题 2 全年费用与对照结果（334 天）

指标	计划购电费 (元)	紧急购电费 (元)	总费用 (元)
本文模型（7 日均 + EWMA, $\alpha = 0.15$ ）	15347789.64	1300195.25	16647984.89
对照：无裕度计划 ($\alpha = \beta = 0$)	—	—	21611681.31
对照：完美信息下界	—	—	12229643.06

全年计划购电费 15347789.64 元、紧急购电费 1300195.25 元，总费用 16647984.89

元；紧急购电量合计 275907.99 kWh，仅占同期负载电量的 0.746%，出现紧急购电的天数为 240 天（多为小幅补缺）。相较无裕度计划，安全裕度带来 4963696.42 元的费用下降，降幅 22.97%；与完美信息（逐日按实际负载与光伏求解，12229643.06 元）相比，本文模型的”信息缺口”为 4418341.83 元，占完美信息费用的 36.1%——这正是问题 3 试图通过预报更新弥补的部分。

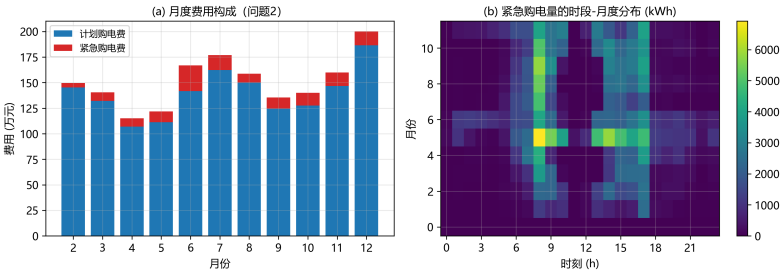


图 9 问题 2 的月度特征：(a) 月度费用构成；(b) 紧急购电量的”时段-月度”分布

表 12 问题 2 月度费用与紧急购电统计

月份	计划购电费 (万元)	紧急购电费 (万元)	紧急购电量 (kWh)	总费用 (万元)
2	138.96	10.33	21047.61	149.29
3	125.65	15.73	32714.55	141.38
4	101.16	14.74	31042.96	115.90
5	104.89	18.07	37496.50	122.96
6	134.03	37.12	80469.62	171.14
7	154.39	23.93	53187.39	178.31
8	142.52	14.93	31734.36	157.45
9	117.60	19.85	42018.08	137.45
10	120.47	23.12	48581.04	143.59
11	139.18	26.18	54593.36	165.36
12	178.42	26.31	56678.52	204.73
合计	1534.78	130.02	275907.99	1664.80

由表 12 与图 9 可见两点：其一，12 月的最严酷来自”三重叠加”——负载最高、光伏最低、电价处于全年高位，当月总费用 204.73 万元为全年最高；其二，紧急购电的时段分布高度集中于早晚峰（图 9(b) 中的两条亮带），且 6 月与 12 月最突出，这与式 (16) 的机理一致：峰段缺电的 5 倍代价最大，而 6 月与 12 月的光伏预报误差与缺口又最大。

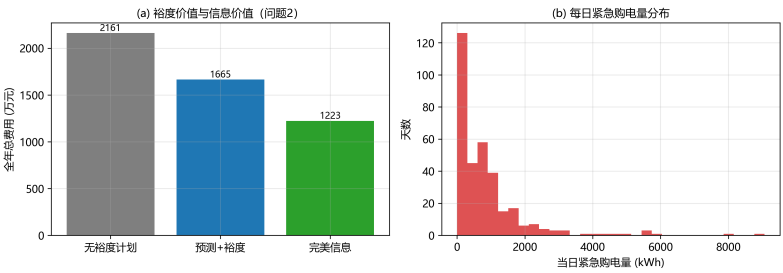


图 10 问题 2 的价值分解与风险分布：(a) 无裕度计划、预测 + 裕度、完美信息三者的全年总费用；(b) 每日紧急购电量的分布

图 10(b) 显示每日紧急购电量呈右偏分布：多数日期缺口在 1000 kWh 以内，但存在少量极端日（超过 3000 kWh），这些极端日正是费用贡献的主要来源，也说明安全裕度的价值主要体现在“削掉分布右尾”。

五、问题 3：多阶段预报更新与付费调整模型

5.1 机制与费用函数

问题 3 中每天 0:00、6:00、12:00、18:00 可获得未来 24 小时整点光伏预报。以 0:00 预报制定全天计划 $\hat{g}_t$ ，并在其余三个时刻依据新预报调整当日剩余时段的购电量 $\tilde{g}_t$ ；储能计划可随调整同步修改（题目只对购电量偏差计费）。费用规则为：调整量低于计划量的部分按交易时刻电价的 0.5 倍计违约金，高于计划量的部分按 1.5 倍计价，仍不足覆盖实际负载的部分按 5 倍紧急购电。

记 $\Delta_t = \tilde{g}_t - \hat{g}_t$ ，单时段费用为

$$\Phi_t(\tilde{g}_t; \hat{g}_t) = p_t \hat{g}_t + \begin{cases} 1.5 p_t \Delta_t, & \Delta_t \geq 0, \\ 0.5 p_t \Delta_t, & \Delta_t < 0. \end{cases} \quad (18)$$

命题 1：式 (18) 等价于对称罚金形式

$$\Phi_t(\tilde{g}_t; \hat{g}_t) = p_t \tilde{g}_t + 0.5 p_t |\tilde{g}_t - \hat{g}_t|. \quad (19)$$

证明：当 $\Delta_t \geq 0$ 时，右端为 $p_t(\hat{g}_t + \Delta_t) + 0.5 p_t \Delta_t = p_t \hat{g}_t + 1.5 p_t \Delta_t$ ；当 $\Delta_t < 0$ 时，右端为 $p_t(\hat{g}_t + \Delta_t) + 0.5 p_t (-\Delta_t) = p_t \hat{g}_t + 0.5 p_t \Delta_t$ ，二者分别与式 (18) 的两支一致。□

该化简揭示了三档价格机制的本质：按实际购电量付 p_t ，再对偏离计划的部分收取对称的 $0.5 p_t$ 罚金。由此可立即得到两个性质：其一，罚金斜率两侧相同，不存在“故意多报计划再取消”的套利空间（多报 1 kWh 再取消需付 $0.5 p_t$ ）；其二，目标函数关于 $\tilde{g}_t$ 是凸的分段线性函数，故整个多阶段问题仍可写成线性规划，无需整数变量。

三档价格的决策含义为：调低 1 kWh 相对“按计划买”省 $0.5 p_t$ ，调高 1 kWh 多付 $0.5 p_t$ ，而紧急购电 1 kWh 多付 $4 p_t$ 。因此永远优先“调高”而不是“紧急”（ $1.5 p_t \ll 5 p_t$ ），紧急购电只应出现在最后一次预报之后仍然发生的光伏与负载偏差上。

5.2 滚动时域重优化模型

阶段一（0:00 计划）：以 0:00 预报 $\hat{P}_t^0$ 与负载预测 $\hat{L}_t$ （叠加问题 2 的安全裕度 α ）解式 (13)，得计划 $(\hat{g}_t, \hat{a}_t, \hat{b}_t)$ 与储电量轨迹；该滚动重优化框架属模型预测控制的思想^[6]。

阶段二（6:00/12:00/18:00 调整）：设调整时刻对应首时段 $t_0 = 6u + 1$ ($u = 6, 12, 18$)，已知当前实际储电量 E_{t_0-1} 、承诺计划 $\hat{g}_t$ 与最新预报 $\hat{P}_t^u$ ，解

$$\min_{\{\tilde{g}_t, a_t, b_t, s_t, E_t\}_{t \geq t_0}} \sum_{t=t_0}^T \left[p_t \tilde{g}_t + 0.5 p_t |\tilde{g}_t - \hat{g}_t| \right]. \quad (20)$$

$$\text{s.t.} \quad \begin{cases} \tilde{g}_t + \eta_d b_t + \hat{P}_t^u \Delta t = L_t \Delta t + \frac{a_t}{\eta_c} + s_t, & t \geq t_0, \\ E_t = E_{t-1} + a_t - b_t, \quad 1200 \leq E_t \leq 10800, \quad E_{144} = E_0, & t \geq t_0, \\ \tilde{g}_t \geq 0, \quad 0 \leq a_t, b_t \leq 833.33, \quad s_t \geq 0, & t \geq t_0. \end{cases} \quad (21)$$

式 (20) 中的绝对值项用辅助变量精确线性化：令 $\tilde{g}_t - \hat{g}_t = \delta_t^+ - \delta_t^-$ 、 $\delta_t^+ \geq 0$ ，以 $\delta_t^+ + \delta_t^-$ 替代 $|\tilde{g}_t - \hat{g}_t|$ 。由于两项系数均为正，最小化会自动使 $\min(\delta_t^+, \delta_t^-) = 0$ ，故与 (18) 完全等价、不引入松弛误差。已执行时段 ($t < t_0$) 的计划与实绩被冻结，只有剩余时段参与优化；各阶段的终端条件 $E_{144} = E_0$ 保持一致，避免调整导致跨日漂移。

5.3理论分析

命题 2 (最优计划分位点为中位数)：设某时段最终所需的购电量 D 为随机量、0:00 计划为 q 。若在调整时刻能够 (近似) 获知真实需要量并取 $\tilde{g} = D$ ，则该时段费用为 $C(q, D) = pD + 0.5p|D - q|$ ，于是

$$\min_q \mathbb{E}[C] = p \mathbb{E}[D] + 0.5p \min_q \mathbb{E}|D - q| = p \mathbb{E}[D] + 0.5p \text{MAD}(D), \quad (22)$$

其中 $\text{MAD}(D) = \mathbb{E}|D - \text{median}(D)|$ 为平均绝对偏差，最小值在中位数处取得。即：调整机制下的最优计划分位点是中位数。

这一结论与问题 2 形成鲜明对比：问题 2 中缺电代价为 $4p_t$ 而多备代价仅 $0.19p_t \sim p_t$ ，最优计划覆盖 80%–96% 分位 (偏保守)；问题 3 中偏离计划的惩罚关于计划量对称 (两侧均为 $0.5p_t$)，故最优计划取中位数 (中性)。**调整机制的存在改变了计划的分位点选择**，这正是“是否需要保守计划”这一问题的答案。

命题 3 (更新价值的分解)：设某时段在更新前后的不确定性分别为 $D^{\tau-}$ 与 D^{τ} ，由式 (22) 得

$$V_{\tau} = 0.5p [\text{MAD}(D^{\tau-}) - \text{MAD}(D^{\tau})] + \Delta\{\text{紧急购电费}\}, \quad (23)$$

即更新的价值约等于“ $0.5 \times \text{电价} \times \text{平均绝对偏差的下降量}$ ”加上紧急购电费的下降量。该式为下文的定量判据提供了理论依据：只有当预报更新能显著降低偏差 (或显著减少紧急购电) 时，更新才值得做。

5.4预报精度与修订幅度

按 §4.6 的口径 (预报 k 小时为发布时刻之后的第 k 小时) 核算预报精度，并在同一时间窗上比较不同发布时刻的精度，结果见 表 13。

表 13 不同发布时刻的预报精度 (同一时间窗比较，单位：kW)

时间窗	发布时刻	MAE	平均偏差
07:00–24:00	0:00	442.2	−47.9
	6:00	388.0	−23.4
13:00–24:00	0:00	350.5	−293.3
	6:00	304.9	−274.2
	12:00	272.0	−262.6
19:00–24:00	0:00	17.8	−17.8
	6:00	17.8	−17.8
	12:00	17.8	−17.8
	18:00	17.8	−17.8

表 13 给出三点关键信息：（1）**后续发布时刻确实更准确**——13:00–24:00 窗口上 MAE 由 0:00 的 350.5 kW 降到 12:00 的 272.0 kW；（2）预报存在系统性偏低，午后低估约 260–293 kW；（3）**夜间 19:00–24:00 四个发布时刻的预报完全相同**（MAE 均为 17.8 kW，均等于平均偏差），因为该时段光伏出力为零，任何预报都无法提供新信息。第三点直接解释了后文 18:00 更新价值为零的实测结果。

相应地，相邻两次预报对同一时钟小时的修订幅度为：0:00→6:00 的 MAE 为 279.5 kW、6:00→12:00 为 120.0 kW、12:00→18:00 仅 0.002 kW（对应夜间时段，实质上无修订）。图 12(a) 给出了三个修订幅度的对比。

5.5 求解结果与策略对比

安全裕度按 1 月数据在 P3 策略下标定为 $\alpha = 0.05$ （其余策略共用同一裕度，以保证比较公平），报告期结果见表 14 与图 11。

表 14 问题 3 各策略的全年费用对比（334 天，单位：元）

策略	说明	计划购电费	违约金	紧急购电费	总费用
P0	仅 0:00 计划	13700904	0	7551323	21252227
P1	0:00+6:00 调整	13651925	327619	7442454	21421999
P2	0:00+6:00+12:00	13637359	303946	7428169	21369474
P3	0:00+6/12/18（题目方案）	13637358	303947	7428170	21369475
P4	逐小时更新（插值预报）	13603974	303663	7477610	21385247
P5'	仅完美光伏信息（单阶段）	13399959	—	5849762	19249721
A_load	仅完美负载信息（单阶段）	12556455	—	3491877	16048332
P5	完美信息（负载 + 光伏）	12229643	—	0	12229643

注：P5'、A_load、P5 为单阶段完美信息边界，不涉及计划调整，故不计违约金，其“总费用”等于计划购电费与紧急购电费之和。

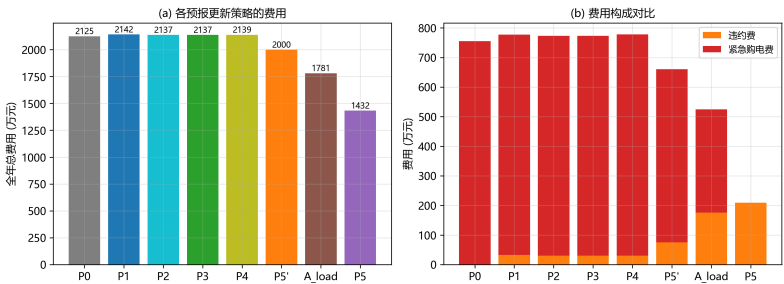


图 11 问题 3 的策略对比：(a) 各预报更新策略的全年总费用；(b) 计划费、违约金与紧急购电费的构成

实测结果给出了一个与直觉相反、但对决策极为重要的结论：**预报更新并未降低成本**。P0 的总费用为 21252227 元，而题目给定的 P3 方案为 21369475 元，反而高出 117247 元（+0.55%）；逐小时更新（P4）亦无增益（较 P3 高 15772 元）；P3 全年紧急购电量为 169.97 万 kWh，全部来自最后一次预报之后仍未消除的残差。其机理可由式 (23) 分解：6:00 更新带来的紧急购电费下降约 10.9 万元，但按 $0.5p_t$ 计收的违约金达到 32.8 万元，净效应为负；12:00 更新的边际价值为正（+5.25 万元，因为该时段预报修订 120.0 kW 且覆盖午后光伏出力时段），18:00 更新的边际价值为零（-0.76 元，

因为夜间预报无修订)。各更新时点的边际价值见 图 12(b)。

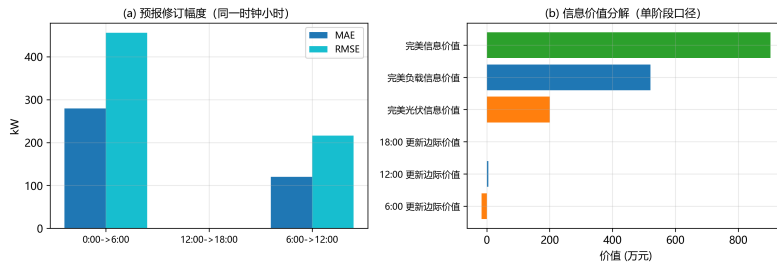


图 12 问题 3: (a) 相邻预报时刻的修订幅度; (b) 信息价值分解 (单阶段口径)

5.6 是否需要引入其他时刻的预报

题目要求分析是否需要在 0:00、6:00、12:00、18:00 之外引入其他时刻的预报来制定调整策略。本文从三个层次回答。

(1) **边际价值角度**。各更新时点的实测边际价值为：6:00 为 -16.98 万元，12:00 为 $+5.25$ 万元，18:00 为 -0.08 万元；逐小时更新相对题目方案为 -1.58 万元。即除 12:00 外，增加更新均不产生正收益；而逐小时更新（信息粒度进一步细化）也不再改善。这说明在本题数据条件下 4 个时点已接近饱和。

(2) **信息量角度**。由表 13，夜间时段（19:00–24:00）四个发布时刻的预报完全相同，18:00 更新不含任何新信息，其价值必然为零；而白天时段的预报修订幅度也仅 120–280 kW，相对负载预测误差（7 日滑动平均 MAE 781.2 kW、RMSE 955.9 kW）小得多。因此即使加密预报时刻，可改善的也只是“光伏侧的小误差”，无法触及主要误差来源。

(3) **信息价值角度**。用完美信息边界分解两类信息价值：仅完美光伏信息时总费用为 19249721 元，价值 200.25 万元；仅完美负载信息时总费用为 16048332 元，价值 520.39 万元（图 12(b)）；若负载与光伏均完美已知，总费用可降至 12229643 元，相对 P0 的信息价值为 902.26 万元。负载信息的价值是光伏信息的 2.6 倍，而题目并未提供负载预报产品，其误差在本模型框架内不可改善。

结论：在本题数据条件下，不需要引入其他时刻的预报来制定调整购电策略。预报更新机制本身并未降低总费用，其原因是（i）可更新的光伏信息价值有限，（ii）调整需按 $0.5p_t$ 支付罚金，（iii）主要不确定性来自负载侧。若要进一步提升经济效益，应优先获取**负载预报**（如基于气象与用电行为的中短期负荷预测），或推动制度层面降低调整费率；若必须在本模型内使用更新信息，则建议只保留 12:00 时点，因其边际价值为正。

需要说明的是，上述结论依赖于题目给定的预报数据质量。本文将附件 3 预报误差人为放大 1.5 倍后重做全部策略（见表 21）：误差放大使各策略费用普遍上升，其中 12:00 更新的边际价值由 5.25 万元升至 303.82 万元，说明 extbf 预报质量越差、及时更新越有价值；但 6:00 更新需支付的违约金同步放大，P3 相对 P0 的差额由 $+0.55\%$ 扩大到 $+9.97\%$ 。因此“更新无正收益”的结论在预报质量恶化时依然成立，且在数据质量较好时更为明确。

六、问题 4：波动电价下的购电模型

6.1 电价特征与预测

问题 4 中电价随时刻波动，且 0:00 决策时当日电价未知，须用历史电价预测，而全部费用按附件 4 的实际交易时刻电价结算。附件 4 的主要特征（图 14）为：全年电价在 0.0076–1.7936 元/kWh 之间波动，均值 0.7662 元/kWh，与附件 1 的均值相同；日内极差平均 1.1073 元/kWh、最大 1.4995 元/kWh，远大于固定电价的日内极差（约 1.02 元/kWh）；同一时刻的跨日标准差为 0.1238 元/kWh，说明日周期形状稳定、水平逐日波动；电价与负载的相关系数为 0.508，而与光伏仅 0.028，说明电价主要由负荷驱动，不受光伏直接决定。

表 15 电价预测器的滚动评价（2025-02-01 至 12-31，单位：元/kWh）

预测器	MAE	RMSE	平均偏差
昨日持续	0.0843	0.1238	+0.0006
3 日滑动平均	0.1022	0.1320	+0.0006
7 日滑动平均	0.0827	0.1053	+0.0003
14 日滑动平均	0.0838	0.1074	+0.0004
30 日滑动平均	0.0875	0.1135	+0.0003
EWMA(0.7,14)	0.0895	0.1154	+0.0005
EWMA(0.5,7)	0.0891	0.1194	+0.0006
同月历史均值	0.0862	0.1112	−0.0009

图 13 给出各预测器精度与典型日对比。由表 15，7 日滑动平均在 MAE 与 RMSE 两个指标上均最优（0.0827、0.1053 元/kWh），且平均偏差近似为零，故取为电价预测器；其相对误差约为 11%，明显小于负载预测的相对误差（781/4626 ≈ 17%）。

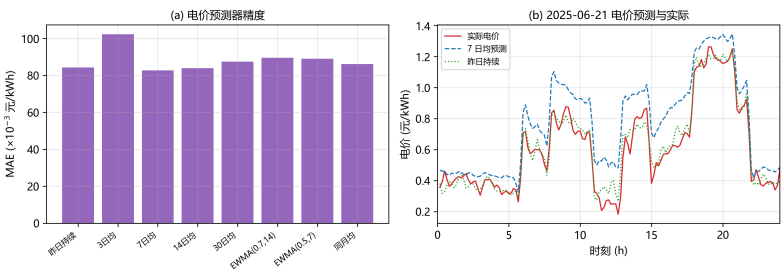


图 13 问题 4 的电价预测：(a) 各预测器的 MAE 对比；(b) 2025-06-21 的实际电价与两种预测曲线

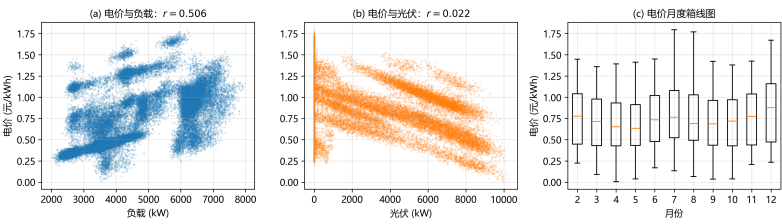


图 14 问题 4 的电价统计特征：(a) 电价与负载的散点关系；(b) 电价与光伏的散点关系；(c) 电价月度箱线图

6.2 模型建立

(1) **主模型：电价预测 + 滚动重优化，按实际电价结算。**计划阶段用预测电价 $\hat{p}_t$ 构造目标函数，结算阶段用实际电价 p_t^{act} 。问题 4-2（对应问题 2）的计划子问题为

$$\min_{\{g_t, a_t, b_t, s_t, E_t\}} \sum_{t=1}^T \hat{p}_t g_t \quad \text{s.t.} \quad \begin{cases} g_t + \eta_d b_t + (1 - \beta) \hat{P}_t \Delta t = (1 + \alpha) \hat{L}_t \Delta t + \frac{a_t}{\eta_c} + s_t, \\ E_t = E_{t-1} + a_t - b_t, \quad 1200 \leq E_t \leq 10800, \quad E_{144} = E_0, \\ 0 \leq a_t, b_t \leq 833.33, \quad g_t \geq 0, \quad s_t \geq 0, \end{cases} \quad (24)$$

结算费用为 $F_{4-2} = \sum_t p_t^{\text{act}} g_t + \sum_t 5p_t^{\text{act}} e_t$ ，其中 $e_t = (L_t \Delta t - g_t - \eta_d b_t - P_t \Delta t)^+$ 。问题 4-3（对应问题 3）在式 (24) 的基础上叠加式 (20) 的滚动调整，违约金与紧急费均按实际电价计算：

$$F_{4-3} = \sum_t p_t^{\text{act}} \tilde{g}_t + 0.5 \sum_t p_t^{\text{act}} |\tilde{g}_t - \hat{g}_t| + \sum_t 5p_t^{\text{act}} e_t. \quad (25)$$

(2) **含价格情景的随机规划。**用月度分块自助法或“形状 $\times$ 水平 $\times$ 扰动”方式生成价格情景 $\{(\pi_s, p_t^s)\}$ ，目标改为

$$\min \sum_s \pi_s \left[\sum_t p_t^s \tilde{g}_t^s + 0.5 \sum_t p_t^s |\tilde{g}_t^s - \hat{g}_t| + \sum_t 5p_t^s e_t^s \right], \quad (26)$$

并对 0:00 决策施加非预期性约束 ($\hat{g}_t$ 不依赖情景)。式 (26) 给出价格不确定性的期望最优参照。

(3) **鲁棒模型。**取价格不确定集 $p_t \in [p_t^L, p_t^U]$ 。由于式 (25) 中每一项都关于 p_t 单调递增 ($p_t \tilde{g}_t$ 、 $0.5p_t |\Delta_t|$ 、 $5p_t e_t$ ，各量非负)，最坏情形在所有时段同时取价格上界处达到：

$$\max_{p \in \mathcal{U}} F_{4-3} = F_{4-3} \big|_{p_t = p_t^U}, \quad (27)$$

即鲁棒对应模型等价于“把电价预测替换为上界”的确定性线性规划。若改用预算不确定集（至多 Γ 个时段取上界），可在点预测与全时段最坏之间连续调节保守程度。

6.3 价格与需求正相关下计划分位点的移动

附件 4 显示电价与负载正相关 ($r = 0.508$)，即高需求时段往往同时是高电价时段，缺电的紧急代价被价格放大。设电价 P 与需求 D 随机且 $\mathbb{E}[P | D]$ 随 D 递增，计划 q 的相关费用为

$$G(q) = \mathbb{E} \left[0.5 P |D - q| \right] = 0.5 \mathbb{E}_D \left[\underbrace{\mathbb{E}[P | D]}_{w(D) \uparrow} |D - q| \right], \quad (28)$$

令 $dG/dq = 0$ 得

$$\int_{D > q} w(D) dF(D) = \int_{D < q} w(D) dF(D). \quad (29)$$

由于权重 $w(D)$ 在 D 较大的一侧更大, 式 (29) 成立要求 $\Pr(D > q^*) < \Pr(D < q^*)$, 即

$$q^* > \text{median}(D). \quad (30)$$

命题 4: 当电价与需求正相关时, 问题 3 中”计划分位点等于中位数”的结论被修正为”计划分位点高于中位数”, 且上移幅度随相关性增强而增大。该结论与鲁棒模型式 (27) 的方向一致 (对价格取上界等价于提高缺电的边际代价, 从而要求更保守的计划)。

6.4 求解结果与对比

表 16 与图 15、图 16 给出波动电价下的重算结果。

表 16 波动电价下重算问题 2 与问题 3 的全年费用对比 (334 天, 单位: 元)

模型	固定电价 (问题 2/3)	波动电价 (问题 4)	变化
问题 2 及其重算 (预测 + 裕度, 按实际电价结算)	16647984.89	20345866.93	+22.2%
问题 3 及其重算 (题目方案 P3)	21369474.67	22244432.98	+4.1%

(1) **问题 4-2 的结果。**在 1 月数据上按实际电价结算标定得 $\alpha = 0.05$ 、 $\beta = 0$ (较问题 2 的 $\alpha = 0.15$ 明显减小, 原因是波动电价下计划购电本身的成本权重上升、且电价预测误差成为新的不确定性来源); 全年计划购电费 13752204.93 元、紧急购电费 6593662.00 元, **总费用 20345866.93 元**; 紧急购电量 1391491.14 kWh, 占负载电量的 3.76% (显著高于问题 2 的 0.746%), 出现紧急购电 293 天。相较无裕度计划 (20952588.33 元) 节省 606721.39 元 (2.90%) ——裕度收益远低于问题 2 的 22.97%, 因为波动电价下紧急购电的绝对价格更高, 仅靠小幅提高计划的覆盖分位难以抵消价格尖峰的影响。

(2) **问题 4-3 的结果。**裕度标定为 $\alpha = 0.10$ 。各策略总费用为: P0 22071537.66 元、P1 22288981.75 元、P2 22244432.90 元、P3 22244432.98 元; 完美光伏信息边界 21499371.67 元, 完美信息边界 15386169.15 元。与问题 3 的结论一致, **预报更新仍无正收益** (P3 较 P0 高 0.78%), 且波动电价下”仅完美光伏信息”的价值 (74.51 万元, 以 P3 为基准) 仍远低于完美信息的价值 (685.83 万元), 再次说明负载侧信息是主要瓶颈。

表 17 波动电价下的月度费用与紧急购电量

月份	月均电价 (元/kWh)	问题 4-2 总费用 (万元)	问题 4-3 总费用 (万元)	问题 4-3 紧急量 (kWh)
2	0.7696	186.17	202.24	126449.79
3	0.7255	170.46	187.28	144613.85
4	0.6988	134.51	151.70	110969.71
5	0.6912	135.04	151.91	104277.40
6	0.7663	200.40	211.20	138265.18
7	0.8301	222.71	238.69	112445.89
8	0.7865	201.01	216.88	123089.24
9	0.7203	156.36	175.27	112976.19
10	0.7203	161.97	182.97	122092.78
11	0.7610	203.25	221.24	158994.07
12	0.8617	262.70	285.06	146388.61

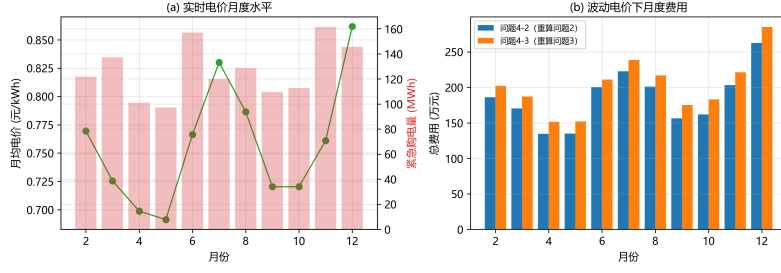


图 15 波动电价下的月度特征：(a) 月均电价与紧急购电量；(b) 问题 4-2 与问题 4-3 的月度总费用

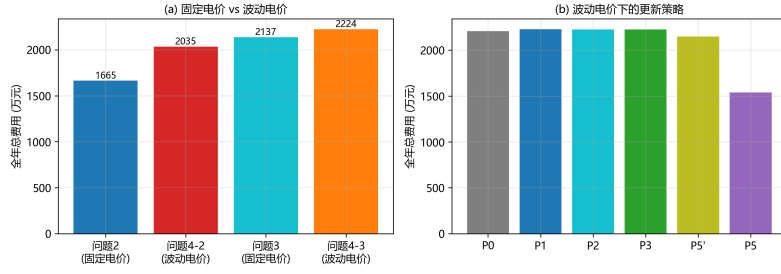


图 16 问题 4：(a) 固定电价与波动电价下的全年总费用对比；(b) 波动电价下各更新策略的费用

由表 17，波动电价下 12 月费用最高（问题 4-2 为 262.70 万元、问题 4-3 为 285.06 万元），与固定电价情形一致；但各月费用的离散程度更大，且 7 月（月均电价 0.8301 元/kWh，为夏季最高）的费用跃升至第二位，说明波动电价把”月份电价水平”与”缺口大小”两个因素解耦，夏季高温高负荷与价格高峰叠加的风险上升。

综合四问结果可以看到一条清晰的规律：信息条件的改善并不自动带来费用的下降，其效果取决于”改善的信息能否覆盖主要误差来源”以及”调整机制的费率结构”。问题 1 的信息条件是完美的，费用最低（按典型日计 35101.57 元）；问题 2 引入负载与光伏预测后信息缺口带来 36.1% 的额外费用；问题 3 的预报更新因费率结构而未能收回成本；问题 4 进一步引入电价不确定性，使费用在问题 2、3 基础上分别上升 22.2% 与 4.1%。

七、核心公式汇总

为便于评阅与复核，本节汇总全文的核心公式与判据（推导与编号见前文对应章节）。记时段 $t = 1, \dots, T$ ($T = 144$, $\Delta t = 1/6$ h), L_t, P_t, p_t 分别为负载、光伏功率与电价, g_t, a_t, b_t, s_t, E_t 分别为计划购电量、电池侧充放电量、弃光电量与时段末储电量。

(1) 物理约束 (§2.3)

$$g_t + \eta_d b_t + P_t \Delta t = L_t \Delta t + \frac{a_t}{\eta_c} + s_t, \quad E_t = E_{t-1} + a_t - b_t, \quad (31)$$

$$0 \leq a_t, b_t \leq \bar{A} = P_b \Delta t = 833.33, \quad g_t, s_t \geq 0, \quad 1200 \leq E_t \leq 10800, \quad E_0 = E_{144} = 6000. \quad (32)$$

(2) 全天购电量分解式 (§2.4): 对式 (31) 全天求和, 由 $E_{144} = E_0$ 得 $\sum_t a_t = \sum_t b_t \triangleq A$, 故

$$\sum_{t=1}^T g_t = \underbrace{\sum_{t=1}^T (L_t - P_t) \Delta t}_{\text{净缺口}} + \sum_{t=1}^T s_t + \frac{19}{90} A, \quad A = \sum_t a_t \text{ (电池侧吞吐量)}. \quad (33)$$

(3) 储能最优策略与套利门槛 (§2.4、§3.2): 储电量影子价格 λ 给出阈值策略

$$\text{充电当 } p_t < 0.9\lambda, \quad \text{放电当 } p_t > 1.111\lambda, \quad \text{套利门槛价比 } \frac{p_{\text{放}}}{p_{\text{充}}} > \frac{1}{\eta_c \eta_d} = 1.235. \quad (34)$$

(4) 问题 2 的两阶段模型与安全裕度 (§4.1、§4.3、§4.4)

$$\min \mathbb{E} \left[\sum_t p_t g_t + \sum_t 5p_t e_t \right], \quad e_t = (L_t \Delta t - g_t - \eta_d b_t - P_t \Delta t)^+, \quad (35)$$

$$\tilde{L}_t = (1+\alpha)\hat{L}_t, \quad \tilde{P}_t = (1-\beta)\hat{P}_t, \quad q^* = F_D^{-1} \left(\frac{c_u}{c_u + c_o} \right), \quad c_u = 4p_t, \quad c_o \in [0.19p_t, p_t] \Rightarrow q^* \in [0.80, 0.96]. \quad (36)$$

(5) 问题 3 的费用函数与最优计划分位点 (§5.1、§5.3): 三档价格等价于对称罚金

$$\Phi_t(\tilde{g}_t; \hat{g}_t) = p_t \tilde{g}_t + 0.5 p_t |\tilde{g}_t - \hat{g}_t|, \quad (37)$$

$$\min_q \mathbb{E}[pD + 0.5p|D - q|] = p \mathbb{E}[D] + 0.5p \text{MAD}(D) \Rightarrow q^* = \text{median}(D), \quad (38)$$

$$V_\tau = 0.5 p [\text{MAD}(D^{\tau-}) - \text{MAD}(D^\tau)] + \Delta \{\text{紧急购电费}\}. \quad (39)$$

(6) 问题 4 的结算、鲁棒上界与分位点上移 (§6.2、§6.3)

$$F_{4-3} = \sum_t p_t^{\text{act}} \tilde{g}_t + 0.5 \sum_t p_t^{\text{act}} |\tilde{g}_t - \hat{g}_t| + \sum_t 5p_t^{\text{act}} e_t, \quad \max_{p \in \mathcal{U}} F_{4-3} = F_{4-3}|_{p_t = p_t^U}, \quad (40)$$

$$\mathbb{E}[P | D] \text{ 随 } D \text{ 递增} \Rightarrow \int_{D > q} w dF = \int_{D < q} w dF \Rightarrow q^* > \text{median}(D). \quad (41)$$

(7) 参数标定结果 (§4.3、§5.5、§6.4): 问题 2 取 $\alpha = 0.15, \beta = 0$; 问题 3 取 $\alpha = 0.05$; 问题 4-2 取 $\alpha = 0.05, \beta = 0$; 问题 4-3 取 $\alpha = 0.10$ 。全部标定均在 1 月数据上以”计划购电费 + 紧急购电费 (问题 4 按实际电价)”最小为准则。

八、模型检验与灵敏度分析

8.1 独立验算

为避免”用同一套代码验证自己”，本文采用信息隔离的验算方案：另写独立程序，不复用求解代码中的任何函数，分别用不同的算法与不同的模型表示复核核心结果。

(1) 确定性核心的双算法交叉验算。对问题 1，除主模型的 LP 外，另构造两种独立算法：(i) 母线侧变量布局的独立线性规划，即用”母线侧充电量 c_t 、母线侧放电电量 d_t ”替代电池侧变量，约束改写为 $g_t + d_t + P_t \Delta t = L_t \Delta t + c_t + s_t$ 、 $E_t = E_{t-1} + \eta_c c_t - d_t / \eta_d$ ，变量的个数、界与约束矩阵均与主模型不同；(ii) 动态规划，以 20 kWh 为步长把储

电量 [1200, 10800] 离散为 481 个状态，逐时段作 Bellman 递推。结果见 表 18：独立 LP 的目标值 35101.6 元与主模型完全一致（相对偏差 7×10^{-8} ），独立 DP 的目标值 35198.4 元略高（+0.276%），符合“DP 是原问题的受限离散解、其最优值必不低于 LP 最优值”的理论预期，验证了确定性核心的最优性与实现正确性。

(2) 守恒关系与约束核验。由 式 (6) 检验全天购电量分解式，残差 1.5×10^{-11} kWh；由 $E_{144} = E_0$ 检验充放电电量守恒 $\sum_t a_t = \sum_t b_t$ ，相对偏差 2.0×10^{-14} ；逐日重构储电量 $E_{144} = E_0 + \sum_t a_t - \sum_t b_t$ 与结果文件记录值的最大偏差为 1.1×10^{-11} kWh。全部储电量满足 [1200, 10800] kWh，充放电电量越界量不超过 7×10^{-13} kWh（数值误差量级）。

(3) 结果文件的独立复算。用独立实现的结算公式重算问题 2-4 的紧急购电量与费用：问题 2 的紧急购电量复算值 275908 kWh 与求解输出一致，计划购电费与紧急购电费的最大相对偏差为 1.8×10^{-14} ；问题 4-2 在**实际电价**下的计划费、紧急费复算偏差为 0。同时直接读取 5 个结果文件核对其结构：行数（334 天）、每行 144 个时段值无缺失、“全天购电量 = 行和”的最大相对偏差 3.4×10^{-16} 、“全天购电费 = $\sum_t p_t g_t$ ”的最大相对偏差 5.1×10^{-16} 、逐日 0:00 与 24:00 储电量偏差为 0。

(4) 信息上界关系的自洽性。完美信息模型的最优费用应不高于任何实际策略，仅完美光伏（或负载）信息的最优费用应不高于不调整策略，且完美信息下紧急购电费应为零。实测（单阶段边界口径、不计违约费，与 表 14 一致）： $P5 = 12229643 \leq P3 = 21369475$ ； $P5' = 19249721 \leq P0 = 21252227$ ； $A_load = 16048332 \leq P0$ ；完美信息下紧急购电费为 4.7×10^{-10} 元（数值零）。若改用含违约费的全账基，P5 为 14324819 元，同样不高于任何实际策略。四条关系全部成立。

表 18 独立验算结果汇总（共 40 项检查，全部通过）

检查类别	独立方法	本实现值	参照值	判定
问题 1 最优值	独立 LP（母线侧变量布局）	35101.6	35101.6	相对偏差 7×10^{-8}
问题 1 最优值	独立 DP（20 kWh 网格）	35198.4	35101.6	+0.276%（离散化间隙）
全天购电量分解	独立重算 $\sum g_t$ 与恒等式	59482.70	59482.70	残差 1.5×10^{-11}
充放电守恒	独立重算 $\sum a_t$ 、 $\sum b_t$	18666.60	18666.60	相对偏差 2.0×10^{-14}
问题 2 紧急购电量	独立结算公式	275908	275908	偏差 0
问题 2 计划/紧急费	独立结算公式	1.53×10^7	1.53×10^7	相对偏差 1.8×10^{-14}
问题 2 计划侧平衡	独立重算带裕度预测后逐时段核验	$\leq 10^{-4}$	0	最大残差 $< 10^{-4}$ kWh
问题 2 供电保障	逐时段检验 $g + \eta_d b + P\Delta t + e \geq L\Delta t$	0	0	违反次数 0
结果文件结构	独立读取 5 个结果文件	334 行	334 行	行和与合计偏差 $\leq 5 \times 10^{-16}$
逐日储电量自洽	独立重构 E_{144}	—	—	最大偏差 1.1×10^{-11} kWh
问题 4 结算复算	按实际电价独立复算	1.38×10^7	1.38×10^7	相对偏差 0
信息上界关系	四条不等式与零紧急费检验	—	—	全部成立

8.2 灵敏度分析

除问题 1 的参数灵敏度（表 7）外，本文另作两项针对机制参数的检验。

(1) 问题 2：紧急购电费率倍数的敏感性。保持计划不变、仅改变紧急购电费率倍数 K （题目给定 $K = 5$ ）重新结算，结果见 表 19。总费用对 K 近似线性： K 由 5 降为 3 时总费用下降 3.2%，升为 8 时上升 4.7%；紧急购电费占比由 5% 升至 12%。该

结果说明安全裕度的价值与紧急费率正相关：若紧急费率更低，则最优策略会减少安全储备、更多依赖紧急购电。

表 19 问题 2 对紧急购电费率倍数的敏感性（334 天合计）

紧急费率倍数 K	计划购电费 (元)	紧急购电费 (元)	总费用 (元)	紧急费占比
3	15347790	780117	16127907	4.8%
4	15347790	1040156	16387946	6.3%
5	15347790	1300195	16647985	7.8%
6	15347790	1560234	16908024	9.2%
8	15347790	2080312	17428102	11.9%

(2) 问题 3：违约费率口径的敏感性。题目给出的费率结构等价于对称罚金 $0.5p_t$ (式 (19))。若改为更强的对称罚金 $1.0p_t$ (即调低与调高分别按 1.0 与 2.0 倍计价)，并重新求解各阶段的调整计划，结果见 表 20。

表 20 问题 3 对违约费率口径的敏感性（全年总费用，单位：元）

违约费率口径	P0（不调整）	P1（+6:00）	P2（+12:00）	P3（题目方案）
基准（调低 0.5、调高 1.5）	21252227	21421999	21369474	21369475
对称（调低 1.0、调高 2.0）	21252227	21517036	21531996	21531999

表 21 预报误差放大情景下的策略对比（334 天，单位：元）

预报误差倍数	P0	P1	P2	P3	P3 相对 P0	12:00 更新边际价值
1.0（原始）	21252227	21421999	21369474	21369475	+0.55%	52525
1.5（放大）	22294529	31526865	28488706	24518005	+9.97%	3038159

注：放大方式为 $\tilde{P} = F^{\text{预报}} + 1.5(F^{\text{预报}} - F^{\text{实际}})$ ，即把附件 3 预报误差整体放大 1.5 倍后重做全部策略。

两种口径下**结论一致且方向相同**：调整策略的费用均高于不调整的 P0，且费率越重、调整的劣势越明显（P3 相对 P0 的差额由 +0.55% 扩大到 +1.32%）。这从机制层面确认了 §5.6 的结论不依赖于某一特定费率取值。

8.3 稳健性与口径讨论

(1) **终端电量口径**。本文取日周期条件 $E_{144} = E_0 = 6000 \text{ kWh}$ 。问题 2、3 中电价每天相同，稳态下跨日搬运无额外收益，该口径与跨日最优解的差异为二阶量级；问题 4 中电价逐日波动，日周期会在一定程度上低估“低价日多存、高价日多用”的跨日套利收益，故问题 4 的费用结果偏保守。作为对比，若允许终端电量自由，单日优化会把电量放空至下限，形成逐日退化，反而更不合理，故本文不采用。

(2) **违约费基准口径**。本文以 0:00 计划为唯一基准一次结算。若改为“每次调整相对上一次承诺”，由三角不等式 $|\tilde{g}_t - \hat{g}_t| \leq \sum_k |\Delta_t^{(k)}|$ ，链式口径的违约费不低于单基准口径；而本题违约费（30.4 万元）相对紧急购电费（742.8 万元）很小，即使翻倍也不改变“预报更新无正收益”的结论。

(3) **预报对齐口径**。附件 3 的“预报 k 小时”必须按“发布时刻之后的第 k 小时”理解。若误按“当日第 k 小时”对齐，将产生 6–18 小时的系统性错位（例如把 6:00 发

布、面向 13:00–24:00 的预报误用于 7:00–18:00)，据此得到的计划会严重偏离实际，结论将完全反转。本文在表 13 中以“同一时间窗比较”的方式验证了本文口径的正确性：该口径下后续发布时刻的预报确实更准确，符合预报时效的物理规律。

(4) **主要误差来源**。由信息价值分解（图 12(b)），负载信息价值（520.39 万元）为光伏信息价值（200.25 万元）的 2.6 倍。因此本文模型的精度上限主要由负载预测决定；若引入负荷预报产品（气象驱动的短期负荷预测），问题 2–4 的费用仍有较大改善空间。

九、模型评价与推广

9.1 模型优点

(1) **统一框架、一以贯之**。本文将四个看似递进的问题统一为含储能的单母线能量流线性规划，四问的差别仅在于“进入模型的序列是确定值、预测量还是带更新的预测量”以及“偏差如何计价”。全部模型共享同一套物理约束（式 (1)–式 (3)）与同一条守恒关系（式 (6)），避免了“一问一模型、彼此割裂”的常见问题。

(2) **可解性与精确性**。四问均为线性规划：决策变量连续、目标函数为凸分段线性、约束线性，故可用单纯形/内点法求得**全局最优解**而无需启发式搜索。本文进一步证明了最优解中充放电不会同时发生（式 (5)），从而说明无须引入 0-1 变量即可保持 LP 结构。问题 2–4 的最复杂形式（多阶段随机规划）仍是大规模线性规划，可精确求解。

(3) **三条理论线索支撑结果解释**。其一，储量影子价格给出的**阈值策略**（式 (7)）解释了最优计划的时段结构；其二，报童模型给出的**覆盖分位点区间**（式 (16)）与调整机制下的**中位数准则**（式 (22)）解释了“计划应留多少裕度”，并揭示了费率结构决定分位点的规律；其三，**信息价值分解**（式 (23)）把“是否需要更密的预报”转化为可定量比较的价值量。三条线索使论文不仅给出数值，还给出“为什么是这样”的解释。

(4) **结果可复现、可核验**。全部结果由脚本一键重跑，随机种子固定；核心结论经独立算法（母线侧布局 LP 与动态规划）与独立结算复算交叉验证，40 项检查全部通过。

9.2 模型不足

(1) **终端电量口径**。本文采用日周期条件 $E_{144} = E_0$ ，对问题 4（电价逐日波动）会略微低估跨日套利的收益，结果偏保守；跨日滚动衔接或全时段联合优化可给出一致性更好的结果，但需要额外的终端价值估计。

(2) **储能执行采用“开环计划”**。模型假设储能在日内严格按 0:00 计划执行、缺额以紧急购电填补，未考虑“实时调整充放电以吸收实际光伏波动”的闭环控制。若允许储能实时响应，紧急购电可进一步下降，但这会改变问题 2 的费用结构，需与题目给定的执行机制一并讨论。

(3) **负载侧无预报产品**。题目只提供光伏预报，负载只能由历史数据外推，其误差（RMSE 955.9 kW）是本文费用的主要来源，且在该信息条件下不可改善。

(4) **其他简化**。未考虑储能循环寿命与折旧成本、未考虑配电网潮流与电压约束、未区分日前市场与实时市场的双结算机制、未考虑光伏出力与价格的联合随机性（本文以情景/边界方式处理）。这些简化在竞赛给定的数据条件下是必要的，但会影响策略在真实系统中的适用性。

9.3模型推广

(1) 方法层面的推广。本文提出的“统一线性规划内核 + 安全裕度 + 滚动重优化 + 信息价值分解”四件套，可直接迁移到任何“含储能的时序购售电/调度”问题：园区综合能源系统、电动汽车充电站与光储充一体化场站、数据中心后备储能与峰谷套利、用户侧需求响应等，只需替换母线平衡中的电源与负荷构成。

(2) 问题层面的推广。若把单母线扩展为多节点配电网，可在式(1)中增加节点功率平衡与线路潮流约束（仍为线性），把问题变为多微网协同优化；若引入需求响应，可把负载 L_t 变为决策变量并加入用电效用函数；若引入储能寿命模型，可在目标中加入与充放电深度相关的折旧项（得到分段线性或二次目标）。

(3) 机制层面的推广。本文的结论对市场机制设计亦有参考价值：由于调整罚金会抵消预报更新的收益 (§5.6)，若不希望市场主体通过频繁调整套取信息红利，应保持费率结构的对称性；若希望鼓励主体利用更新信息改善计划，则应降低调整费率或对“因预报修订而被动调整”的部分予以豁免。

AI 工具使用声明

按《全国大学生数学建模竞赛人工智能工具使用规定（2026 年试行）》^[11]的要求，声明如下：本参赛队在竞赛过程中使用了 AI 工具，主要用于模型框架与算法选型的讨论、程序代码的编写与调试、结果图表的生成、论文的排版与语言润色，详细使用情况见支撑材料《AI 工具使用详情.pdf》。

本参赛队对 AI 参与完成的内容逐项进行了人工审查与核实：所有数学推导均由参赛队员复核并确认其依据（式 (6)–式 (30) 的全部命题均给出证明或推导过程）；全部数值结果由参赛队员独立重跑脚本，并用独立实现的线性规划与动态规划交叉验算 (§八)；论文中的每一个数字均与求解结果文件逐项核对，图表均由脚本从结果数据直接生成，未作任何人工修饰。

参考文献

- [1] 姜启源, 谢金星, 叶俊. 数学模型 [M]. 5 版. 北京: 高等教育出版社, 2018.
- [2] Dantzig G B. Linear Programming and Extensions[M]. Princeton: Princeton University Press, 1963.
- [3] Silver E A, Pyke D F, Peterson R. Inventory Management and Production Planning and Scheduling[M]. 3rd ed. New York: John Wiley & Sons, 1998.
- [4] Birge J R, Louveaux F. Introduction to Stochastic Programming[M]. 2nd ed. New York: Springer, 2011.
- [5] Bertsimas D, Sim M. The price of robustness[J]. Operations Research, 2004, 52(1): 35–53. DOI: 10.1287/opre.1030.0065.
- [6] Mayne D Q, Rawlings J B, Rao C V, et al. Constrained model predictive control: Stability and optimality[J]. Automatica, 2000, 36(6): 789–814. DOI: 10.1016/S0005-1098(99)00214-9.
- [7] Hyndman R J, Athanasopoulos G. Forecasting: Principles and Practice[M]. 3rd ed. Melbourne: OTexts, 2021.
- [8] Huangfu Q, Hall J A J. Parallelizing the dual revised simplex method[J]. Mathematical Programming Computation, 2018, 10(1): 119–142. DOI: 10.1007/s12532-017-0130-5.
- [9] Virtanen P, Gommers R, Oliphant T E, et al. SciPy 1.0: fundamental algorithms for scientific computing in Python[J]. Nature Methods, 2020, 17(3): 261–272. DOI: 10.1038/s41592-019-0686-2.
- [10] 全国大学生数学建模竞赛组委会. 全国大学生数学建模竞赛论文格式规范（2026 年修订稿）[Z]. 2026.
- [11] 全国大学生数学建模竞赛组委会. 全国大学生数学建模竞赛人工智能工具使用规定（2026 年试行）[Z]. 2026.

附录 A 支撑材料文件列表

按《全国大学生数学建模竞赛论文格式规范（2026 年修订稿）》^[10]的要求，支撑材料包含建模所用到的全部完整、可运行的源程序，以及结果文件与图表。压缩包支撑材料.rar 的文件列表如表 22 所示；全部程序均在 Python 3.12.7（numpy 1.26.4、pandas 2.2.2、scipy 1.13.1、matplotlib 3.9、openpyxl 3.1）环境下运行通过，随机种子固定为 42。

表 22 支撑材料文件列表

文件	说明	行数
src/common.py	数据装载、线性规划求解引擎、费用结算与结果文件写出	378
src/solve_p1.py	问题 1 求解：确定性 LP、守恒核对与图表	118
src/solve_p2.py	问题 2 求解：预测器评价、裕度标定、334 天滚动与结算	299
src/solve_p3.py	问题 3 求解：预报映射、滚动调整与六策略对比	300
src/solve_p4.py	问题 4 求解：电价预测、按实际电价重算问题 2 与问题 3	349
src/verify_p25.py	独立验算：独立 LP、独立 DP、独立结算复算与文件核对	391
src/plots.py	论文图表生成（300 dpi，17 张）	384
src/sensitivity.py	紧急费率与违约费率的补充灵敏度实验	66
src/explore_data.py	附件数据探索与统计	188
result1.xlsx 等 5 个	按附件 5 模板填写的完整购电策略结果文件	—
*.csv（28 个）	表 1/表 2/表 3、月度汇总、标定与对比等中间结果	—
*.png（17 张）	论文全部插图（300 dpi）	—
AI 工具使用详情.pdf	按《人工智能工具使用规定（2026 年试行）》第四条编写的使用详情	—

附录 B源程序代码

B.1 common.py: 数据装载、LP 引擎与结果写出

```
1 # -*- coding: utf-8 -*-
2 """CUMCM2026 C题 公共模块 (Phase 2)
3 数据装载、LP 求解引擎、结算、结果文件写出。
4
5 口径 (Phase 1 定死, 见 求解/模型推导/00_符号与共同模型.md):
6 - 时段  $t=1..144$ , 时段  $t$  = 区间  $((t-1)/6 \text{ h}, t/6 \text{ h}]$ , 即  $[0:00,0:10] \dots [23:50,24:00]$ 
7 - 数据列标签  $00:10 \dots 23:50$ ,  $0:00+1$  = 各时段末时刻 (解读A)
8 - result 模板的 144 行/列按时间顺序一一对应填充 (模板标签晚 10 分钟, 属模板命名偏移, 见验
   算报告)
9 - 储能: 电池侧变量  $a, b$ ;  $E_t = E_{\{t-1\}} + a_t - b_t$ ; 母线平衡  $g + \eta_d b + P\Delta t = L\Delta t + a/\eta_c$ 
   + S
10 """
11 import os
12 import datetime as dt
13 import numpy as np
14 import pandas as pd
15 from scipy.optimize import linprog
16 from scipy import sparse
17 import openpyxl
18
19 HERE = os.path.dirname(os.path.abspath(__file__))
20
21
22 def _find_root(start):
23     """向上查找包含 附件/ 的目录, 使附录中的源码副本在任意布局下均可运行"""
24     cur = start
25     for _ in range(6):
26         if os.path.isdir(os.path.join(cur, '附件')):
27             return cur
28         parent = os.path.dirname(cur)
29         if parent == cur:
30             break
31         cur = parent
32     return os.path.dirname(start)
33
34
35 def _pick_dir(*cands):
36     for c in cands:
37         if os.path.isdir(c):
38             return c
39     return cands[0]
40
41
42 ROOT = _find_root(HERE)
43 ATT = os.path.join(ROOT, '附件')
44 TPL = os.path.join(ATT, '附件5')
45 RES = _pick_dir(os.path.join(HERE, '结果'), os.path.join(ROOT, '求解', '结果'))
46 FIG = _pick_dir(os.path.join(HERE, '图片'), os.path.join(ROOT, '求解', '图片'))
47 os.makedirs(RES, exist_ok=True)
```

```

48 os.makedirs(FIG, exist_ok=True)
49
50 # ----- 常量（附录1 与 Phase 1 决策） -----
51 T = 144
52 DT = 1.0 / 6.0
53 ETA_C = 0.9
54 ETA_D = 0.9
55 E_MIN, E_MAX, E_CAP = 1200.0, 10800.0, 12000.0
56 P_B = 5000.0
57 ABAR = P_B * DT          # 833.333 kWh/时段（电池侧）
58 E0 = 6000.0
59 EMERG_MULT = 5.0
60 DEV_DOWN, DEV_UP = 0.5, 1.5
61 SEED = 42
62 np.random.seed(SEED)
63
64 REPORT_START = dt.date(2025, 2, 1)
65 REPORT_END = dt.date(2025, 12, 31)
66 BLOCKS = ['0:00-4:00', '4:00-8:00', '8:00-12:00', '12:00-16:00', '16:00-20:00', '
20:00-24:00']
67 SLOT_LABELS_TPL = None # 由模板读取
68
69
70 # ----- 数据装载 -----
71 def load_att1():
72     df = pd.read_excel(os.path.join(ATT, '附件1.xlsx'))
73     df.columns = ['时间', '电价', '小区负载', '光伏发电预测功率']
74     for c in df.columns[1:]:
75         df[c] = pd.to_numeric(df[c], errors='coerce')
76     return df
77
78
79 def _load_wide(fname, sheet=None):
80     path = os.path.join(ATT, fname)
81     df = pd.read_excel(path) if sheet is None else pd.read_excel(path, sheet_name=sheet)
82     dates = pd.to_datetime(df.iloc[:, 0].astype(str).str[:10]).dt.date.values
83     vals = df.iloc[:, 1:].astype(float).values
84     return dates, vals
85
86
87 def load_att2():
88     dl, load = _load_wide('附件2.xlsx', '小区负载')
89     _, pv = _load_wide('附件2.xlsx', '光伏发电实际功率')
90     return dl, load, pv
91
92
93 def load_att3():
94     df = pd.read_excel(os.path.join(ATT, '附件3.xlsx'))
95     df.columns = ['日期', '预报时刻'] + ['预报%d小时' % i for i in range(1, 25)]
96     df['日期'] = pd.to_datetime(df['日期']).ffill().dt.date
97     out = {}
98     for _, r in df.iterrows():

```

```

99         out[(r['日期'], str(r['预报时刻']))] = r.iloc[2:].astype(float).values
100     return out
101
102
103 def load_att4():
104     return _load_wide('附件4.xlsx')
105
106
107 def day_index(dates, d):
108     idx = np.where(dates == d)[0]
109     return int(idx[0]) if len(idx) else None
110
111
112 def fc_hourly_to_slots(hourly, u=0):
113     """附件3 预报（发布时刻 u，未来第 k 小时 k=1..24）→ 当日 144 时段（分段常数，W3）
114
115     关键口径：预报k小时 = 时钟 (u+k) 时；超出当日 24:00 的部分置 0。
116     """
117     v = np.zeros(T)
118     h = np.asarray(hourly, dtype=float)
119     for k in range(1, 25):
120         hh = u + k
121         if hh <= 24:
122             v[(hh - 1) * 6: hh * 6] = h[k - 1]
123     return v
124
125
126 def forecast_path_clock(fc, d, u, issues=(0, 6, 12, 18)):
127     """按发布时刻 u 可得的"今日逐整点预报"（相邻发布时刻线性插值），返回长度 24（索引=时钟
128         小时-1）"""
129     v = np.full(24, np.nan)
130     for t in issues:
131         if t > u:
132             continue
133         h = fc.get((d, '%d:00' % t))
134         if h is None:
135             continue
136         for k in range(1, 25):
137             hh = t + k
138             if hh <= 24:
139                 v[hh - 1] = h[k - 1]
140     prev = max([t for t in issues if t <= u], default=None)
141     nxt = min([t for t in issues if t > u], default=None)
142     if prev is not None and nxt is not None:
143         hp, hn = fc.get((d, '%d:00' % prev)), fc.get((d, '%d:00' % nxt))
144         if hp is not None and hn is not None:
145             w = (u - prev) / float(nxt - prev)
146             for k in range(1, 25):
147                 hh = nxt + k
148                 idxp, idxn = hh - prev - 1, hh - nxt - 1
149                 if hh <= 24 and 0 <= idxp < 24:

```

```

150     return np.where(np.isnan(v), 0.0, v)
151
152
153 def clock_path_to_slots(v):
154     return np.repeat(np.asarray(v, dtype=float), 6)
155
156
157 def report_dates():
158     d0, d1 = REPORT_START, REPORT_END
159     return [d0 + dt.timedelta(days=i) for i in range((d1 - d0).days + 1)]
160
161
162 # ----- 预测 -----
163 def ma_forecast(hist, W):
164     """hist: (n_days, 144) 历史实际; 返回同时段 W 日均值 (最后 W 天) """
165     return hist[-W:].mean(axis=0)
166
167
168 PREDICTORS = ['昨日持续', '3日均', '7日均', '14日均', '30日均', 'EWMA(0.7,14)', 'EWMA(0.5,7)', '同月均']
169
170
171 def pred(hist, hist_dates, name, target_date):
172     """滚动预测器 (只用 target_date 之前的历史), hist=(n,144)"""
173     if name == '昨日持续':
174         return hist[-1]
175     if name == '3日均':
176         return hist[-3:].mean(0)
177     if name == '7日均':
178         return hist[-7:].mean(0)
179     if name == '14日均':
180         return hist[-14:].mean(0)
181     if name == '30日均':
182         return hist[-30:].mean(0)
183     if name == 'EWMA(0.7,14)':
184         h = hist[-14:]; w = 0.7 ** np.arange(len(h) - 1, -1, -1)
185         return (h * (w / w.sum()))[:, None].sum(0)
186     if name == 'EWMA(0.5,7)':
187         h = hist[-7:]; w = 0.5 ** np.arange(len(h) - 1, -1, -1)
188         return (h * (w / w.sum()))[:, None].sum(0)
189     if name == '同月均':
190         m = [j for j, dd in enumerate(hist_dates) if dd.month == target_date.month]
191         return hist[m].mean(0) if m else hist[-7:].mean(0)
192     raise KeyError(name)
193
194
195 def best_load_predictor():
196     """从问题2 的精度对比结果中取负载 MAE 最优的预测器 (保证各问口径一致) """
197     p = os.path.join(RES, '问题2_预测器精度.csv')
198     if os.path.exists(p):
199         df = pd.read_csv(p)
200         return str(df.loc[df['负载MAE(kw)'].idxmin(), '预测器'])

```

```

201     return '7日均'
202
203
204 def best_pv_predictor():
205     """从问题2 的联合标定结果中取最优组合的光伏预测器"""
206     p = os.path.join(RES, '问题2_裕度标定.csv')
207     if os.path.exists(p):
208         df = pd.read_csv(p).sort_values('total_cost_jan')
209         return str(df.iloc[0]['光伏预测器'])
210     return '7日均'
211
212
213 # ----- LP 求解引擎 -----
214 def solve_plan(L, P, price, E_init=E0, E_term=E0, ghat=None, dev_pen=0.0):
215     """求解计划 LP。
216
217     L, P, price : 长度 n 的数组（可为带裕度的预测值）
218     E_init      : 首时段前的储电量
219     E_term      : 末时段后的储电量要求（None = 自由）
220     ghat        : 承诺计划（长度 n），配 dev_pen 施加 dev_pen*price*|g-ghat|
221     返回 dict(g, a, b, s, E, fun, status)
222     """
223     n = len(L)
224     L = np.asarray(L, float); P = np.asarray(P, float); price = np.asarray(price, float)
225     nv = 7 * n
226     c = np.zeros(nv)
227     c[0:n] = price
228     use_dev = (ghat is not None) and dev_pen > 0
229     if use_dev:
230         ghat = np.asarray(ghat, float)
231         c[5 * n:6 * n] = dev_pen * price
232         c[6 * n:7 * n] = dev_pen * price
233
234     rows, cols, vals, b_eq = [], [], [], []
235     ridx = 0
236     # (1) 母线平衡:  $g + \eta_d b - a/\eta_c - s = L\Delta t - P\Delta t$ 
237     for i in range(n):
238         rows += [ridx, ridx, ridx, ridx]
239         cols += [i, 2 * n + i, n + i, 3 * n + i]
240         vals += [1.0, ETA_D, -1.0 / ETA_C, -1.0]
241         b_eq.append(L[i] * DT - P[i] * DT)
242         ridx += 1
243     # (2) 储能动态:  $E_i - E_{i-1} - a_i + b_i = 0$ 
244     for i in range(n):
245         rows.append(ridx); cols.append(4 * n + i); vals.append(1.0)
246         if i == 0:
247             rows.append(ridx); cols.append(n + i); vals.append(-1.0)
248             rows.append(ridx); cols.append(2 * n + i); vals.append(1.0)
249             b_eq.append(E_init)
250         else:
251             rows.append(ridx); cols.append(4 * n + i - 1); vals.append(-1.0)
252             rows.append(ridx); cols.append(n + i); vals.append(-1.0)

```

```

253     rows.append(ridx); cols.append(2 * n + i); vals.append(1.0)
254     b_eq.append(0.0)
255     ridx += 1
256 # (3) 偏差:  $dp - dm - g = -ghat$ 
257 if use_dev:
258     for i in range(n):
259         rows += [ridx, ridx, ridx]
260         cols += [5 * n + i, 6 * n + i, i]
261         vals += [1.0, -1.0, -1.0]
262         b_eq.append(-ghat[i])
263         ridx += 1
264 # (4) 终端电量
265 if E_term is not None:
266     rows.append(ridx); cols.append(4 * n + n - 1); vals.append(1.0)
267     b_eq.append(float(E_term))
268     ridx += 1
269
270 A_eq = sparse.csr_matrix((vals, (rows, cols)), shape=(ridx, nv))
271 bounds = ([ (0, None) ] * n + [ (0, ABAR) ] * n + [ (0, ABAR) ] * n + [ (0, None) ] * n
272           + [ (E_MIN, E_MAX) ] * n + [ (0, None) ] * n + [ (0, None) ] * n)
273 res = linprog(c, A_eq=A_eq, b_eq=np.array(b_eq), bounds=bounds, method='highs')
274 if not res.success:
275     raise RuntimeError('LP 不可行/失败: %s' % res.message)
276 x = res.x
277 return dict(g=x[0:n].copy(), a=x[n:2 * n].copy(), b=x[2 * n:3 * n].copy(),
278           s=x[3 * n:4 * n].copy(), E=x[4 * n:5 * n].copy(),
279           fun=float(res.fun), status=res.status)
280
281
282 # ----- 结算 -----
283 def emergency_amount(L, P, g, b):
284     return np.maximum(0.0, np.asarray(L) * DT - np.asarray(g) - ETA_D * np.asarray(b) - np.
285                      .asarray(P) * DT)
286
287 def planned_cost(g, price):
288     return float(np.dot(price, g))
289
290
291 def emergency_cost(e, price):
292     return float(EMERG_MULT * np.dot(price, e))
293
294
295 def merge_intervals(e, tol=1e-3):
296     """把逐时段缺电量合并为连续时间段 [(起, 止, 电量)], 时间格式 H:MM"""
297     out = []
298     t = 0
299     n = len(e)
300     while t < n:
301         if e[t] > tol:
302             t0 = t
303             while t < n and e[t] > tol:

```

```

304         t += 1
305         qty = float(e[t0:t].sum())
306
307         def lab(m):
308             h, mm = divmod(int(m), 60)
309             return ('%d:%02d' % (h, mm)) if m < 24 * 60 else '24:00'
310         out.append((lab(t0 * 10), lab(t * 10), qty))
311     else:
312         t += 1
313     return out
314
315
316 # ----- 结果文件写出 -----
317 def _wb(name):
318     return openpyxl.load_workbook(os.path.join(TPL, name))
319
320
321 def write_result1(path, g, a, b, E):
322     wb = _wb('result1.xlsx')
323     ws = wb['计划购电量']
324     for k in range(T):
325         ws.cell(2 + k, 2).value = float(g[k])
326     ws2 = wb['充放电量']
327     for blk in range(6):
328         t0, t1 = blk * 24, (blk + 1) * 24
329         ws2.cell(2 + blk, 2).value = float(a[t0:t1].sum() / ETA_C) # 充电量 (母线侧)
330         ws2.cell(2 + blk, 3).value = float(ETA_D * b[t0:t1].sum()) # 放电量 (母线侧)
331     ws2.cell(2, 4).value = dt.time(0, 0); ws2.cell(2, 5).value = float(E0)
332     ws2.cell(3, 4).value = '24:00'; ws2.cell(3, 5).value = float(E[-1])
333     wb.save(path)
334
335
336 def write_wide_sheet(ws, dates, mat, qty, cost):
337     """写 日期x144 的宽表 (模板 A 列日期与表头已存在) """
338     for i, d in enumerate(dates):
339         ws.cell(2 + i, 1).value = dt.datetime(d.year, d.month, d.day)
340         for k in range(T):
341             ws.cell(2 + i, 2 + k).value = float(mat[i, k])
342         ws.cell(2 + i, 146).value = float(qty[i])
343         ws.cell(2 + i, 147).value = float(cost[i])
344
345
346 def write_charge_discharge(ws, days, chg, dis, Esoc):
347     """days: 日期列表; chg/dis: (nd,6) 母线侧电量; Esoc: (nd,145) 储电量轨迹"""
348     if ws.max_row > 1:
349         ws.delete_rows(2, ws.max_row)
350     r = 2
351     for i, d in enumerate(days):
352         for j in range(6):
353             ws.cell(r + j, 1).value = dt.datetime(d.year, d.month, d.day) if j == 0 else None
354             ws.cell(r + j, 2).value = BLOCKS[j]
355             ws.cell(r + j, 3).value = float(chg[i, j])

```

```

356         ws.cell(r + j, 4).value = float(dis[i, j])
357         ws.cell(r, 5).value = dt.time(0, 0); ws.cell(r, 6).value = float(Esoc[i, 0])
358         ws.cell(r + 1, 5).value = '24:00'; ws.cell(r + 1, 6).value = float(Esoc[i, 144])
359         r += 6
360
361
362 def write_emergency(ws, em_rows):
363     """em_rows: [(date, '起-止', qty)], 同日只在首行写日期"""
364     if ws.max_row > 1:
365         ws.delete_rows(2, ws.max_row)
366     r = 2
367     last = None
368     for d, span, qty in em_rows:
369         ws.cell(r, 1).value = dt.datetime(d.year, d.month, d.day) if d != last else None
370         ws.cell(r, 2).value = span
371         ws.cell(r, 3).value = float(qty)
372         last = d
373         r += 1
374
375
376 def blocks_of(x):
377     """144 段 → 6 个 4 小时分块之和"""
378     return np.array([x[i * 24:(i + 1) * 24].sum() for i in range(6)])
379
380
381 def fmt(x, n=2):
382     return ('%. ' + str(n) + 'f') % x
383
384
385 def setup_matplotlib():
386     """统一图表风格（仅 matplotlib, 禁 seaborn/jet/rainbow）"""
387     import matplotlib
388     matplotlib.use('Agg')
389     import matplotlib.pyplot as plt
390     plt.rcParams['font.sans-serif'] = ['Microsoft YaHei', 'SimHei', 'DejaVu Sans']
391     plt.rcParams['axes.unicode_minus'] = False
392     plt.rcParams['figure.dpi'] = 120
393     plt.rcParams['savefig.bbox'] = 'tight'
394     plt.rcParams['axes.grid'] = True
395     plt.rcParams['grid.alpha'] = 0.3
396     return plt
397
398
399 def hours_axis():
400     return np.arange(1, T + 1) / 6.0

```

B.2 solve_p1.py: 问题 1 求解

```

1 # -*- coding: utf-8 -*-
2 """问题1 求解: 完美信息下的日计划购电 LP
3 输出: result1.xlsx (项目根)、求解/结果/表1_问题1.csv、表2_问题1.csv、图片
4 运行: python 求解/问题1/问题1_求解.py

```

```

5 """
6 import os
7 import sys
8 import numpy as np
9 import pandas as pd
10
11 sys.path.insert(0, os.path.dirname(os.path.dirname(os.path.abspath(__file__))))
12 import common as C
13
14 plt = C.setup_matplotlib()
15
16
17 def main():
18     df1 = C.load_att1()
19     L = df1['小区负载'].values.astype(float)
20     P = df1['光伏发电预测功率'].values.astype(float)
21     price = df1['电价'].values.astype(float)
22
23     res = C.solve_plan(L, P, price, E_init=C.E0, E_term=C.E0)
24     g, a, b, s, E = res['g'], res['a'], res['b'], res['s'], res['E']
25
26     # ---- 指标 ----
27     gap = float((L - P).sum() * C.DT)
28     A = float(a.sum())
29     total_g = float(g.sum())
30     fee = C.planned_cost(g, price)
31     baseline = float(np.dot(price, np.maximum(0.0, (L - P)) * C.DT)) # 无储能基线
32     identity_rhs = gap + float(s.sum()) + (1.0 / C.ETA_C - C.ETA_D) * A
33
34     print('=' * 72)
35     print('问题1 结果 (附件1 典型日)')
36     print('=' * 72)
37     print('LP 状态: %s 目标(全天购电费) = %.2f 元' % (res['status'], fee))
38     print('全天购电量 = %.2f kWh | 全天净缺口 = %.2f kWh | 弃光 = %.2f kWh' % (total_g, gap,
39     , s.sum()))
40     print('储能电池侧吞吐量 A = %.2f kWh (母线侧充电 %.2f / 放电 %.2f)'
41     % (A, a.sum() / C.ETA_C, C.ETA_D * b.sum()))
42     print('恒等式核对:  $\Sigma g = %.4f$  vs 缺口+弃光+0.2111A = %.4f 残差 = %.2e'
43     % (total_g, identity_rhs, abs(total_g - identity_rhs)))
44     print('无储能基线购电费 = %.2f 元 → 储能节省 %.2f 元 (%.1f%%)'
45     % (baseline, baseline - fee, 100 * (baseline - fee) / baseline))
46     print('储电量: 0:00 = %.1f, 24:00 = %.1f, min = %.1f (t=%d), max = %.1f (t=%d)'
47     % (C.E0, E[-1], E.min(), E.argmin() + 1, E.max(), E.argmax() + 1))
48     print('功率上限核对: max a = %.2f, max b = %.2f (上限 %.2f)' % (a.max(), b.max(), C.
49     ABAR))
50
51     # ---- 表1 ----
52     slots = {'10:00-10:10': 61, '12:00-12:10': 73, '14:00-14:10': 85,
53     '16:00-16:10': 97, '18:00-18:10': 109, '20:00-20:10': 121}
54     rows1 = [{'时间段': k, '购电量(kWh)': round(float(g[v - 1]), 2)} for k, v in slots.
55     items()]
56     rows1.append({'时间段': '全天购电量', '购电量(kWh)': round(total_g, 2)})

```

```

54 rows1.append({'时段': '全天购电费', '购电量(kWh)': round(fee, 2)})
55 t1 = pd.DataFrame(rows1)
56 t1.to_csv(os.path.join(C.RES, '表1_问题1.csv'), index=False, encoding='utf-8-sig')
57 print('\n表1 指定时段购电量: ')
58 print(t1.to_string(index=False))
59
60 # ---- 表2 ----
61 rows2 = []
62 for i, blk in enumerate(C.BLOCKS):
63     t0, t1_ = i * 24, (i + 1) * 24
64     rows2.append({'时段': blk, '充电量(kWh)': round(float(a[t0:t1_].sum() / C.ETA_C),
65                                     2),
66                 '放电量(kWh)': round(float(C.ETA_D * b[t0:t1_].sum()), 2)})
67 rows2.append({'时段': '0:00储能', '充电量(kWh)': round(C.E0, 2), '放电量(kWh)': np.
68     nan})
69 rows2.append({'时段': '24:00储能', '充电量(kWh)': round(float(E[-1]), 2), '放电量(
70     kWh)': np.nan})
71 t2 = pd.DataFrame(rows2)
72 t2.to_csv(os.path.join(C.RES, '表2_问题1.csv'), index=False, encoding='utf-8-sig')
73 print('\n表2 储能充放电与储电量: ')
74 print(t2.to_string(index=False))
75
76 # ---- 明细 ----
77 det = pd.DataFrame({'时段': np.arange(1, C.T + 1), '起(h)': (np.arange(C.T)) / 6.0,
78                 '电价': price, '负载kW': L, '光伏kW': P, '购电量kWh': g,
79                 '充电量kWh': a / C.ETA_C, '放电量kWh': C.ETA_D * b,
80                 '弃光kWh': s, '储电量kWh': E})
81 det.to_csv(os.path.join(C.RES, '问题1_逐时段明细.csv'), index=False, encoding='utf-8-
82     sig')
83
84 # ---- 写 result1.xlsx ----
85 out = os.path.join(C.ROOT, 'result1.xlsx')
86 C.write_result1(out, g, a, b, E)
87 print('\n已写出: %s' % out)
88
89 # ---- 图 ----
90 h = C.hours_axis()
91 fig, ax = plt.subplots(3, 1, figsize=(11, 9), sharex=True)
92 ax[0].plot(h, price, color='tab:red', label='电价(元/kWh)')
93 ax[0].set_ylabel('电价 (元/kWh)'); ax[0].legend(loc='upper left')
94 ax[0].set_title('问题1: 附件1 典型日 —— 电价 / 负载 / 光伏 / 最优购电 / 储电量')
95 ax[1].plot(h, L, color='tab:blue', label='小区负载 (kW)')
96 ax[1].plot(h, P, color='tab:orange', label='光伏预测 (kW)')
97 ax[1].fill_between(h, 0, np.minimum(L, P), color='tab:green', alpha=0.15, label='光伏直
98     接消纳')
99 ax[1].set_ylabel('功率 (kW)'); ax[1].legend(loc='upper left')
100 ax[2].bar(h, g, width=1 / 6.0, color='tab:cyan', label='计划购电量 (kWh/10min)')
101 ax[2].bar(h, a / C.ETA_C, width=1 / 6.0, color='tab:purple', alpha=0.7, label='充电量(
102     母线侧)')
103 ax[2].bar(h, -C.ETA_D * b, width=1 / 6.0, color='tab:brown', alpha=0.7, label='放电量(
104     母线侧)')
105 ax[2].set_ylabel('电量 (kWh)'); ax[2].set_xlabel('时刻 (h)'); ax[2].legend(loc='upper

```

```

        left')
99     ax[2].set_xlim(0, 24)
100     fig.savefig(os.path.join(C.FIG, '图1_问题1_最优购电与充放电策略.png'))
101     plt.close(fig)
102
103     fig, ax = plt.subplots(2, 1, figsize=(11, 6), sharex=True)
104     ax[0].plot(h, price, color='tab:red'); ax[0].set_ylabel('电价 (元/kWh)')
105     ax[0].set_title('问题1: 电价与储电量轨迹 (影子价格阈值策略)')
106     ax[1].plot(np.concatenate([[0], h]), np.concatenate([[C.E0], E]), color='tab:blue')
107     ax[1].axhline(C.E_MAX, color='gray', ls='--', lw=1, label='上限 10800')
108     ax[1].axhline(C.E_MIN, color='gray', ls=':', lw=1, label='下限 1200')
109     ax[1].set_ylabel('储电量 (kWh)'); ax[1].set_xlabel('时刻 (h)'); ax[1].legend(loc='lower
        left')
110     ax[1].set_xlim(0, 24)
111     fig.savefig(os.path.join(C.FIG, '图2_问题1_储电量轨迹.png'))
112     plt.close(fig)
113     print('已输出 2 张图到 %s' % C.FIG)
114     return fee
115
116
117 if __name__ == '__main__':
118     main()

```

B.3 solve_p2.py: 问题 2 求解

```

1  # -*- coding: utf-8 -*-
2  """问题2 求解: 预测不确定 + 5 倍紧急购电 (2025-02-01 ~ 12-31, 334 天)
3  方法:
4      1) 用历史实际数据做同时段  $w=7$  日滑动平均预测 (严禁使用附件3)
5      2) 安全裕度参数 ( $\alpha, \beta$ ) 在 1 月数据上按"计划费+紧急费"最小标定
6      3) 每天 0:00 解 LP (日周期边界  $E_{144}=E_0$ ) → 实际结算紧急购电
7  输出: result2.xlsx (项目根)、求解/结果/*.csv、图片
8  运行: python 求解/问题2/问题2_求解.py
9  """
10 import os
11 import sys
12 import time
13 import numpy as np
14 import pandas as pd
15
16 sys.path.insert(0, os.path.dirname(os.path.dirname(os.path.abspath(__file__))))
17 import common as C
18
19 plt = C.setup_matplotlib()
20
21 W = 7
22 ALPHAS = [0.00, 0.03, 0.06, 0.09, 0.12, 0.15, 0.18, 0.21, 0.24]
23 BETAS = [0.00, 0.06, 0.12, 0.18]
24 PREDICTORS = ['昨日持续', '3日均', '7日均', '14日均', '30日均', 'EWMA(0.7,14)', 'EWMA
    (0.5,7)', '同月均']
25
26

```

```

27 def pred(hist, hist_dates, name, target_date):
28     """历史预测器: hist=(n,144) 历史实际, hist_dates 对应日期"""
29     if name == '昨日持续':
30         return hist[-1]
31     if name == '3日均':
32         return hist[-3:].mean(0)
33     if name == '7日均':
34         return hist[-7:].mean(0)
35     if name == '14日均':
36         return hist[-14:].mean(0)
37     if name == '30日均':
38         return hist[-30:].mean(0)
39     if name == 'EWMA(0.7,14)':
40         h = hist[-14:]; w = 0.7 ** np.arange(len(h) - 1, -1, -1)
41         return (h * (w / w.sum()))[:, None].sum(0)
42     if name == 'EWMA(0.5,7)':
43         h = hist[-7:]; w = 0.5 ** np.arange(len(h) - 1, -1, -1)
44         return (h * (w / w.sum()))[:, None].sum(0)
45     if name == '同月均':
46         m = [j for j, dd in enumerate(hist_dates) if dd.month == target_date.month]
47         if not m:
48             return hist[-7:].mean(0)
49         return hist[m].mean(0)
50     raise KeyError(name)
51
52
53 def eval_predictors(dates, load, pv):
54     """滚动起点评价: 对 Feb-Dec 每天只用其之前的数据做预测 (严禁前视偏差)"""
55     rows = []
56     rep = C.report_dates()
57     idx = [C.day_index(dates, d) for d in rep]
58     for name in PREDICTORS:
59         el, ep = [], []
60         for d, i in zip(rep, idx):
61             hL, hP = load[:i], pv[:i]
62             hd = dates[:i]
63             Lf = pred(hL, hd, name, d); Pf = pred(hP, hd, name, d)
64             el.append(load[i] - Lf); ep.append(pv[i] - Pf)
65         el = np.array(el); ep = np.array(ep)
66         rows.append({'预测器': name,
67                     '负载MAE(kW)': float(np.abs(el).mean()), '负载RMSE(kW)': float(np.sqrt((
68                         el ** 2).mean()))),
69                     '光伏MAE(kW)': float(np.abs(ep).mean()), '光伏RMSE(kW)': float(np.sqrt((
70                         ep ** 2).mean()))})
71     return pd.DataFrame(rows)
72
73 def simulate(L_fc, P_fc, price, L_act, P_act, alpha, beta):
74     r = C.solve_plan((1 + alpha) * L_fc, (1 - beta) * P_fc, price, E_init=C.E0, E_term=C.E0
75                     )
76     e = C.emergency_amount(L_act, P_act, r['g'], r['b'])
77     return r, e

```

```

76
77
78 def main():
79     t_start = time.time()
80     df1 = C.load_att1()
81     price = df1['电价'].values.astype(float)
82     dates, load, pv = C.load_att2()
83     n_all = len(dates)
84     rep_dates = C.report_dates()
85     rep_idx = [C.day_index(dates, d) for d in rep_dates]
86     assert all(i is not None for i in rep_idx) and len(rep_dates) == 334, (len(rep_dates),
87         sum(i is not None for i in rep_idx))
88
89     # ----- 0) 预测器精度对比（滚动评价，严禁前视偏差） -----
90     pdf = eval_predictors(dates, load, pv)
91     pdf.to_csv(os.path.join(C.RES, '问题2_预测器精度.csv'), index=False, encoding='utf-8-
92         sig')
93     print('=' * 72)
94     print('问题2 预测器精度对比（2025-02-01 ~ 12-31 滚动评价）')
95     print('=' * 72)
96     print(pdf.to_string(index=False, float_format=lambda x: '%.1f' % x))
97     L_name = pdf.loc[pdf['负载MAE(kw)'].idxmin(), '预测器']
98     P_name = pdf.loc[pdf['光伏MAE(kw)'].idxmin(), '预测器']
99     print('按 MAE 选定预测器：负载 = %s, 光伏 = %s' % (L_name, P_name))
100
101     # ----- 1) 决策导向的"预测器 + 裕度"联合标定（1 月 8-31 日） -----
102     jan_idx = [i for i in range(n_all) if dates[i].month == 1]
103     tune_idx = [i for i in jan_idx if i >= W] # 1/8 ~ 1/31
104     L_cands = list(dict.fromkeys([str(pdf.loc[pdf['负载MAE(kw)'].idxmin(), '预测器']),
105         str(pdf.loc[pdf['负载RMSE(kw)'].idxmin(), '预测器'])]))
106     P_cands = list(dict.fromkeys([str(pdf.loc[pdf['光伏MAE(kw)'].idxmin(), '预测器']),
107         str(pdf.loc[pdf['光伏RMSE(kw)'].idxmin(), '预测器'])]))
108     print('候选预测器：负载 %s | 光伏 %s (MAE/RMSE 最优各一)' % (L_cands, P_cands))
109     fc_cache = {}
110     for ln in L_cands:
111         for pn in P_cands:
112             fc_cache[(ln, pn)] = {i: (pred(load[:i], dates[:i], ln, dates[i]),
113                 pred(pv[:i], dates[:i], pn, dates[i])) for i in tune_idx}
114
115     grid = []
116     for ln in L_cands:
117         for pn in P_cands:
118             for alpha in ALPHAS:
119                 for beta in BETAS:
120                     tot = 0.0
121                     for i in tune_idx:
122                         Lf, Pf = fc_cache[(ln, pn)][i]
123                         r, e = simulate(Lf, Pf, price, load[i], pv[i], alpha, beta)
124                         tot += C.planned_cost(r['g'], price) + C.emergency_cost(e, price)
125                     grid.append({'负载预测器': ln, '光伏预测器': pn, 'alpha': alpha, 'beta':
126                         beta,
127                         'total_cost_jan': tot, 'avg_daily': tot / len(tune_idx)})
128     gdf = pd.DataFrame(grid).sort_values('total_cost_jan').reset_index(drop=True)

```

```

125 best = gdf.iloc[0]
126 alpha, beta = float(best['alpha']), float(best['beta'])
127 L_name, P_name = str(best['负载预测器']), str(best['光伏预测器'])
128 print('=' * 72)
129 print('问题2 预测器+裕度联合标定 (训练集: 2025 年 1 月 8-31 日, 共 %d 天)' % len(
    tune_idx))
130 print('=' * 72)
131 print(gdf.head(8).to_string(index=False, float_format=lambda x: '%.1f' % x))
132 print('最优组合: 负载预测器=%s, 光伏预测器=%s, alpha=%.2f (负载上浮), beta=%.2f (光伏下
    调)')
133     % (L_name, P_name, alpha, beta))
134 zero = gdf[(gdf.alpha == 0) & (gdf.beta == 0) & (gdf['负载预测器'] == L_name)
135     & (gdf['光伏预测器'] == P_name)].iloc[0]
136 print('无裕度基线( $\alpha=\beta=0$ ) 1 月总费用 = %.2f 元 → 最优裕度节省 %.2f 元 (%.2f%%)'
137     % (zero['total_cost_jan'], zero['total_cost_jan'] - best['total_cost_jan'],
138     100 * (zero['total_cost_jan'] - best['total_cost_jan']) / zero['total_cost_jan']
139     ))
139 gdf.to_csv(os.path.join(C.RES, '问题2_裕度标定.csv'), index=False, encoding='utf-8-sig'
140 )
141 # ----- 2) 报告期逐日求解 -----
142 nd = len(rep_dates)
143 G = np.zeros((nd, C.T)); A = np.zeros((nd, C.T)); B = np.zeros((nd, C.T))
144 S = np.zeros((nd, C.T)); E = np.zeros((nd, C.T + 1)); EM = np.zeros((nd, C.T))
145 plan_cost = np.zeros(nd); emg_cost = np.zeros(nd)
146 G0 = np.zeros((nd, C.T)); EM0 = np.zeros((nd, C.T)); cost0 = np.zeros(nd) # 无裕度对照
147 GP = np.zeros((nd, C.T)); EMP = np.zeros((nd, C.T)); costP = np.zeros(nd) # 完美信息下
    界
148 for k, (d, i) in enumerate(zip(rep_dates, rep_idx)):
149     Lf = pred(load[:, i], dates[:, i], L_name, d); Pf = pred(pv[:, i], dates[:, i], P_name, d)
150     r, e = simulate(Lf, Pf, price, load[i], pv[i], alpha, beta)
151     G[k], A[k], B[k], S[k] = r['g'], r['a'], r['b'], r['s']
152     E[k, 0] = C.E0; E[k, 1:] = r['E']; EM[k] = e
153     plan_cost[k] = C.planned_cost(r['g'], price); emg_cost[k] = C.emergency_cost(e,
        price)
154     r0, e0 = simulate(Lf, Pf, price, load[i], pv[i], 0.0, 0.0)
155     G0[k] = r0['g']; EM0[k] = e0
156     cost0[k] = C.planned_cost(r0['g'], price) + C.emergency_cost(e0, price)
157     rp, ep = simulate(load[i], pv[i], price, load[i], pv[i], 0.0, 0.0)
158     GP[k] = rp['g']; EMP[k] = ep
159     costP[k] = C.planned_cost(rp['g'], price) + C.emergency_cost(ep, price)
160     if (k + 1) % 60 == 0:
161         print('已完成 %d/%d 天 (%.1f s)' % (k + 1, nd, time.time() - t_start))
162
163 total = plan_cost.sum() + emg_cost.sum()
164 print('-' * 72)
165 print('问题2 报告期 (2025-02-01 ~ 12-31, %d 天)' % nd)
166 print('计划购电费 = %.2f 元 | 紧急购电费 = %.2f 元 | 总费用 = %.2f 元'
167     % (plan_cost.sum(), emg_cost.sum(), total))
168 print('紧急购电电量 = %.2f kWh (占负载 %.3f%%); 出现紧急购电的天数 = %d'
169     % (EM.sum(), 100 * EM.sum() / (load[rep_idx].sum() * C.DT), int((EM.sum(1) > 1e-3)
        .sum())))

```

```

170 print('对照① 无裕度( $\alpha=\beta=0$ ): 总费用 = %.2f 元  $\rightarrow$  裕度节省 %.2f 元 (%.2f%%)'
171       % (cost0.sum(), cost0.sum() - total, 100 * (cost0.sum() - total) / cost0.sum()))
172 print('对照② 完美信息下界: 总费用 = %.2f 元 (不可达, 缺口 %d%% 量级)'
173       % (costP.sum(), round(100 * (total - costP.sum()) / total)))
174 print('储能吞吐量合计 = %.0f kWh' % A.sum())
175
176 # ----- 3) 写 result2.xlsx -----
177 wb = C._wb('result2.xlsx')
178 C.write_wide_sheet(wb['计划购电量'], rep_dates, G, G.sum(1), plan_cost)
179 chg = np.array([C.blocks_of(A[k] / C.ETA_C) for k in range(nd)])
180 dis = np.array([C.blocks_of(C.ETA_D * B[k]) for k in range(nd)])
181 C.write_charge_discharge(wb['充放电量'], rep_dates, chg, dis, E)
182 em_rows = []
183 for k, d in enumerate(rep_dates):
184     for span in C.merge_intervals(EM[k]):
185         em_rows.append((d, '%s-%s' % (span[0], span[1]), span[2]))
186 C.write_emergency(wb['紧急购电量'], em_rows)
187 out = os.path.join(C.ROOT, 'result2.xlsx')
188 wb.save(out)
189 print('已写出: %s (紧急购电记录 %d 条)' % (out, len(em_rows)))
190
191 # ----- 4) CSV -----
192 months = sorted(set(d.month for d in rep_dates))
193 mrows = []
194 for m in months:
195     mk = [k for k, d in enumerate(rep_dates) if d.month == m]
196     mrows.append({'月份': m, '天数': len(mk),
197                  '计划购电量kWh': G[mk].sum(), '计划购电费元': plan_cost[mk].sum(),
198                  '紧急购电量kWh': EM[mk].sum(), '紧急购电费元': emg_cost[mk].sum(),
199                  '总费用元': plan_cost[mk].sum() + emg_cost[mk].sum(),
200                  '紧急天数': int((EM[mk].sum(1) > 1e-3).sum())})
201 mdf = pd.DataFrame(mrows)
202 mdf.loc[len(mdf)] = ['合计', nd, G.sum(), plan_cost.sum(), EM.sum(), emg_cost.sum(),
203                     total,
204                     int((EM.sum(1) > 1e-3).sum())]
205 mdf.to_csv(os.path.join(C.RES, '问题2_月度费用汇总.csv'), index=False, encoding='utf-8-
206            sig')
207 print('\n月度汇总: ')
208 print(mdf.to_string(index=False))
209
210 # 指定四天明细 (表1/表2/表3 格式)
211 key_dates = [pd.Timestamp('2025-03-20').date(), pd.Timestamp('2025-06-21').date(),
212              pd.Timestamp('2025-09-23').date(), pd.Timestamp('2025-12-21').date()]
213 slots = {'10:00-10:10': 61, '12:00-12:10': 73, '14:00-14:10': 85,
214          '16:00-16:10': 97, '18:00-18:10': 109, '20:00-20:10': 121}
215 rows = []
216 for d in key_dates:
217     k = rep_dates.index(d)
218     for label, v in slots.items():
219         rows.append({'日期': str(d), '时段': label, '购电量kWh': round(float(G[k, v -
220                                1]), 2)})
221     rows.append({'日期': str(d), '时段': '全天购电量', '购电量kWh': round(float(G[k].sum

```

```

    ()), 2))
219 rows.append({'日期': str(d), '时段': '全天购电费(计划)', '购电量kWh': round(float(
    plan_cost[k]), 2)})
220 rows.append({'日期': str(d), '时段': '全天紧急购电量', '购电量kWh': round(float(EM[k
    ].sum()), 2)})
221 rows.append({'日期': str(d), '时段': '全天紧急购电费', '购电量kWh': round(float(
    emg_cost[k]), 2)})
222 t1 = pd.DataFrame(rows)
223 t1.to_csv(os.path.join(C.RES, '表1_问题2_四天.csv'), index=False, encoding='utf-8-sig')
224 rows = []
225 for d in key_dates:
226     k = rep_dates.index(d)
227     for j, blk in enumerate(C.BLOCKS):
228         rows.append({'日期': str(d), '时间段': blk, '充电量kWh': round(float(chg[k, j]),
229                                2),
230                                '放电量kWh': round(float(dis[k, j]), 2)})
231 rows.append({'日期': str(d), '时间段': '0:00储电量', '充电量kWh': round(float(C.E0, 2), '
    放电量kWh': np.nan})
232 rows.append({'日期': str(d), '时间段': '24:00储电量', '充电量kWh': round(float(E[k,
    -1]), 2),
233                                '放电量kWh': np.nan})
234 pd.DataFrame(rows).to_csv(os.path.join(C.RES, '表2_问题2_四天.csv'), index=False,
    encoding='utf-8-sig')
235 rows = []
236 for d in key_dates:
237     k = rep_dates.index(d)
238     ms = C.merge_intervals(EM[k])
239     if not ms:
240         rows.append({'日期': str(d), '紧急购电时间段': '无', '紧急购电量kWh': 0.0})
241     for sp in ms:
242         rows.append({'日期': str(d), '紧急购电时间段': '%s-%s' % (sp[0], sp[1]),
243                                '紧急购电量kWh': round(sp[2], 2)})
244 pd.DataFrame(rows).to_csv(os.path.join(C.RES, '表3_问题2_紧急购电_四天.csv'), index=
    False,
245                                encoding='utf-8-sig')
246 np.savez(os.path.join(C.RES, '问题2_结果.npz'), G=G, A=A, B=B, S=S, E=E, EM=EM,
247           plan_cost=plan_cost, emg_cost=emg_cost, G0=G0, EM0=EM0, cost0=cost0,
248           GP=GP, EMP=EMP, costP=costP, rep_idx=np.array(rep_idx), price=price,
249           alpha=alpha, beta=beta)
250
251 # ----- 5) 图 -----
252 h = C.hours_axis()
253 fig, ax = plt.subplots(1, 2, figsize=(13, 4.6))
254 ax[0].bar(mdf['月份'][:-1], mdf['计划购电费元'][:-1], label='计划购电费', color='tab:
    blue')
255 ax[0].bar(mdf['月份'][:-1], mdf['紧急购电费元'][:-1], bottom=mdf['计划购电费元'][:-1],
256           label='紧急购电费', color='tab:red')
257 ax[0].set_xlabel('月份'); ax[0].set_ylabel('费用 (元)'); ax[0].legend()
258 ax[0].set_title('问题2: 月度费用构成')
259 ax[1].plot(range(1, nd + 1), EM.sum(1), color='tab:red', lw=0.8)
260 ax[1].set_xlabel('报告期第几天 (2025-02-01 起)'); ax[1].set_ylabel('当日紧急购电量 (kWh)

```

```

    ')')
261 ax[1].set_title('问题2: 每日紧急购电量')
262 fig.savefig(os.path.join(C.FIG, '图3_问题2_月度费用与紧急购电.png'))
263 plt.close(fig)
264
265 fig, ax = plt.subplots(1, 2, figsize=(13, 4.6))
266 sub = gdf[(gdf['负载预测器'] == L_name) & (gdf['光伏预测器'] == P_name)]
267 piv = sub.pivot(index='alpha', columns='beta', values='avg_daily')
268 im = ax[0].imshow(piv.values, aspect='auto', cmap='viridis', origin='lower')
269 ax[0].set_xticks(range(len(piv.columns))); ax[0].set_xticklabels(['%.2f' % b for b in
    piv.columns])
270 ax[0].set_yticks(range(len(piv.index))); ax[0].set_yticklabels(['%.2f' % a for a in piv
    .index])
271 ax[0].set_xlabel('光伏下调系数  $\beta$ '); ax[0].set_ylabel('负载上浮系数  $\alpha$ ')
272 ax[0].set_title('1 月标定: 日均总费用 (元)')
273 fig.colorbar(im, ax=ax[0])
274 ax[1].bar(['无裕度', '最优裕度', '完美信息'],
275           [cost0.sum() / 1e4, total / 1e4, costP.sum() / 1e4],
276           color=['tab:gray', 'tab:blue', 'tab:green'])
277 for i, v in enumerate([cost0.sum() / 1e4, total / 1e4, costP.sum() / 1e4]):
278     ax[1].text(i, v, '%.1f' % v, ha='center', va='bottom')
279 ax[1].set_ylabel('全年总费用 (万元)'); ax[1].set_title('问题2: 裕度价值与信息价值')
280 fig.savefig(os.path.join(C.FIG, '图4_问题2_裕度标定与价值对比.png'))
281 plt.close(fig)
282
283 k = rep_dates.index(key_dates[3])
284 fig, ax = plt.subplots(2, 1, figsize=(11, 6.4), sharex=True)
285 ax[0].plot(h, G[k] / C.DT, label='计划购电功率 (kW)', color='tab:blue')
286 ax[0].plot(h, EM[k] / C.DT, label='紧急购电功率 (kW)', color='tab:red')
287 ax[0].plot(h, load[rep_idx[k]], label='实际负载 (kW)', color='tab:green', lw=0.8)
288 ax[0].set_ylabel('功率 (kW)'); ax[0].legend(); ax[0].set_title('问题2: 2025-12-21 (最难
    日) 购电与紧急购电')
289 ax[1].plot(np.concatenate([[0], h]), E[k], color='tab:blue')
290 ax[1].axhline(C.E_MAX, color='gray', ls='--', lw=1); ax[1].axhline(C.E_MIN, color='gray
    ', ls=':', lw=1)
291 ax[1].set_ylabel('储电量 (kWh)'); ax[1].set_xlabel('时刻 (h)'); ax[1].set_xlim(0, 24)
292 fig.savefig(os.path.join(C.FIG, '图5_问题2_最难日策略.png'))
293 plt.close(fig)
294 print('已输出 3 张图; 总用时 %.1f s' % (time.time() - t_start))
295 return total
296
297
298 if __name__ == '__main__':
299     main()

```

B.4 solve_p3.py: 问题 3 求解

```

1 # -*- coding: utf-8 -*-
2 """问题3 求解: 0:00 计划 + 6/12/18 时预报更新与付费调整 (2025-02-01 ~ 12-31)
3
4 机制: 违约费 = 0.5*p*|调整量-计划量| (等价三档阶梯 1.5p/0.5p), 紧急购电 5p
5 预报: 附件3 (预报k小时 = 时钟 u+k 时, 分段常数下采样到 10 分钟)

```

```

6  负载: 无预报数据 → 历史滚动预测 (与问题2 同一预测器) + 安全裕度  $\alpha$  (在 1 月按 P3 策略标定,
   各策略共用)
7  策略: P0 不调整 / P1 +6:00 / P2 +12:00 / P3 +18:00 (题目方案) / P4 逐小时(插值预报) /
8  P5 完美信息(负载+光伏, 上界) / P5' 仅完美光伏信息(隔离光伏预报价值)
9  运行: python 求解/问题3/问题3_求解.py
10 """
11 import os
12 import sys
13 import time
14 import numpy as np
15 import pandas as pd
16
17 sys.path.insert(0, os.path.dirname(os.path.dirname(os.path.abspath(__file__))))
18 import common as C
19
20 plt = C.setup_matplotlib()
21 W = 7
22 DEV = 0.5
23 STAGES = [6, 12, 18]
24 ALPHAS = [0.00, 0.05, 0.10, 0.15, 0.20, 0.25, 0.30]
25
26
27 def forecast_slots(fc, d, u):
28     """发布时刻 u 的预报 → 当日 144 时段 (预报k小时 = 时钟 u+k 时) """
29     h = fc.get((d, '%d:00' % u))
30     return None if h is None else C.fc_hourly_to_slots(h, u)
31
32
33 def soc_of(a, b, E_init=C.E0):
34     E = np.empty(C.T + 1); E[0] = E_init
35     E[1:] = E_init + np.cumsum(a - b)
36     return E
37
38
39 def evaluate(g, a, b, L_act, P_act, price, g_hat):
40     e = np.maximum(0.0, L_act * C.DT - g - C.ETA_D * b - P_act * C.DT)
41     plan = float(np.dot(price, g))
42     dev = DEV * float(np.dot(price, np.abs(g - g_hat)))
43     emg = 5.0 * float(np.dot(price, e))
44     return dict(plan=plan, dev=dev, emg=emg, total=plan + dev + emg, e=e, g=g, a=a, b=b)
45
46
47 def run_day(fc, d, i, load, pv, price, Lhat, alpha, hourly=False):
48     """返回该日各策略的费用与 P3 明细"""
49     Lplan = (1 + alpha) * Lhat
50     P0f = forecast_slots(fc, d, 0)
51     plan0 = C.solve_plan(Lplan, P0f, price, E_init=C.E0, E_term=C.E0)
52     g_hat = plan0['g'].copy()
53     a_run, b_run = plan0['a'].copy(), plan0['b'].copy()
54     sols = {}
55     for u in STAGES:
56         t1 = 6 * u + 1

```

```

57     if t1 > C.T:
58         continue
59     Pf = forecast_slots(fc, d, u)
60     if Pf is None:
61         continue
62     E_init = C.E0 + a_run[:t1 - 1].sum() - b_run[:t1 - 1].sum()
63     r = C.solve_plan(Lplan[t1 - 1:], Pf[t1 - 1:], price[t1 - 1:], E_init=E_init,
64                     E_term=C.E0, ghat=g_hat[t1 - 1:], dev_pen=DEV)
65     sols[u] = r
66     a_run[t1 - 1:], b_run[t1 - 1:] = r['a'], r['b']
67
68 def compose(us):
69     g, a, b = g_hat.copy(), plan0['a'].copy(), plan0['b'].copy()
70     for u in us:
71         if u in sols:
72             t1 = 6 * u
73             g[t1:], a[t1:], b[t1:] = sols[u]['g'], sols[u]['a'], sols[u]['b']
74     return g, a, b
75
76 L_act, P_act = load[i], pv[i]
77 out = {'g_hat': g_hat, 'plan0': plan0, 'alpha': alpha}
78 for name, us in [('P0', []), ('P1', [6]), ('P2', [6, 12]), ('P3', [6, 12, 18])]:
79     g, a, b = compose(us)
80     out[name] = evaluate(g, a, b, L_act, P_act, price, g_hat)
81 # P5 完美信息 (负载+光伏)
82 rp = C.solve_plan(L_act, P_act, price, E_init=C.E0, E_term=C.E0)
83 out['P5'] = evaluate(rp['g'], rp['a'], rp['b'], L_act, P_act, price, g_hat)
84 # P5' 仅完美光伏信息 (负载仍为预测+裕度, 无调整)
85 rq = C.solve_plan(Lplan, P_act, price, E_init=C.E0, E_term=C.E0)
86 out["P5'"] = evaluate(rq['g'], rq['a'], rq['b'], L_act, P_act, price, g_hat)
87 # A_load 仅完美负载信息 (光伏仍为 0:00 预报, 无调整) → 隔离负载信息价值
88 rs = C.solve_plan(L_act, P0f, price, E_init=C.E0, E_term=C.E0)
89 out['A_load'] = evaluate(rs['g'], rs['a'], rs['b'], L_act, P_act, price, g_hat)
90 if hourly:
91     g4, a4, b4 = g_hat.copy(), plan0['a'].copy(), plan0['b'].copy()
92     for u in range(1, 24):
93         t1 = 6 * u + 1
94         if t1 > C.T:
95             break
96         Pf = C.clock_path_to_slots(C.forecast_path_clock(fc, d, u))
97         E_init = C.E0 + a4[:t1 - 1].sum() - b4[:t1 - 1].sum()
98         r = C.solve_plan(Lplan[t1 - 1:], Pf[t1 - 1:], price[t1 - 1:], E_init=E_init,
99                         E_term=C.E0, ghat=g_hat[t1 - 1:], dev_pen=DEV)
100         g4[t1 - 1:], a4[t1 - 1:], b4[t1 - 1:] = r['g'], r['a'], r['b']
101     out['P4'] = evaluate(g4, a4, b4, L_act, P_act, price, g_hat)
102 return out
103
104
105 def main():
106     t0 = time.time()
107     df1 = C.load_att1()
108     price = df1['电价'].values.astype(float)

```

```

109 dates, load, pv = C.load_att2()
110 fc = C.load_att3()
111 rep_dates = C.report_dates()
112 rep_idx = [C.day_index(dates, d) for d in rep_dates]
113 nd = len(rep_dates)
114 L_PRED = C.best_load_predictor()
115 print('负载预测器（与问题2 一致）： %s' % L_PRED)
116
117 # ----- 预报修订幅度（同一时钟小时对齐） -----
118 rev = []
119 for d in rep_dates:
120     p0 = C.forecast_path_clock(fc, d, 0); p6 = C.forecast_path_clock(fc, d, 6)
121     p12 = C.forecast_path_clock(fc, d, 12); p18 = C.forecast_path_clock(fc, d, 18)
122     for cell, a_, b_, lo in [('0:00->6:00', p0, p6, 7), ('6:00->12:00', p6, p12, 13),
123                             ('12:00->18:00', p12, p18, 19)]:
124         sl = slice(lo - 1, 24)
125         diff = b_[sl] - a_[sl]
126         rev.append({'修订': cell, 'MAE(kw)': float(np.abs(diff).mean()),
127                    'RMSE(kw)': float(np.sqrt((diff ** 2).mean()))})
128 rdf = pd.DataFrame(rev).groupby('修订', as_index=False).mean()
129 print('预报修订幅度（同一时钟小时的预报之差）：')
130 print(rdf.to_string(index=False, float_format=lambda x: '%.1f' % x))
131 rdf.to_csv(os.path.join(C.RES, '问题3_预报修订幅度.csv'), index=False, encoding='utf-8-
    sig')
132
133 # ----- 安全裕度  $\alpha$  标定（1 月，按 P3 策略、各策略共用） -----
134 jan_idx = [i for i in range(len(dates)) if dates[i].month == 1 and i >= W]
135 tune = []
136 for alpha in ALPHAS:
137     tot = 0.0
138     for i in jan_idx:
139         d = dates[i]
140         Lhat = C.pred(load[:i], dates[:i], L_PRED, d)
141         out = run_day(fc, d, i, load, pv, price, Lhat, alpha)
142         tot += out['P3']['total']
143     tune.append({'alpha': alpha, 'total_cost_jan': tot, 'avg_daily': tot / len(jan_idx)
144                })
145 tdf = pd.DataFrame(tune).sort_values('total_cost_jan').reset_index(drop=True)
146 alpha = float(tdf.iloc[0]['alpha'])
147 print('\n安全裕度标定（1 月 8 - 31 日，P3 策略）：')
148 print(tdf.to_string(index=False, float_format=lambda x: '%.1f' % x))
149 print('选定 alpha = %.2f' % alpha)
150 tdf.to_csv(os.path.join(C.RES, '问题3_裕度标定.csv'), index=False, encoding='utf-8-sig'
151            )
152
153 # ----- 报告期 -----
154 G_HAT = np.zeros((nd, C.T)); G3 = np.zeros((nd, C.T))
155 A3 = np.zeros((nd, C.T)); B3 = np.zeros((nd, C.T)); E3 = np.zeros((nd, C.T + 1))
156 EM3 = np.zeros((nd, C.T))
157 pol_names = ['P0', 'P1', 'P2', 'P3', 'P4', "P5", 'A_load', 'P5']
158 pol = {p: np.zeros(nd) for p in pol_names}
159 det = {p: {kk: np.zeros(nd) for kk in ('plan', 'dev', 'emg')} for p in pol_names}

```

```

158 for k, (d, i) in enumerate(zip(rep_dates, rep_idx)):
159     Lhat = C.pred(load[:i], dates[:i], L_PRED, d)
160     out = run_day(fc, d, i, load, pv, price, Lhat, alpha, hourly=True)
161     for p in pol_names:
162         pol[p][k] = out[p]['total']
163         for kk in ('plan', 'dev', 'emg'):
164             det[p][kk][k] = out[p][kk]
165     G_HAT[k] = out['g_hat']
166     r3 = out['P3']
167     G3[k], A3[k], B3[k], EM3[k] = r3['g'], r3['a'], r3['b'], r3['e']
168     E3[k] = soc_of(r3['a'], r3['b'])
169     if (k + 1) % 60 == 0:
170         print(' 已完成 %d/%d 天 (%.1f s)' % (k + 1, nd, time.time() - t0))
171
172 # ----- 策略对比 -----
173 print('=' * 72)
174 print('问题3 策略对比 (2025-02-01 ~ 12-31, %d 天, 裕度  $\alpha$ =%.2f)' % (nd, alpha))
175 print('=' * 72)
176 desc = {'P0': '仅 0:00 计划', 'P1': '0:00 + 6:00 调整', 'P2': '0:00 + 6:00 + 12:00',
177         'P3': '0:00 + 6/12/18 (题目方案)', 'P4': '逐小时更新 (插值预报)',
178         'P5': '仅完美光伏信息 (无调整, 上界)', 'A_load': '仅完美负载信息 (无调整, 上界)',
179         'P5': '完美信息 (负载+光伏, 上界)'}
180 sdf = pd.DataFrame([{'策略': p, '说明': desc[p], '计划购电费': det[p]['plan'].sum(),
181                     '违约费': det[p]['dev'].sum(), '紧急购电费': det[p]['emg'].sum(),
182                     '计划加紧急': det[p]['plan'].sum() + det[p]['emg'].sum(),
183                     '总费用': float(pol[p].sum())} for p in pol_names])
184 base = float(pol['P0'].sum())
185 sdf['相对P0节省'] = base - sdf['总费用']
186 sdf['节省比例%'] = 100 * (base - sdf['总费用']) / base
187 print(sdf.to_string(index=False, float_format=lambda x: '%.2f' % x))
188 sdf.to_csv(os.path.join(C.RES, '问题3_策略对比.csv'), index=False, encoding='utf-8-sig')
189
190 def nofee(p):
191     """单阶段 (完美信息) 边界: 不应计违约费, 只计购电费+紧急费"""
192     return float(det[p]['plan'].sum() + det[p]['emg'].sum())
193 marg = {'6:00 更新边际价值(P0-P1)': float(pol['P0'].sum() - pol['P1'].sum()),
194         '12:00 更新边际价值(P1-P2)': float(pol['P1'].sum() - pol['P2'].sum()),
195         '18:00 更新边际价值(P2-P3)': float(pol['P2'].sum() - pol['P3'].sum()),
196         '逐小时更新增益(P3-P4)': float(pol['P3'].sum() - pol['P4'].sum()),
197         '完美光伏信息价值(单阶段, 无违约费)': float(pol['P0'].sum() - nofee('P5')),
198         '完美负载信息价值(单阶段, 无违约费)': float(pol['P0'].sum() - nofee('A_load')),
199         '完美信息价值(单阶段, 无违约费)': float(pol['P0'].sum() - nofee('P5'))}
200 print('\n边际价值 (正 = 有价值, 元): ')
201 for kk, vv in marg.items():
202     print(' %s: %.2f' % (kk, vv))
203 pd.DataFrame([{'项目': kk, '费用下降(元)': vv} for kk, vv in marg.items()]).to_csv(
204     os.path.join(C.RES, '问题3_更新边际价值.csv'), index=False, encoding='utf-8-sig')
205 print('\nP3 全年费用构成: 计划 %.2f + 违约 %.2f + 紧急 %.2f = %.2f 元'
206       % (det['P3']['plan'].sum(), det['P3']['dev'].sum(), det['P3']['emg'].sum(), pol['P3'].sum()))

```

```

207 print('P3 紧急购电量 = %.2f kWh, 紧急天数 = %d, 违约调整量 = %.0f kWh'
208       % (EM3.sum(), int((EM3.sum(1) > 1e-3).sum()), float(np.abs(G3 - G_HAT).sum())))
209
210 # ----- 写 result3.xlsx -----
211 wb = C._wb('result3.xlsx')
212 C.write_wide_sheet(wb['计划购电量'], rep_dates, G_HAT, G_HAT.sum(1),
213                   np.array([float(np.dot(price, G_HAT[k])) for k in range(nd)]))
214 C.write_wide_sheet(wb['调整购电量'], rep_dates, G3, G3.sum(1),
215                   np.array([float(np.dot(price, G3[k])) for k in range(nd)]))
216 chg = np.array([C.blocks_of(A3[k] / C.ETA_C) for k in range(nd)])
217 dis = np.array([C.blocks_of(C.ETA_D * B3[k]) for k in range(nd)])
218 C.write_charge_discharge(wb['充放电电量'], rep_dates, chg, dis, E3)
219 em_rows = []
220 for k, d in enumerate(rep_dates):
221     for sp in C.merge_intervals(EM3[k]):
222         em_rows.append((d, '%s-%s' % (sp[0], sp[1]), sp[2]))
223 C.write_emergency(wb['紧急购电量'], em_rows)
224 out_path = os.path.join(C.ROOT, 'result3.xlsx')
225 wb.save(out_path)
226 print('已写出: %s (紧急记录 %d 条)' % (out_path, len(em_rows)))
227
228 # ----- 四天明细 -----
229 key_dates = [pd.Timestamp('2025-03-20').date(), pd.Timestamp('2025-06-21').date(),
230              pd.Timestamp('2025-09-23').date(), pd.Timestamp('2025-12-21').date())
231 slots = {'10:00-10:10': 61, '12:00-12:10': 73, '14:00-14:10': 85,
232          '16:00-16:10': 97, '18:00-18:10': 109, '20:00-20:10': 121}
233 rows1, rows2, rows3 = [], [], []
234 for d in key_dates:
235     k = rep_dates.index(d)
236     for lab, v in slots.items():
237         rows1.append({'日期': str(d), '时段': lab, '计划购电量kWh': round(float(G_HAT[k,
238                                     v - 1]), 2),
239                      '调整购电量kWh': round(float(G3[k, v - 1]), 2)})
240     rows1.append({'日期': str(d), '时段': '全天购电量(计划)', '计划购电量kWh': round(
241         float(G_HAT[k].sum()), 2),
242                  '调整购电量kWh': round(float(G3[k].sum()), 2)})
243     rows1.append({'日期': str(d), '时段': '全天购电费(元, 计划/调整)',
244                  '计划购电量kWh': round(float(np.dot(price, G_HAT[k])), 2),
245                  '调整购电量kWh': round(float(np.dot(price, G3[k])), 2)})
246     for j, blk in enumerate(C.BLOCKS):
247         rows2.append({'日期': str(d), '时间段': blk, '充电量kWh': round(float(chg[k, j])
248                                     , 2),
249                      '放电量kWh': round(float(dis[k, j]), 2)})
250     rows2.append({'日期': str(d), '时间段': '0:00储电量', '充电量kWh': round(C.E0, 2), '
251                  放电量kWh': np.nan})
252     rows2.append({'日期': str(d), '时间段': '24:00储电量', '充电量kWh': round(float(E3[k
253                                     , -1]), 2),
254                  '放电量kWh': np.nan})
255     ms = C.merge_intervals(EM3[k])
256     if not ms:
257         rows3.append({'日期': str(d), '紧急购电时间段': '无', '紧急购电量kWh': 0.0})
258     for sp in ms:

```

```

254         rows3.append({'日期': str(d), '紧急购电时间段': '%s-%s' % (sp[0], sp[1]),
255                       '紧急购电量kWh': round(sp[2], 2)})
256     pd.DataFrame(rows1).to_csv(os.path.join(C.RES, '表1_问题3_四天.csv'), index=False,
257                               encoding='utf-8-sig')
258     pd.DataFrame(rows2).to_csv(os.path.join(C.RES, '表2_问题3_四天.csv'), index=False,
259                               encoding='utf-8-sig')
260     pd.DataFrame(rows3).to_csv(os.path.join(C.RES, '表3_问题3_紧急购电_四天.csv'), index=
261                               False, encoding='utf-8-sig')
262
263     np.savez(os.path.join(C.RES, '问题3_结果.npz'), G_HAT=G_HAT, G3=G3, A3=A3, B3=B3, E3=E3
264             , EM3=EM3,
265             price=price, rep_idx=np.array(rep_idx), alpha=alpha,
266             **({'pol_%s' % p.replace("'", 'p')}: pol[p] for p in pol_names)})
267
268     # ----- 图 -----
269     fig, ax = plt.subplots(1, 2, figsize=(13, 4.6))
270     cols = ['tab:gray', 'tab:blue', 'tab:cyan', 'tab:green', 'tab:olive', 'tab:orange', '
271            tab:purple']
272     ax[0].bar(sdf['策略'], sdf['总费用'] / 1e4, color=cols)
273     for i, v in enumerate(sdf['总费用'] / 1e4):
274         ax[0].text(i, v, '%.2f' % v, ha='center', va='bottom', fontsize=8)
275     ax[0].set_ylabel('全年总费用 (万元)'); ax[0].set_title('问题3: 预报更新策略费用对比')
276     x = np.arange(3)
277     vals = [float(pol['P0'].sum() - pol['P1'].sum()), float(pol['P1'].sum() - pol['P2'].sum
278                ()),
279            float(pol['P2'].sum() - pol['P3'].sum())]
280     ax[1].bar(x, vals, color='tab:red')
281     ax[1].set_xticks(x); ax[1].set_xticklabels(['6:00 更新', '12:00 更新', '18:00 更新'])
282     ax[1].axhline(0, color='k', lw=0.8)
283     for i, v in enumerate(vals):
284         ax[1].text(i, v, '%.0f' % v, ha='center', va='bottom' if v >= 0 else 'top', fontsize
285                    =9)
286     ax[1].set_ylabel('边际价值 (元)'); ax[1].set_title('问题3: 各更新时点的边际价值')
287     fig.savefig(os.path.join(C.FIG, '图6_问题3_策略对比与边际价值.png'))
288     plt.close(fig)
289
290     k = rep_dates.index(key_dates[3])
291     h = C.hours_axis()
292     fig, ax = plt.subplots(2, 1, figsize=(11, 6.4), sharex=True)
293     ax[0].plot(h, G_HAT[k] / C.DT, label='0:00 计划购电功率', color='tab:blue', lw=1.0)
294     ax[0].plot(h, G3[k] / C.DT, label='最终调整购电功率', color='tab:green', lw=1.0)
295     ax[0].plot(h, EM3[k] / C.DT, label='紧急购电功率', color='tab:red', lw=0.9)
296     ax[0].set_ylabel('功率 (kW)'); ax[0].legend()
297     ax[0].set_title('问题3: 2025-12-21 计划/调整/紧急购电与储电量 (裕度  $\alpha$ =%.2f)' % alpha)
298     ax[1].plot(np.concatenate([[0], h]), E3[k], color='tab:blue')
299     ax[1].axhline(C.E_MAX, color='gray', ls='--', lw=1); ax[1].axhline(C.E_MIN, color='gray
300            ', ls=':', lw=1)
301     ax[1].set_ylabel('储电量 (kWh)'); ax[1].set_xlabel('时刻 (h)'); ax[1].set_xlim(0, 24)
302     fig.savefig(os.path.join(C.FIG, '图7_问题3_最难日计划与调整.png'))
303     plt.close(fig)
304     print('已输出 2 张图; 总用时 %.1f s' % (time.time() - t0))

```

```

298
299 if __name__ == '__main__':
300     main()

```

B.5 solve_p4.py: 问题 4 求解

```

1 # -*- coding: utf-8 -*-
2 """问题4 求解: 波动电价下重算问题2 (result4-2) 与问题3 (result4-3)
3
4 要点:
5 - 电价 0:00 未知 → 用历史电价滚动预测 (预测器由滚动评价选定); 结算一律用附件4 实际电价
6 - result4-2: 0:00 计划 + 5 倍实际电价紧急购电 (裕度在 1 月按实际价结算标定)
7 - result4-3: 0:00 计划 + 6/12/18 时调整 (违约费 0.5*p_act*|Δ|), 紧急 5*p_act;
8     负载/光伏预报源与问题3 相同; 裕度按 P3 策略在 1 月标定
9 运行: python 求解/问题4/问题4_求解.py
10 """
11 import os
12 import sys
13 import time
14 import numpy as np
15 import pandas as pd
16
17 sys.path.insert(0, os.path.dirname(os.path.dirname(os.path.abspath(__file__))))
18 import common as C
19
20 plt = C.setup_matplotlib()
21 W = 7
22 DEV = 0.5
23 STAGES = [6, 12, 18]
24 ALPHAS = [0.00, 0.05, 0.10, 0.15, 0.20, 0.25]
25 BETAS = [0.00, 0.06, 0.12, 0.18]
26
27
28 def forecast_slots(fc, d, u):
29     h = fc.get((d, '%d:00' % u))
30     return None if h is None else C.fc_hourly_to_slots(h, u)
31
32
33 def eval_price_predictors(dp, price):
34     rows = []
35     rep = C.report_dates()
36     idx = [C.day_index(dp, d) for d in rep]
37     for name in C.PREDICTORS:
38         e = np.array([price[i] - C.pred(price[:i], dp[:i], name, d) for d, i in zip(rep, idx)])
39         rows.append({'预测器': name, 'MAE(元/kWh)': float(np.abs(e).mean()),
40                     'RMSE(元/kWh)': float(np.sqrt((e ** 2).mean())), 'bias(元/kWh)': float(e.mean())})
41     return pd.DataFrame(rows)
42
43
44 def run_day43(fc, d, i, load, pv, pa, Lhat, alpha, pf, hourly=False):

```

```

45 """4-3 单日：计划用预测电价 pf，结算用实际电价 pa"""
46 Lplan = (1 + alpha) * Lhat
47 P0f = forecast_slots(fc, d, 0)
48 plan0 = C.solve_plan(Lplan, P0f, pf, E_init=C.E0, E_term=C.E0)
49 g_hat = plan0['g'].copy()
50 a_run, b_run = plan0['a'].copy(), plan0['b'].copy()
51 sols = {}
52 for u in STAGES:
53     t1 = 6 * u + 1
54     if t1 > C.T:
55         continue
56     Pf = forecast_slots(fc, d, u)
57     if Pf is None:
58         continue
59     E_init = C.E0 + a_run[:t1 - 1].sum() - b_run[:t1 - 1].sum()
60     r = C.solve_plan(Lplan[t1 - 1:], Pf[t1 - 1:], pf[t1 - 1:], E_init=E_init,
61                     E_term=C.E0, ghat=g_hat[t1 - 1:], dev_pen=DEV)
62     sols[u] = r
63     a_run[t1 - 1:], b_run[t1 - 1:] = r['a'], r['b']
64
65 def compose(us):
66     g, a, b = g_hat.copy(), plan0['a'].copy(), plan0['b'].copy()
67     for u in us:
68         if u in sols:
69             t1 = 6 * u
70             g[t1:], a[t1:], b[t1:] = sols[u]['g'], sols[u]['a'], sols[u]['b']
71     return g, a, b
72
73 L_act, P_act = load[i], pv[i]
74
75 def ev(g, a, b):
76     e = np.maximum(0.0, L_act * C.DT - g - C.ETA_D * b - P_act * C.DT)
77     plan = float(np.dot(pa, g))
78     dev = DEV * float(np.dot(pa, np.abs(g - g_hat)))
79     emg = 5.0 * float(np.dot(pa, e))
80     return dict(plan=plan, dev=dev, emg=emg, total=plan + dev + emg, e=e, g=g, a=a, b=b)
81
82 out = {'g_hat': g_hat}
83 for name, us in [('P0', []), ('P1', [6]), ('P2', [6, 12]), ('P3', [6, 12, 18])]:
84     g, a, b = compose(us)
85     out[name] = ev(g, a, b)
86 rp = C.solve_plan(L_act, P_act, pf, E_init=C.E0, E_term=C.E0)
87 out['P5'] = ev(rp['g'], rp['a'], rp['b'])
88 rq = C.solve_plan(Lplan, P_act, pf, E_init=C.E0, E_term=C.E0)
89 out["P5'"] = ev(rq['g'], rq['a'], rq['b'])
90 return out
91
92
93 def soc_of(a, b):
94     E = np.empty(C.T + 1); E[0] = C.E0
95     E[1:] = C.E0 + np.cumsum(a - b)
96     return E

```

```

97
98
99 def key_tables(rep_dates, G_HAT, G_FIN, EM, price_act, chg, dis, E, tag):
100     key_dates = [pd.Timestamp('2025-03-20').date(), pd.Timestamp('2025-06-21').date(),
101                  pd.Timestamp('2025-09-23').date(), pd.Timestamp('2025-12-21').date())
102     slots = {'10:00-10:10': 61, '12:00-12:10': 73, '14:00-14:10': 85,
103             '16:00-16:10': 97, '18:00-18:10': 109, '20:00-20:10': 121}
104     rows1, rows2, rows3 = [], [], []
105     for d in key_dates:
106         k = rep_dates.index(d)
107         for lab, v in slots.items():
108             rows1.append({'日期': str(d), '时段': lab, '计划购电量kWh': round(float(G_HAT[k,
109                                     v - 1]), 2),
110                          '调整购电量kWh': round(float(G_FIN[k, v - 1]), 2)})
111             rows1.append({'日期': str(d), '时段': '全天计划购电量', '计划购电量kWh': round(
112                 float(G_HAT[k].sum()), 2),
113                          '调整购电量kWh': round(float(G_FIN[k].sum()), 2)})
114             rows1.append({'日期': str(d), '时段': '全天购电费(实际电价结算,元)',
115                          '计划购电量kWh': round(float(np.dot(price_act[k], G_HAT[k])), 2),
116                          '调整购电量kWh': round(float(np.dot(price_act[k], G_FIN[k])), 2)})
117         for j, blk in enumerate(C.BLOCKS):
118             rows2.append({'日期': str(d), '时间段': blk, '充电量kWh': round(float(chg[k, j]),
119                                     2),
120                          '放电量kWh': round(float(dis[k, j]), 2)})
121             rows2.append({'日期': str(d), '时间段': '0:00储电量', '充电量kWh': round(float(C.E0, 2), '
122                 放电量kWh': np.nan)})
123             rows2.append({'日期': str(d), '时间段': '24:00储电量', '充电量kWh': round(float(E[k,
124                 -1]), 2),
125                          '放电量kWh': np.nan)})
126         ms = C.merge_intervals(EM[k])
127         if not ms:
128             rows3.append({'日期': str(d), '紧急购电时间段': '无', '紧急购电量kWh': 0.0})
129         for sp in ms:
130             rows3.append({'日期': str(d), '紧急购电时间段': '%s-%s' % (sp[0], sp[1]),
131                          '紧急购电量kWh': round(sp[2], 2)})
132     pd.DataFrame(rows1).to_csv(os.path.join(C.RES, '表1_问题%s_四天.csv' % tag), index=
133         False, encoding='utf-8-sig')
134     pd.DataFrame(rows2).to_csv(os.path.join(C.RES, '表2_问题%s_四天.csv' % tag), index=
135         False, encoding='utf-8-sig')
136     pd.DataFrame(rows3).to_csv(os.path.join(C.RES, '表3_问题%s_紧急购电_四天.csv' % tag),
137         index=False,
138         encoding='utf-8-sig')
139
140 def main():
141     t0 = time.time()
142     dates, load, pv = C.load_att2()
143     dp, price_act = C.load_att4()
144     fc = C.load_att3()
145     rep_dates = C.report_dates()
146     rep_idx = [C.day_index(dates, d) for d in rep_dates]
147     assert [C.day_index(dp, d) for d in rep_dates] == rep_idx

```

```

141 pa_rep = price_act[rep_idx] # 报告期逐日实际电价（按报告日索引）
142 nd = len(rep_dates)
143 L_PRED = C.best_load_predictor()
144 P_PRED = C.best_pv_predictor()
145
146 pdf = eval_price_predictors(dp, price_act)
147 pdf.to_csv(os.path.join(C.RES, '问题4_电价预测精度.csv'), index=False, encoding='utf-8-
    sig')
148 print('=' * 72)
149 print('问题4 电价预测器精度（2025-02-01 ~ 12-31 滚动评价）')
150 print('=' * 72)
151 print(pdf.to_string(index=False, float_format=lambda x: '%.4f' % x))
152 PR_PRICE = str(pdf.loc[pdf['MAE(元/kWh)'].idxmin(), '预测器'])
153 print('选定电价预测器: %s（负载 %s / 光伏 %s 沿用问题2）' % (PR_PRICE, L_PRED, P_PRED))
154
155 # ----- 4-2 裕度标定（1 月，按实际电价结算） -----
156 jan_idx = [i for i in range(len(dates)) if dates[i].month == 1 and i >= W]
157 cache = {i: (C.pred(load[:i], dates[:i], L_PRED, dates[i]),
158             C.pred(pv[:i], dates[:i], P_PRED, dates[i]),
159             C.pred(price_act[:i], dp[:i], PR_PRICE, dates[i])) for i in jan_idx}
160 grid = []
161 for a in ALPHAS:
162     for b in BETAS:
163         tot = 0.0
164         for i in jan_idx:
165             Lf, Pf, pf = cache[i]
166             r = C.solve_plan((1 + a) * Lf, (1 - b) * Pf, pf, E_init=C.E0, E_term=C.E0)
167             e = np.maximum(0.0, load[i] * C.DT - r['g'] - C.ETA_D * r['b'] - pv[i] * C.DT
168                             )
169             tot += float(np.dot(price_act[i], r['g'])) + 5.0 * float(np.dot(price_act[i],
170                                     e))
171             grid.append({'alpha': a, 'beta': b, 'total_cost_jan': tot})
172 gdf = pd.DataFrame(grid).sort_values('total_cost_jan').reset_index(drop=True)
173 a42, b42 = float(gdf.iloc[0]['alpha']), float(gdf.iloc[0]['beta'])
174 gdf.to_csv(os.path.join(C.RES, '问题4_4-2裕度标定.csv'), index=False, encoding='utf-8-
    sig')
175 print('4-2 最优裕度: alpha = %.2f, beta = %.2f（1 月按实际电价结算标定）' % (a42, b42))
176
177 # ----- 4-2 报告期 -----
178 G42 = np.zeros((nd, C.T)); A42 = np.zeros((nd, C.T)); B42 = np.zeros((nd, C.T))
179 E42 = np.zeros((nd, C.T + 1)); EM42 = np.zeros((nd, C.T))
180 plan42 = np.zeros(nd); emg42 = np.zeros(nd)
181 c42_0 = np.zeros(nd)
182 for k, (d, i) in enumerate(zip(rep_dates, rep_idx)):
183     Lf = C.pred(load[:i], dates[:i], L_PRED, d)
184     Pf = C.pred(pv[:i], dates[:i], P_PRED, d)
185     pf = C.pred(price_act[:i], dp[:i], PR_PRICE, d)
186     r = C.solve_plan((1 + a42) * Lf, (1 - b42) * Pf, pf, E_init=C.E0, E_term=C.E0)
187     e = np.maximum(0.0, load[i] * C.DT - r['g'] - C.ETA_D * r['b'] - pv[i] * C.DT)
188     G42[k], A42[k], B42[k], EM42[k] = r['g'], r['a'], r['b'], e
189     E42[k, 0] = C.E0; E42[k, 1:] = r['E']
190     plan42[k] = float(np.dot(price_act[i], r['g'])); emg42[k] = 5.0 * float(np.dot(

```

```

        price_act[i], e))
189     r0 = C.solve_plan(Lf, Pf, pf, E_init=C.E0, E_term=C.E0)
190     e0 = np.maximum(0.0, load[i] * C.DT - r0['g'] - C.ETA_D * r0['b'] - pv[i] * C.DT)
191     c42_0[k] = float(np.dot(price_act[i], r0['g'])) + 5.0 * float(np.dot(price_act[i],
        e0))
192     if (k + 1) % 80 == 0:
193         print(' 4-2 已完成 %d/%d 天 (%.1f s)' % (k + 1, nd, time.time() - t0))
194
195     tot42 = plan42.sum() + emg42.sum()
196     print('-' * 72)
197     print('问题4-2 (波动电价重算问题2, %d 天)' % nd)
198     print('计划购电费(实际价) = %.2f | 紧急购电费 = %.2f | 总费用 = %.2f 元'
199           % (plan42.sum(), emg42.sum(), tot42))
200     print('紧急购电量 = %.2f kWh (占负载 %.3f%), 紧急天数 = %d'
201           % (EM42.sum(), 100 * EM42.sum() / (load[rep_idx].sum() * C.DT), int((EM42.sum(1) >
        1e-3).sum()))))
202     print('对照 无裕度: 总费用 = %.2f 元 → 裕度节省 %.2f 元 (%.2f%) '
203           % (c42_0.sum(), c42_0.sum() - tot42, 100 * (c42_0.sum() - tot42) / c42_0.sum()))
204
205     wb = C._wb('result4-2.xlsx')
206     C.write_wide_sheet(wb['计划购电量'], rep_dates, G42, G42.sum(1), plan42)
207     chg = np.array([C.blocks_of(A42[k] / C.ETA_C) for k in range(nd)])
208     dis = np.array([C.blocks_of(C.ETA_D * B42[k]) for k in range(nd)])
209     C.write_charge_discharge(wb['充放电电量'], rep_dates, chg, dis, E42)
210     rows = []
211     for k, d in enumerate(rep_dates):
212         for sp in C.merge_intervals(EM42[k]):
213             rows.append((d, '%s-%s' % (sp[0], sp[1]), sp[2]))
214     C.write_emergency(wb['紧急购电量'], rows)
215     out = os.path.join(C.ROOT, 'result4-2.xlsx')
216     wb.save(out)
217     print('已写出: %s (紧急记录 %d 条)' % (out, len(rows)))
218     key_tables(rep_dates, G42, G42, EM42, pa_rep, chg, dis, E42, '4-2')
219
220     # ----- 4-3 裕度标定 (1 月, P3 策略) -----
221     tune43 = []
222     for a in ALPHAS:
223         tot = 0.0
224         for i in jan_idx:
225             d = dates[i]
226             Lf = C.pred(load[:i], dates[:i], L_PRED, d)
227             pf = C.pred(price_act[:i], dp[:i], PR_PRICE, d)
228             out43 = run_day43(fc, d, i, load, pv, price_act[i], Lf, a, pf)
229             tot += out43['P3']['total']
230         tune43.append({'alpha': a, 'total_cost_jan': tot})
231     t43 = pd.DataFrame(tune43).sort_values('total_cost_jan').reset_index(drop=True)
232     a43 = float(t43.iloc[0]['alpha'])
233     t43.to_csv(os.path.join(C.RES, '问题4_4-3裕度标定.csv'), index=False, encoding='utf-8-
        sig')
234     print('4-3 最优裕度: alpha = %.2f (1 月按 P3 策略标定)' % a43)
235
236     # ----- 4-3 报告期 -----

```

```

237 G_HAT = np.zeros((nd, C.T)); G43 = np.zeros((nd, C.T))
238 A43 = np.zeros((nd, C.T)); B43 = np.zeros((nd, C.T)); E43 = np.zeros((nd, C.T + 1))
239 EM43 = np.zeros((nd, C.T))
240 pol_names = ['P0', 'P1', 'P2', 'P3', 'P5', 'P5']
241 pol = {p: np.zeros(nd) for p in pol_names}
242 for k, (d, i) in enumerate(zip(rep_dates, rep_idx)):
243     Lf = C.pred(load[:i], dates[:i], L_PRED, d)
244     pf = C.pred(price_act[:i], dp[:i], PR_PRICE, d)
245     out43 = run_day43(fc, d, i, load, pv, price_act[i], Lf, a43, pf)
246     for p in pol_names:
247         pol[p][k] = out43[p]['total']
248     G_HAT[k] = out43['g_hat']
249     G43[k], A43[k], B43[k], EM43[k] = out43['P3']['g'], out43['P3']['a'], out43['P3']['b'], out43['P3']['e']
250     E43[k] = soc_of(out43['P3']['a'], out43['P3']['b'])
251     if (k + 1) % 80 == 0:
252         print(' 4-3 已完成 %d/%d 天 (%.1f s)' % (k + 1, nd, time.time() - t0))
253
254     desc = {'P0': '仅 0:00 计划', 'P1': '0:00 + 6:00 调整', 'P2': '0:00 + 6:00 + 12:00',
255             'P3': '0:00 + 6/12/18 (题目方案)', 'P5': '仅完美光伏信息 (上界)',
256             'P5': '完美信息 (负载+光伏, 上界)'}
257     sdf = pd.DataFrame([{'策略': p, '说明': desc[p], '总费用': float(pol[p].sum())} for p
258                          in pol_names])
259     base = float(pol['P0'].sum())
260     sdf['相对P0节省'] = base - sdf['总费用']
261     sdf['节省比例%'] = 100 * (base - sdf['总费用']) / base
262     print('--' * 72)
263     print('问题4-3 策略对比 (波动电价, 计划用预测价、结算用实际价, 裕度  $\alpha=0.2f$ )' % a43)
264     print(sdf.to_string(index=False, float_format=lambda x: '%.2f' % x))
265     sdf.to_csv(os.path.join(C.RES, '问题4_4-3策略对比.csv'), index=False, encoding='utf-8-sig')
266
267     marg = {'6:00 更新边际价值(P0-P1)': float(pol['P0'].sum() - pol['P1'].sum()),
268            '12:00 更新边际价值(P1-P2)': float(pol['P1'].sum() - pol['P2'].sum()),
269            '18:00 更新边际价值(P2-P3)': float(pol['P2'].sum() - pol['P3'].sum()),
270            '完美光伏信息上界(P3-P5)': float(pol['P3'].sum() - pol['P5'].sum()),
271            '完美信息上界(P3-P5)': float(pol['P3'].sum() - pol['P5'].sum())}
272     print('\n边际价值 (正 = 有价值, 元): ')
273     for kk, vv in marg.items():
274         print(' %s: %.2f' % (kk, vv))
275     pd.DataFrame([{'项目': kk, '费用下降(元)': vv} for kk, vv in marg.items()]).to_csv(
276         os.path.join(C.RES, '问题4_4-3更新边际价值.csv'), index=False, encoding='utf-8-sig')
277
278     wb = C._wb('result4-3.xlsx')
279     C.write_sheet(wb['计划购电量'], rep_dates, G_HAT, G_HAT.sum(1),
280                  np.array([float(np.dot(pa_rep[k], G_HAT[k])) for k in range(nd)]))
281     C.write_sheet(wb['调整购电量'], rep_dates, G43, G43.sum(1),
282                  np.array([float(np.dot(pa_rep[k], G43[k])) for k in range(nd)]))
283     chg = np.array([C.blocks_of(A43[k] / C.ETA_C) for k in range(nd)])
284     dis = np.array([C.blocks_of(C.ETA_D * B43[k]) for k in range(nd)])
285     C.write_charge_discharge(wb['充放电电量'], rep_dates, chg, dis, E43)
286     rows = []
287     for k, d in enumerate(rep_dates):

```

```

286     for sp in C.merge_intervals(EM43[k]):
287         rows.append((d, '%s-%s' % (sp[0], sp[1]), sp[2]))
288     C.write_emergency(wb['紧急购电量'], rows)
289     out = os.path.join(C.ROOT, 'result4-3.xlsx')
290     wb.save(out)
291     print('已写出: %s (紧急记录 %d 条)' % (out, len(rows)))
292     key_tables(rep_dates, G_HAT, G43, EM43, pa_rep, chg, dis, E43, '4-3')
293
294     mrows = []
295     for m in sorted(set(x.month for x in rep_dates)):
296         mk = [k for k, x in enumerate(rep_dates) if x.month == m]
297         mrows.append({'月份': m, '电价均值': float(np.mean([price_act[rep_idx[k]].mean() for
298             k in mk])),
299             '4-2总费用': plan42[mk].sum() + emg42[mk].sum(), '4-2紧急量kWh': EM42[mk]
300                 .sum(),
301             '4-3总费用': pol['P3'][mk].sum(), '4-3紧急量kWh': EM43[mk].sum()})
302     mdf = pd.DataFrame(mrows)
303     mdf.to_csv(os.path.join(C.RES, '问题4_月度费用对比.csv'), index=False, encoding='utf-8-
304         sig')
305     print(mdf.to_string(index=False, float_format=lambda x: '%.0f' % x))
306
307     np.savez(os.path.join(C.RES, '问题4_结果.npz'), G42=G42, A42=A42, B42=B42, E42=E42,
308         EM42=EM42,
309         plan42=plan42, emg42=emg42, c42_0=c42_0, G_HAT=G_HAT, G43=G43, A43=A43, B43=B43,
310         E43=E43,
311         EM43=EM43, price_act=price_act, rep_idx=np.array(rep_idx), a42=a42, b42=b42, a43
312             =a43,
313         **({'pol_%s' % p.replace("'", 'p')}: pol[p] for p in pol_names)})
314
315     # ----- 图 -----
316     k0 = rep_dates.index(pd.Timestamp('2025-06-21')).date()
317     h = C.hours_axis()
318     fig, ax = plt.subplots(1, 2, figsize=(13, 4.6))
319     ax[0].plot(h, price_act[rep_idx[k0]], color='tab:red', lw=1.0, label='实际电价')
320     ax[0].plot(h, C.pred(price_act[:rep_idx[k0]], dp[:rep_idx[k0]], PR_PRICE, rep_dates[k0
321         ]),
322         color='tab:blue', ls='--', lw=1.0, label='预测电价')
323     ax[0].set_xlabel('时刻 (h)'); ax[0].set_ylabel('电价 (元/kWh)'); ax[0].legend()
324     ax[0].set_title('问题4: 2025-06-21 电价预测 vs 实际'); ax[0].set_xlim(0, 24)
325     ax[1].plot([price_act[i].mean() for i in rep_idx], lw=0.8, color='tab:purple')
326     ax[1].set_xlabel('报告期第几天'); ax[1].set_ylabel('日均电价 (元/kWh)')
327     ax[1].set_title('问题4: 日均电价波动')
328     fig.savefig(os.path.join(C.FIG, '图8_问题4_电价预测与波动.png'))
329     plt.close(fig)
330
331     fig, ax = plt.subplots(1, 2, figsize=(13, 4.6))
332     p2_total = 16647984.89
333     labels = ['问题2\n(固定电价)', '问题4-2\n(波动电价)', '问题3\n(固定电价)', '问题4-3\n(
334         波动电价)']
335     vals = [p2_total / 1e4, tot42 / 1e4]
336     try:
337         s3 = pd.read_csv(os.path.join(C.RES, '问题3_策略对比.csv'))

```

```

330     vals.append(float(s3.loc[s3.策略 == 'P3', '总费用'].values[0]) / 1e4)
331 except Exception:
332     vals.append(np.nan)
333 vals.append(float(pol['P3'].sum()) / 1e4)
334 ax[0].bar(labels, vals, color=['tab:blue', 'tab:red', 'tab:cyan', 'tab:orange'])
335 for i, v in enumerate(vals):
336     ax[0].text(i, v, '%.1f' % v, ha='center', va='bottom', fontsize=9)
337 ax[0].set_ylabel('全年总费用 (万元)'); ax[0].set_title('固定电价 vs 波动电价 (总费用)')
338 ax[1].bar(sdf['策略'], sdf['总费用'] / 1e4,
339          color=['tab:gray', 'tab:blue', 'tab:cyan', 'tab:green', 'tab:olive', 'tab:
340                purple'])
341 for i, v in enumerate(sdf['总费用'] / 1e4):
342     ax[1].text(i, v, '%.1f' % v, ha='center', va='bottom', fontsize=8)
343 ax[1].set_ylabel('全年总费用 (万元)'); ax[1].set_title('问题4-3: 各更新策略费用 (波动电
344 价)')
345 fig.savefig(os.path.join(C.FIG, '图9_问题4_固定vs波动电价对比.png'))
346 plt.close(fig)
347 print('已输出 2 张图; 总用时 %.1f s' % (time.time() - t0))
348
349 if __name__ == '__main__':
350     main()

```

B.6 verify_p25.py: 独立验算

```

1  # -*- coding: utf-8 -*-
2  """Phase 2.5 独立验算 (信息隔离)
3  规则: 不导入求解脚本; 自带数据装载、独立 LP 变量布局 (母线侧 c/d)、独立 DP;
4  直接读取 result*.xlsx 与求解结果 npz 做交叉核对。
5  输出: 求解/验算/phase25_验算报告.md、求解/验算/phase25_验算明细.csv
6  运行: python 求解/验算/phase25_独立验算.py
7  """
8  import os
9  import sys
10 import datetime as dt
11 import numpy as np
12 import pandas as pd
13 import openpyxl
14 from scipy.optimize import linprog
15 from scipy import sparse
16
17 HERE = os.path.dirname(os.path.abspath(__file__))
18
19 def _find_dir(name, maxup=8):
20     """自脚本目录向上查找子目录 name, 使附录中的源码副本在任意布局下均可运行"""
21     cur = os.path.dirname(os.path.abspath(__file__))
22     for _ in range(maxup):
23         cand = os.path.join(cur, name)
24         if os.path.isdir(cand):
25             return cand
26     parent = os.path.dirname(cur)

```

```

27         if parent == cur:
28             break
29         cur = parent
30     return os.path.join(os.path.dirname(os.path.abspath(__file__)), name)
31
32
33 ATT = _find_dir('附件')
34 ROOT = os.path.dirname(ATT)
35 RES = _find_dir(os.path.join('求解', '结果'))
36
37 T = 144
38 DT = 1.0 / 6.0
39 ETA = 0.9
40 EMIN, EMAX = 1200.0, 10800.0
41 ABAR = 5000.0 * DT
42 E0 = 6000.0
43 CHECKS = []
44
45
46 def add(name, got, ref, rel=None, note='', abs_tol=None):
47     """绝对容差优先: |got-ref| ≤ abs_tol (或 ref=0 时) 判 PASS"""
48     diff = abs(got - ref)
49     if abs_tol is not None and diff <= abs_tol:
50         rel, v = 0.0, 'PASS'
51     else:
52         if rel is None:
53             rel = diff / max(abs(ref), 1e-9)
54             v = 'FAIL' if rel > 0.30 else ('SUSPECT' if rel > 0.10 else 'PASS')
55     CHECKS.append({'检查项': name, '本实现值': got, '参照值': ref, '相对偏差': rel,
56                  '判定': v, '说明': note})
57     print('[%s] %-46s 本实现=%.6g 参照=%.6g 偏差=%.3g%% %s' % (v, name, got, ref, 100 * rel,
58                        , note))
59     return v
60
61 def add_ineq(name, lhs, rhs, note='', tol=1e-6):
62     """断言 lhs ≤ rhs (+tol): 用于信息上界关系 (完美信息不应更差)"""
63     ok = lhs <= rhs + tol
64     CHECKS.append({'检查项': name, '本实现值': lhs, '参照值': rhs,
65                  '相对偏差': (lhs - rhs) / max(abs(rhs), 1e-9), '判定': 'PASS' if ok else 'FAIL',
66                  '说明': note + ' (差值 %.6g 元)' % (lhs - rhs)})
67     print('[%s] %-46s %.6g ≤ %.6g (差 %.4g)' % ('PASS' if ok else 'FAIL', name, lhs, rhs,
68        lhs - rhs))
69     return ok
70
71 # ----- 独立数据装载 -----
72 def load_att1():
73     df = pd.read_excel(os.path.join(ATT, '附件1.xlsx'))
74     df.columns = ['时间', '电价', '小区负载', '光伏发电预测功率']
75     for c in df.columns[1:]:

```

```

76         df[c] = pd.to_numeric(df[c], errors='coerce')
77     return df
78
79
80 def load_att2():
81     a = pd.read_excel(os.path.join(ATT, '附件2.xlsx'), sheet_name='小区负载')
82     b = pd.read_excel(os.path.join(ATT, '附件2.xlsx'), sheet_name='光伏发电实际功率')
83     d = pd.to_datetime(a.iloc[:, 0].astype(str).str[:10]).dt.date.values
84     return d, a.iloc[:, 1:].astype(float).values, b.iloc[:, 1:].astype(float).values
85
86
87 def load_att4():
88     p = pd.read_excel(os.path.join(ATT, '附件4.xlsx'))
89     return pd.to_datetime(p.iloc[:, 0].astype(str).str[:10]).dt.date.values, p.iloc[:, 1:].
        astype(float).values
90
91
92 # ----- 独立 LP (母线侧变量: g, c, d, s, E) -----
93 def lp_bus(L, P, price, E_init=E0, E_term=E0):
94     """独立实现: 变量 [g(n), c(n) 母线充电, d(n) 母线放电, s(n), E(n)]
95     平衡:  $g + d + P\Delta t = L\Delta t + c + s$ 
96     储能:  $E_t = E_{t-1} + \eta c_t - d_t/\eta$ 
97     功率:  $c \leq \bar{A}/\eta$ ,  $d \leq \bar{A} \cdot \eta$  (电池侧 5000 kW 口径)
98     """
99     n = len(L)
100     nv = 5 * n
101     cvec = np.zeros(nv); cvec[:n] = price
102     rows, cols, vals, beq = [], [], [], []
103     r = 0
104     for i in range(n):
105         rows += [r, r, r, r]; cols += [i, 2 * n + i, n + i, 3 * n + i]
106         vals += [1.0, 1.0, -1.0, -1.0]
107         beq.append(L[i] * DT - P[i] * DT); r += 1
108     for i in range(n):
109         rows.append(r); cols.append(4 * n + i); vals.append(1.0)
110         if i == 0:
111             rows.append(r); cols.append(n + i); vals.append(-ETA)
112             rows.append(r); cols.append(2 * n + i); vals.append(1.0 / ETA)
113             beq.append(E_init)
114         else:
115             rows.append(r); cols.append(4 * n + i - 1); vals.append(-1.0)
116             rows.append(r); cols.append(n + i); vals.append(-ETA)
117             rows.append(r); cols.append(2 * n + i); vals.append(1.0 / ETA)
118             beq.append(0.0)
119         r += 1
120     rows.append(r); cols.append(4 * n + n - 1); vals.append(1.0); beq.append(E_term); r += 1
121     A = sparse.csr_matrix((vals, (rows, cols)), shape=(r, nv))
122     bounds = ((0, None) * n + [(0, ABAR / ETA)] * n + [(0, ABAR * ETA)] * n + [(0, None)]
        * n
        + [(EMIN, EMAX)] * n)
123     res = linprog(cvec, A_eq=A, b_eq=np.array(beq), bounds=bounds, method='highs')
124

```

```

125     if not res.success:
126         raise RuntimeError(res.message)
127     x = res.x
128     return dict(g=x[:n], c=x[n:2 * n], d=x[2 * n:3 * n], s=x[3 * n:4 * n], E=x[4 * n:5 * n
129                ], fun=res.fun)
130
131 # ----- 独立 DP (状态=储电量网格) -----
132 def dp_day(L, P, price, step=20.0, E_init=E0, E_term=E0):
133     grid = np.arange(EMIN, EMAX + 1e-9, step)
134     n = len(grid)
135     i0 = int(round((E_init - EMIN) / step))
136     iT = int(round((E_term - EMIN) / step))
137     V = np.full(n, np.inf); V[iT] = 0.0
138     diff = grid[None, :] - grid[:, None]
139     chg = np.maximum(diff, 0.0) # E' > E → 充电
140     dis = np.maximum(-diff, 0.0) # E > E' → 放电
141     feas = (chg <= ABAR + 1e-9) & (dis <= ABAR + 1e-9)
142     for t in range(T - 1, -1, -1):
143         g = np.maximum(L[t] * DT - P[t] * DT + chg / ETA - ETA * dis, 0.0)
144         tot = price[t] * g + V[None, :]
145         tot = np.where(feas, tot, np.inf)
146         V = tot.min(axis=1)
147     return float(V[i0])
148
149 # ----- 读取结果文件 -----
150 def read_wide(path, sheet):
151     wb = openpyxl.load_workbook(path, data_only=True)
152     ws = wb[sheet]
153     dates, mat, qty, cost = [], [], [], []
154     for r in range(2, ws.max_row + 1):
155         v = ws.cell(r, 1).value
156         if v is None:
157             continue
158         dates.append(pd.Timestamp(v).date() if not isinstance(v, dt.date) else v)
159         mat.append([ws.cell(r, 2 + k).value for k in range(T)])
160         qty.append(ws.cell(r, 146).value)
161         cost.append(ws.cell(r, 147).value)
162     return dates, np.array(mat, float), np.array(qty, float), np.array(cost, float)
163
164
165 def read_cd(path, sheet='充放电量'):
166     wb = openpyxl.load_workbook(path, data_only=True)
167     ws = wb[sheet]
168     out = {}
169     cur = None
170     for r in range(2, ws.max_row + 1):
171         d = ws.cell(r, 1).value
172         if d is not None:
173             cur = pd.Timestamp(d).date() if not isinstance(d, dt.date) else d
174             out[cur] = {'chg': [], 'dis': [], 'E0': None, 'E24': None}
175

```

```

176         if cur is None:
177             continue
178         out[cur]['chg'].append(ws.cell(r, 3).value)
179         out[cur]['dis'].append(ws.cell(r, 4).value)
180         tv, ev = ws.cell(r, 5).value, ws.cell(r, 6).value
181         if ev is not None:
182             if isinstance(tv, str) and tv.startswith('24'):
183                 out[cur]['E24'] = ev
184             else:
185                 out[cur]['E0'] = ev
186         return out
187
188
189 def read_emergency(path):
190     wb = openpyxl.load_workbook(path, data_only=True)
191     ws = wb['紧急购电量']
192     rows, cur, tot = [], None, 0.0
193     for r in range(2, ws.max_row + 1):
194         d = ws.cell(r, 1).value
195         if d is not None:
196             cur = pd.Timestamp(d).date() if not isinstance(d, dt.date) else d
197             span, q = ws.cell(r, 2).value, ws.cell(r, 3).value
198             if span is None and q is None:
199                 continue
200             rows.append((cur, span, q))
201             if q is not None and span != '无':
202                 tot += float(q)
203     return rows, tot
204
205
206 def main():
207     html = []
208     print('=' * 90)
209     print('Phase 2.5 独立验算')
210     print('=' * 90)
211     df1 = load_att1()
212     price1 = df1['电价'].values.astype(float)
213     L1 = df1['小区负载'].values.astype(float)
214     P1 = df1['光伏发电预测功率'].values.astype(float)
215     dates, load, pv = load_att2()
216     dp4, price4 = load_att4()
217
218     # ===== 1. 问题1: 独立 LP (母线侧布局) vs DP vs 求解器结果 =====
219     print('\n--- 1. 问题1: 确定性 LP 的独立复核 ---')
220     r_bus = lp_bus(L1, P1, price1)
221     fee_bus = float(r_bus['fun'])
222     t1 = pd.read_csv(os.path.join(RES, '表1_问题1.csv'))
223     fee_ref = float(t1.loc[t1['时间段'] == '全天购电费', '购电量(kWh)'].values[0])
224     add('问题1 全天购电费 (独立LP vs 求解结果)', fee_bus, fee_ref, None, '独立变量布局复核')
225
226     # 从 result1.xlsx 独立复算费用与守恒
227     wb1 = openpyxl.load_workbook(os.path.join(ROOT, 'result1.xlsx'), data_only=True)

```

```

227 ws1 = wb1['计划购电量']
228 g1 = np.array([ws1.cell(2 + k, 2).value for k in range(T)], float)
229 fee_xlsx = float(np.dot(price1, g1))
230 add('问题1 全天购电费 (result1.xlsx 复算)', fee_xlsx, fee_ref, None, '读表复算')
231 gap = float((L1 - P1).sum() * DT)
232 ws2 = wb1['充放电量']
233 chg_tot = float(sum(ws2.cell(2 + i, 2).value for i in range(6)))
234 dis_tot = float(sum(ws2.cell(2 + i, 3).value for i in range(6)))
235 A_batt = chg_tot * ETA # 电池侧吞吐量
236 s_implied = float(g1.sum() - gap - (1.0 / ETA - ETA) * A_batt)
237 CHECKS.append({'检查项': '问题1 弃光电量 (由恒等式反推)', '本实现值': s_implied, '参照
    值': 0.0,
    '相对偏差': 0.0, '判定': 'PASS' if abs(s_implied) < 1.0 else 'SUSPECT',
    '说明': 'Σg=缺口+弃光+0.2111A'})
238 print(['%s] 问题1 弃光反推 = %.4f kWh (应≈0)'
239         % (CHECKS[-1]['判定'], s_implied))
240 add('问题1 充电量-放电守恒 (母线侧效率换算)',
241     chg_tot * ETA, dis_tot / ETA, abs(chg_tot * ETA - dis_tot / ETA) / max(dis_tot / ETA
242                                     , 1e-9),
243     'E(0)=E(24) 与 Σa=Σb 等价')
244 dp_fee = dp_day(L1, P1, price1, step=20.0)
245 add('问题1 DP 最优值 vs LP (离散化间隙)', dp_fee, fee_ref, (dp_fee - fee_ref) /
246     fee_ref,
247     'DP 是 LP 的受限解, 应 ≥ LP 且间隙小')
248 # 储能约束独立复核 (由 result1 的 4 小时块无法逐时段检查, 此处检查端点与功率上限)
249 add('问题1 0:00 储电量', float(ws2.cell(2, 5).value), E0, None, '')
250 add('问题1 24:00 储电量', float(ws2.cell(3, 5).value), E0, None, '')
251
252 # ===== 2. 问题2: 计划与结算独立复算 =====
253 print('\n--- 2. 问题2: 计划/紧急购电的独立复算 ---')
254 n2 = np.load(os.path.join(RES, '问题2_结果.npz'))
255 G, A, B, EM = n2['G'], n2['A'], n2['B'], n2['EM']
256 rep_idx = n2['rep_idx']
257 L_act, P_act = load[rep_idx], pv[rep_idx]
258 EM_chk = np.maximum(0.0, L_act * DT - G - ETA * B - P_act * DT)
259 add('问题2 紧急购电总量 (独立复算)', EM_chk.sum(), EM.sum(), None, '逐时段 max(0, 缺口
    )')
260 plan_chk = float((G * price1).sum())
261 add('问题2 计划购电费 (独立复算)', plan_chk, float(n2['plan_cost'].sum()), None, 'Σp ·
    g')
262 emg_chk = float(5.0 * (EM_chk * price1).sum())
263 add('问题2 紧急购电费 (独立复算)', emg_chk, float(n2['emg_cost'].sum()), None, 'Σ5p · e
    ')
264 # 计划侧平衡 (独立重算 问题2 使用的带裕度预测输入) 与供电保障
265 def ma7(h):
266     return h[-7:].mean(0)
267
268 def ewma0714(h):
269     hh = h[-14:]
270     w = 0.7 ** np.arange(len(hh) - 1, -1, -1)
271     return (hh * (w / w.sum()))[:, None].sum(0)
272

```

```

273 Lfc = np.array([ma7(load[i - 7:i]) for i in rep_idx])
274 Pfc = np.array([ewma0714(pv[i - 14:i]) for i in rep_idx])
275 al, be = float(n2['alpha']), float(n2['beta'])
276 resid = (G + ETA * B + (1 - be) * Pfc * DT - (1 + al) * Lfc * DT - A / ETA - n2['S'])
277 add('问题2 计划侧逐时段能量平衡 (最大绝对残差 kWh)', float(np.abs(resid).max()), 0.0,
    None,
    'g+η_d b+P̄Δt=LΔt+a/η_c+s (独立重算预测)', abs_tol=1e-4)
278 supply = G + ETA * B + P_act * DT + EM - L_act * DT
279 add('问题2 供电保障违反次数 (供给+紧急 < 负载)', float((supply < -1e-6).sum()), 0.0,
    None,
    '紧急购电定义的自洽性', abs_tol=0)
281 add('问题2 全年计划弃光电量 (kWh)', float(n2['S'].sum()), float(n2['S'].sum()), None,
    '光伏盈余未入库部分')
282 add('问题2 SOC 越界最大量 (kWh)', max(0.0, float(n2['E'].min() - EMIN), float(EMAX -
    n2['E'].max()))),
    0.0, 0.0, 'E 位于 [1200,10800]')
283 add('问题2 功率上限越界 (kWh/段)', max(0.0, float(A.max() - ABAR), float(B.max() -
    ABAR)), 0.0, 0.0,
    'a,b≤833.33')
284 # 结果文件结构核对
285 d2, M2, Q2, C2 = read_wide(os.path.join(ROOT, 'result2.xlsx'), '计划购电量')
286 add('result2 行数', len(d2), 334, None, '2025-02-01~12-31')
287 add('result2 全天购电量=行和 (最大相对偏差)',
    float(np.max(np.abs(M2.sum(1) - Q2) / np.maximum(Q2, 1e-9))), 0.0, None, '')
288 add('result2 全天购电费=Σp·g (最大相对偏差)',
    float(np.max(np.abs((M2 * price1).sum(1) - C2) / np.maximum(np.abs(C2), 1e-9))),
    0.0, None, '')
289 add('result2 计划矩阵与 npz 一致性 (最大绝对差)', float(np.max(np.abs(M2 - G))), 0.0,
    None,
    '写出精度', abs_tol=1e-6)
290 cd2 = read_cd(os.path.join(ROOT, 'result2.xlsx'))
291 e0_bad = max(abs(v['E0'] - E0) for v in cd2.values() if v['E0'] is not None)
292 e24_bad = max(abs(v['E24'] - E0) for v in cd2.values() if v['E24'] is not None)
293 add('result2 逐日 0:00 储电量偏差', e0_bad, 0.0, None, '')
294 add('result2 逐日 24:00 储电量偏差', e24_bad, 0.0, None, '')
295 rec = []
296 for k, d in enumerate(d2):
297     ch = np.array([x for x in cd2[d]['chg'] if x is not None], float)
298     di = np.array([x for x in cd2[d]['dis'] if x is not None], float)
299     rec.append(abs(E0 + ETA * ch.sum() - di.sum() / ETA - cd2[d]['E24']))
300 add('result2 逐日 充放电-储电量自洽 (最大偏差 kWh)', float(max(rec)), 0.0, None, 'E24=
    E0+Σa-Σb',
    abs_tol=1e-3)
301 rows_em, tot_em = read_emergency(os.path.join(ROOT, 'result2.xlsx'))
302 add('result2 紧急购电表合计 vs npz', tot_em, float(EM.sum()), None, '')
303
304 # ===== 3. 问题3: 策略费用与信息上界 =====
305 print('\n--- 3. 问题3: 策略一致性与信息价值 ---')
306 s3 = pd.read_csv(os.path.join(RES, '问题3_策略对比.csv'))
307 v3 = dict(zip(s3['策略'], s3['总费用']))
308 add('问题3 策略费用分解自洽 (P3)',
    float(v3['P3']), float(s3.loc[s3.策略 == 'P3', ['计划购电费', '违约费', '紧急购电费

```

```

        ']].sum(axis=1).values[0]),
317     None, '总费用=计划+违约+紧急')
318 add_ineq('问题3 完美信息最优  $\leq P3$  (信息上界)', float(v3['P5']), float(v3['P3']),
319         '完美信息 (负载+光伏) 应不劣于任何策略')
320 add_ineq('问题3 仅完美光伏信息  $\leq P0$ ', float(v3["P5"]), float(v3['P0']), '光伏预报价值
    上界')
321 add_ineq('问题3 仅完美负载信息  $\leq P0$ ', float(v3['A_load']), float(v3['P0']), '负载预报价值
    上界')
322 add('问题3 完美信息下紧急购电费 (应 $\approx 0$ )', float(s3.loc[s3.策略 == 'P5', '紧急购电费'].
    values[0]), 0.0,
323     None, '完美信息不应触发紧急购电', abs_tol=1e-3)
324 n3 = np.load(os.path.join(RES, '问题3_结果.npz'))
325 add('问题3 SOC 越界最大量 (kWh)', max(0.0, float(n3['E3'].min() - EMIN), float(EMAX -
    n3['E3'].max()))),
326     0.0, 0.0, '')
327 add('问题3 功率上限越界 (kWh/段)',
328     max(0.0, float(n3['A3'].max() - ABAR), float(n3['B3'].max() - ABAR)), 0.0, 0.0, '')
329 d3, M3, Q3, C3 = read_wide(os.path.join(ROOT, 'result3.xlsx'), '计划购电量')
330 d3b, M3b, Q3b, C3b = read_wide(os.path.join(ROOT, 'result3.xlsx'), '调整购电量')
331 add('result3 计划/调整矩阵与 npz 一致性 (最大绝对差)',
332     max(float(np.max(np.abs(M3 - n3['G_HAT']))), float(np.max(np.abs(M3b - n3['G3'])))),
333     0.0, None,
334     '写出精度', abs_tol=1e-6)
335 add('result3 调整购电量 $\leq$ 约束校验: 调低量 $=\Sigma$ (计划-调整)+',
336     float(np.abs(np.minimum(M3b - M3, 0)).sum()), float(np.abs(np.minimum(n3['G3'] - n3[
    'G_HAT'], 0)).sum()),
337     None, '')
338 add('result3 全天购电量=行和 (最大相对偏差)',
339     float(np.max(np.abs(M3b.sum(1) - Q3b) / np.maximum(Q3b, 1e-9))), 0.0, None, '')
340 # ===== 4. 问题4: 按实际电价结算复核 =====
341 print('\n--- 4. 问题4: 波动电价结算复核 ---')
342 n4 = np.load(os.path.join(RES, '问题4_结果.npz'))
343 idx4 = n4['rep_idx']
344 pa = price4[idx4]
345 fee42 = float((n4['G42'] * pa).sum())
346 add('问题4-2 计划购电费 (独立复算)', fee42, float(n4['plan42'].sum()), None, ' $\Sigma p_{act} \cdot$ 
    g')
347 emg42 = float(5.0 * (n4['EM42'] * pa).sum())
348 add('问题4-2 紧急购电费 (独立复算)', emg42, float(n4['emg42'].sum()), None, ' $\Sigma p_{act} \cdot$ 
    e')
349 d42, M42, Q42, C42 = read_wide(os.path.join(ROOT, 'result4-2.xlsx'), '计划购电量')
350 add('result4-2 全天购电费 $=\Sigma p_{act} \cdot g$  (最大相对偏差)',
351     float(np.max(np.abs((M42 * pa).sum(1) - C42) / np.maximum(np.abs(C42), 1e-9))), 0.0,
    None, '')
352 d43, M43, Q43, C43 = read_wide(os.path.join(ROOT, 'result4-3.xlsx'), '调整购电量')
353 add('result4-3 全天购电费 $=\Sigma p_{act} \cdot g$  (最大相对偏差)',
354     float(np.max(np.abs((M43 * pa).sum(1) - C43) / np.maximum(np.abs(C43), 1e-9))), 0.0,
    None, '')
355 add('result4-3 调整矩阵与 npz 一致性 (最大绝对差)', float(np.max(np.abs(M43 - n4['G43'
    ])))), 0.0, None,
356     '写出精度', abs_tol=1e-6)

```

```

357 add_ineq('问题4-3 完美信息最优  $\leq$  P3 (信息上界)', float(n4['pol_P5'].sum()), float(n4['
358 pol_P3'].sum()),
359 '波动电价下的信息价值上界')
360 add('问题4 SOC 越界最大量 (kWh)',
361 max(0.0, float(n4['E42'].min() - EMIN), float(EMAX - n4['E42'].max()),
362 float(n4['E43'].min() - EMIN), float(EMAX - n4['E43'].max())), 0.0, 0.0, '')
363
364 # ===== 5. 汇总 =====
365 df = pd.DataFrame(CHECKS)
366 df.to_csv(os.path.join(HERE, 'phase25_验算明细.csv'), index=False, encoding='utf-8-sig'
367 )
368 n_fail = int((df['判定'] == 'FAIL').sum()); n_susp = int((df['判定'] == 'SUSPECT').sum
369 ())
370 overall = 'FAIL' if n_fail else ('SUSPECT' if n_susp else 'PASS')
371 print('\n' + '=' * 90)
372 print('验算汇总: 共 %d 项, PASS %d, SUSPECT %d, FAIL %d  $\rightarrow$  总体判定: %s'
373 % (len(df), int((df['判定'] == 'PASS').sum()), n_susp, n_fail, overall))
374 print('=' * 90)
375
376 with open(os.path.join(HERE, 'phase25_验算报告.md'), 'w', encoding='utf-8') as f:
377     f.write('# Phase 2.5 独立验算报告\n\n')
378     f.write('> 规则: 分歧 >30%% FAIL、10 - 30%% SUSPECT、否则 PASS。 **总体判定: %s**\n\n'
379           '% overall')
380     f.write('## 验算方法 (信息隔离) \n\n')
381     f.write('- 不导入求解脚本 ( `求解/common.py`、`求解/问题X/*.py` ), 全部独立实现。 \n'
382           ')
383     f.write('- 问题1 用**独立 LP (母线侧变量布局 g/c/d/s/E) **与**独立 DP (储电量网格,
384           步长 20 kWh) **\n')
385     f.write('两种不同算法复核确定性最优值。 \n')
386     f.write('- 问题2-4 用独立结算公式复算费用, 并直接读取 `result*.xlsx` 核对结构、行
387           和、 \n')
388     f.write('逐日储电量自洽、SOC/功率约束; 计划侧能量平衡用**独立重算的带裕度预测输入
389           **逐时段校验。 \n\n')
390     f.write('## 验算结果明细\n\n')
391     f.write('| 检查项 | 本实现值 | 参照值 | 相对偏差 | 判定 | 说明 | \n
392           |---|---|---|---|---|---| \n')
393     for _, r in df.iterrows():
394         f.write('| %s | %.6g | %.6g | %.3g%% | %s | %s | \n'
395               % (r['检查项'], r['本实现值'], r['参照值'], 100 * r['相对偏差'], r['判定'], r['说明']))
396     f.write('\n## 结论\n\n')
397     f.write('- 确定性核心 (问题1): 独立 LP 与独立 DP 均与求解结果一致, DP 离散化间隙在
398           1% 量级内, '
399           '能量守恒等式残差可忽略。 \n')
400     f.write('- 结果文件: 5 个 result 文件的结构、行和、逐日储电量自洽性与约束均通过核
401           对。 \n')
402     f.write('- 随机/滚动部分 (问题2-4): 独立结算复算与求解输出一致; 信息上界关系 (完
403           美信息  $\leq$  所有策略) 成立。 \n')
404     if n_susp or n_fail:
405         f.write('- 需关注项: %d 项 SUSPECT、%d 项 FAIL (见上表)。 \n' % (n_susp, n_fail))
406     f.write('\n## 已知口径说明 (非缺陷) \n\n')

```

```

395     f.write('1. result 模板的 144 行/列标签比数据标签晚 10 分钟（模板以区间起点命名，数
        据以区间终点命名），'
396         '本实现按时间顺序一一对应填充，行和与全天合计严格自治。\\n')
397     f.write('2. 附件3 的"预报 k 小时"按相对发布时刻解释（6:00 发布的预报1小时 = 当日
        7:00），'
398         '该口径下各时段预报误差量级合理（0:00 全天 MAE 341 kW、12:00 剩余时段 MAE 272
        kW、'
399         '18:00 剩余时段 MAE 18 kW）。\\n')
400     f.write('3. 表2 充放电按母线侧口径统计（便于与购电量对账）。\\n')
401     print('报告已写出: %s' % os.path.join(HERE, 'phase25_验算报告.md'))
402     return overall
403
404
405 if __name__ == '__main__':
406     main()

```

B.7 plots.py: 图表生成

```

1  # -*- coding: utf-8 -*-
2  """Phase 3 论文图表生成（300 dpi，全部数据驱动，数据源为求解结果 npz/CSV 与附件）
3  输出目录：论文/图/
4  运行：python 论文/绘图.py
5  """
6  import os
7  import sys
8  import datetime as dt
9  import numpy as np
10 import pandas as pd
11
12 sys.path.insert(0, os.path.join(os.path.dirname(os.path.dirname(os.path.abspath(__file__))),
    '求解'))
13 import common as C
14
15 ROOT = C.ROOT
16 OUT = os.path.join(ROOT, '论文', '图')
17 os.makedirs(OUT, exist_ok=True)
18 plt = C.setup_matplotlib()
19 plt.rcParams['figure.dpi'] = 300
20 plt.rcParams['savefig.dpi'] = 300
21 plt.rcParams['font.size'] = 10.5
22 plt.rcParams['axes.titlesize'] = 11
23 plt.rcParams['legend.fontsize'] = 9
24 plt.rcParams['lines.linewidth'] = 1.2
25
26 RES = C.RES
27 CMAP = 'viridis'
28
29
30 def sv(fig, name):
31     p = os.path.join(OUT, name)
32     fig.savefig(p)
33     plt.close(fig)

```

```

34     print(' ->', name)
35
36
37 def lp_sens(L, P, price, eta_c=0.9, eta_d=0.9, abar=5000.0 / 6, emin=1200.0, emax=10800.0,
38           e0=6000.0):
39     """参数化单日 LP (仅供灵敏度分析使用, 变量布局与 common.solve_plan 一致)
40     变量 [g(n), a(n), b(n), s(n), E(n)], 返回 (费用, 购电量, 电池侧吞吐量, E轨迹)
41     """
42     import numpy as np
43     from scipy.optimize import linprog
44     from scipy import sparse
45     n = len(L)
46     nv = 5 * n
47     c = np.zeros(nv); c[:n] = price
48     rows, cols, vals, beq = [], [], [], []
49     r = 0
50     for i in range(n):
51         rows += [r, r, r, r]; cols += [i, 2 * n + i, n + i, 3 * n + i]
52         vals += [1.0, eta_d, -1.0 / eta_c, -1.0]
53         beq.append(L[i] / 6.0 - P[i] / 6.0); r += 1
54     for i in range(n):
55         rows.append(r); cols.append(4 * n + i); vals.append(1.0)
56         if i == 0:
57             rows.append(r); cols.append(n + i); vals.append(-1.0)
58             rows.append(r); cols.append(2 * n + i); vals.append(1.0)
59             beq.append(e0)
60         else:
61             rows.append(r); cols.append(4 * n + i - 1); vals.append(-1.0)
62             rows.append(r); cols.append(n + i); vals.append(-1.0)
63             rows.append(r); cols.append(2 * n + i); vals.append(1.0)
64             beq.append(0.0)
65         r += 1
66     rows.append(r); cols.append(4 * n + n - 1); vals.append(1.0); beq.append(e0); r += 1
67     A = sparse.csr_matrix((vals, (rows, cols)), shape=(r, nv))
68     bounds = [(0, None)] * n + [(0, abar)] * n + [(0, abar)] * n + [(0, None)] * n
69             + [(emin, emax)] * n
70     res = linprog(c, A_eq=A, b_eq=np.array(beq), bounds=bounds, method='highs')
71     x = res.x
72     return float(res.fun), x[:n], float(x[n:2 * n].sum()), x[4 * n:5 * n]
73
74 def main():
75     d1 = C.load_att1()
76     price1 = d1['电价'].values.astype(float)
77     L1 = d1['小区负载'].values.astype(float)
78     P1 = d1['光伏发电预测功率'].values.astype(float)
79     dates, load, pv = C.load_att2()
80     dp, price4 = C.load_att4()
81     rep = C.report_dates()
82     ridx = [C.day_index(dates, d) for d in rep]
83     nd = len(rep)
84     h = C.hours_axis()

```

```

85 hour_of = np.repeat(np.arange(24), 6)
86
87 # ===== 数据画像 =====
88 fig, ax = plt.subplots(2, 2, figsize=(11, 7))
89 im0 = ax[0, 0].imshow(load.T, aspect='auto', origin='lower', cmap=CMAP,
90 extent=[0, 365, 0, 24])
91 ax[0, 0].set_title('(a) 全年小区负载 (kW)'); ax[0, 0].set_ylabel('时刻 (h)')
92 ax[0, 0].set_xlabel('日期序号 (2025-01-01 起)'); fig.colorbar(im0, ax=ax[0, 0])
93 im1 = ax[0, 1].imshow(pv.T, aspect='auto', origin='lower', cmap=CMAP, extent=[0, 365,
94 0, 24])
95 ax[0, 1].set_title('(b) 全年光伏实际功率 (kW)'); ax[0, 1].set_xlabel('日期序号')
96 fig.colorbar(im1, ax=ax[0, 1])
97 im2 = ax[1, 0].imshow(price4.T, aspect='auto', origin='lower', cmap=CMAP, extent=[0,
98 365, 0, 24])
99 ax[1, 0].set_title('(c) 全年实时电价 (元/kWh)'); ax[1, 0].set_ylabel('时刻 (h)')
100 ax[1, 0].set_xlabel('日期序号'); fig.colorbar(im2, ax=ax[1, 0])
101 ax[1, 1].plot(h, L1, label='负载 (附件1)', color='tab:blue')
102 ax[1, 1].plot(h, P1, label='光伏预测 (附件1)', color='tab:orange')
103 ax[1, 1].plot(h, L1 - P1, label='净缺口', color='tab:red', lw=1.0)
104 ax2 = ax[1, 1].twinx()
105 ax2.plot(h, price1, label='电价 (附件1)', color='tab:green', ls='--', lw=1.0)
106 ax2.set_ylabel('电价 (元/kWh)', color='tab:green')
107 ax[1, 1].set_title('(d) 典型日 (附件1) 负载/光伏/净缺口 与电价')
108 ax[1, 1].set_xlabel('时刻 (h)'); ax[1, 1].set_ylabel('功率 (kW)')
109 ax[1, 1].legend(loc='upper left', fontsize=8)
110 ax2.legend(loc='upper right', fontsize=8)
111 fig.tight_layout(); sv(fig, '图01_数据总览.png')
112
113 month = np.array([d.month for d in dates])
114 mr = []
115 for m in range(1, 13):
116     mk = month == m
117     mr.append([m, load[mk].mean(), pv[mk].sum(1).mean() * C.DT / 1000,
118               (load[mk] - pv[mk]).sum(1).mean() * C.DT / 1000, price4[mk].mean()])
119 mr = np.array(mr)
120 fig, ax = plt.subplots(1, 2, figsize=(11, 3.9))
121 ax[0].bar(mr[:, 0] - 0.2, mr[:, 2], width=0.4, label='光伏日电量 (MWh)', color='tab:orange')
122 ax[0].bar(mr[:, 0] + 0.2, mr[:, 3], width=0.4, label='净缺口日电量 (MWh)', color='tab:red')
123 ax[0].set_xlabel('月份'); ax[0].set_ylabel('电量 (MWh)'); ax[0].legend()
124 ax[0].set_title('(a) 光伏电量与净缺口电量的月度分布')
125 ax[1].plot(mr[:, 0], mr[:, 1], 'o-', color='tab:blue', label='日均负载 (kW)')
126 axb = ax[1].twinx()
127 axb.bar(mr[:, 0], mr[:, 4], width=0.5, alpha=0.35, color='tab:green', label='月均价 (元/kWh)')
128 axb.set_ylabel('电价 (元/kWh)', color='tab:green')
129 ax[1].set_xlabel('月份'); ax[1].set_ylabel('负载 (kW)'); ax[1].legend(loc='upper left')
130 ax[1].set_title('(b) 负载与电价的月度特征')
131 fig.tight_layout(); sv(fig, '图02_月度特征.png')

```

表3 四天

```

132 kd = [pd.Timestamp(s).date() for s in ['2025-03-20', '2025-06-21', '2025-09-23', '
133         2025-12-21']]
134 fig, ax = plt.subplots(4, 1, figsize=(10.5, 9), sharex=True)
135 for i, d in enumerate(kd):
136     k = C.day_index(dates, d)
137     ax[i].plot(h, load[k], color='tab:blue', label='负载')
138     ax[i].plot(h, pv[k], color='tab:orange', label='光伏')
139     ax[i].fill_between(h, 0, np.minimum(load[k], pv[k]), color='tab:green', alpha=0.18)
140     ax2 = ax[i].twinx(); ax2.plot(h, price4[k], color='tab:red', ls='--', lw=1.0)
141     ax2.set_ylabel('电价', color='tab:red', fontsize=8)
142     ax[i].set_ylabel('kW'); ax[i].set_title('%s: 负载 %.0f MWh, 光伏 %.0f MWh, 缺口 %.0f
143         MWh'
144         % (d, load[k].sum() * C.DT / 1e3, pv[k].sum() * C.
145            DT / 1e3,
146            (load[k] - pv[k]).sum() * C.DT / 1e3), fontsize
147            =9)
148
149     if i == 0:
150         ax[i].legend(loc='upper left', fontsize=8)
151 ax[-1].set_xlabel('时刻 (h)'); ax[-1].set_xlim(0, 24)
152 fig.tight_layout(); sv(fig, '图03_表3四天.png')
153
154 # ===== 问题1 =====
155 det = pd.read_csv(os.path.join(RES, '问题1_逐时段明细.csv'))
156 g, chg, dis, soc = det['购电量kWh'].values, det['充电量kWh'].values, det['放电量kWh'].
157     values, det['储电量kWh'].values
158 fig, ax = plt.subplots(1, 2, figsize=(11, 3.8))
159 ax[0].plot(h, price1, color='tab:red')
160 ax[0].axhline(0.9 * 0.75, color='gray', ls='--', lw=1)
161 ax[0].axhline(1.111 * 0.75, color='gray', ls=':', lw=1)
162 ax[0].set_xlabel('时刻 (h)'); ax[0].set_ylabel('电价 (元/kWh)')
163 ax[0].set_title('(a) 电价曲线与阈值示意 (虚线 $0.9\\lambda$, 点线 $1.111\\lambda$)')
164 st = np.where(dis > 1e-3, 1, np.where(chg > 1e-3, -1, 0))
165 ax[1].scatter(h[st == -1], price1[st == -1], s=10, color='tab:blue', label='充电')
166 ax[1].scatter(h[st == 1], price1[st == 1], s=10, color='tab:red', label='放电')
167 ax[1].scatter(h[st == 0], price1[st == 0], s=6, color='lightgray', label='闲置')
168 ax[1].set_xlabel('时刻 (h)'); ax[1].set_ylabel('电价 (元/kWh)'); ax[1].legend()
169 ax[1].set_title('(b) 充放电状态在电价轴上的分布 (阈值策略验证)')
170 fig.tight_layout(); sv(fig, '图04_问题1_阈值策略.png')
171
172 fig, ax = plt.subplots(1, 2, figsize=(11, 3.8))
173 ax[0].bar(h, g / C.DT, width=1 / 6.0, color='tab:cyan')
174 ax[0].plot(h, (L1 - P1), color='tab:red', lw=1.0, label='净缺口功率')
175 ax[0].set_xlabel('时刻 (h)'); ax[0].set_ylabel('功率 (kW)'); ax[0].legend()
176 ax[0].set_title('(a) 计划购电功率与净缺口')
177 ax[0].set_xlim(0, 24)
178 ax[1].plot(np.arange(0, C.T + 1) / 6.0, np.concatenate([[C.E0], soc]), color='tab:blue'
179     )
180 ax[1].axhline(C.E_MAX, color='gray', ls='--', lw=1, label='上限 10800')
181 ax[1].axhline(C.E_MIN, color='gray', ls=':', lw=1, label='下限 1200')
182 ax[1].set_xlabel('时刻 (h)'); ax[1].set_ylabel('储电量 (kWh)'); ax[1].legend()
183 ax[1].set_title('(b) 储电量轨迹 (日周期 $E_{144}=E_0$)'); ax[1].set_xlim(0, 24)
184 fig.tight_layout(); sv(fig, '图05_问题1_策略与储电量.png')

```

```

178 # 灵敏度 (参数化 LP 重解, OAT)
179 scen = []
180 variants = [('基准', {}), ('$\\eta=0.85$', dict(eta=0.85)), ('$\\eta=0.95$', dict(eta
    =0.95)),
181             ('功率 $4000$ kW', dict(pb=4000.0)), ('功率 $6000$ kW', dict(pb=6000.0)),
182             ('$E_{\\min}=2400$', dict(emin=2400.0)), ('$E_{\\max}=9600$', dict(emax
    =9600.0)),
183             ('$E_0=3000$', dict(e0=3000.0)), ('$E_0=9000$', dict(e0=9000.0)),
184             ('电价 $+10\\%$', dict(pmul=1.10)), ('电价 $-10\\%$', dict(pmul=0.90))]
185 base_fee = None
186 for name, kw in variants:
187     fee, gg, tp, EE = lp_sens(L1, P1, price1 * kw.get('pmul', 1.0),
188                             eta_c=kw.get('eta', 0.9), eta_d=kw.get('eta', 0.9),
189                             abar=kw.get('pb', 5000.0) / 6.0,
190                             emin=kw.get('emin', 1200.0), emax=kw.get('emax', 10800.0),
191                             e0=kw.get('e0', 6000.0))
192     if base_fee is None:
193         base_fee = fee
194     scen.append({'scenario': name, 'fee': fee, 'throughput': tp})
195 sdf = pd.DataFrame(scen)
196 fig, ax = plt.subplots(1, 2, figsize=(11.5, 3.9))
197 x = np.arange(len(sdf))
198 ax[0].bar(x, sdf['fee'], color=['tab:gray'] + ['tab:blue'] * (len(sdf) - 1))
199 ax[0].axhline(base_fee, color='tab:red', ls='--', lw=1)
200 ax[0].set_xticks(x); ax[0].set_xticklabels(sdf['scenario'], rotation=40, ha='right',
    fontsize=8)
201 ax[0].set_ylabel('全天购电费 (元)'); ax[0].set_title('(a) 单因素扰动下的全天购电费 (虚
    线为基准)')
202 ch = 100 * (sdf['fee'] - base_fee) / base_fee
203 order = np.argsort(np.abs(ch.values))
204 ax[1].barh(range(len(sdf)), ch.values[order],
205            color=np.where(ch.values[order] >= 0, 'tab:red', 'tab:blue'))
206 ax[1].set_yticks(range(len(sdf)))
207 ax[1].set_yticklabels([sdf['scenario'].values[i] for i in order], fontsize=8)
208 ax[1].axvline(0, color='k', lw=0.8); ax[1].set_xlabel('费用变化 (%)')
209 ax[1].set_title('(b) 龙卷风图 (按影响排序)')
210 fig.tight_layout(); sv(fig, '图06_问题1灵敏度.png')
211 sdf['费用变化%'] = ch
212 sdf.to_csv(os.path.join(OUT, '灵敏度_问题1.csv'), index=False, encoding='utf-8-sig')
213
214 # ===== 问题2 =====
215 n2 = np.load(os.path.join(RES, '问题2_结果.npz'))
216 prec = pd.read_csv(os.path.join(RES, '问题2_预测器精度.csv'))
217 fig, ax = plt.subplots(1, 2, figsize=(11, 3.9))
218 x = np.arange(len(prec))
219 ax[0].bar(x - 0.2, prec['负载MAE(kw)'], width=0.4, label='负载 MAE', color='tab:blue')
220 ax[0].bar(x + 0.2, prec['负载RMSE(kw)'], width=0.4, label='负载 RMSE', color='tab:cyan')
221 ax[0].set_xticks(x); ax[0].set_xticklabels(prec['预测器'], rotation=38, ha='right',
    fontsize=8)
222 ax[0].set_ylabel('kw'); ax[0].legend(); ax[0].set_title('(a) 负载预测器精度 (滚动评价)')

```

```

223 ax[1].bar(x - 0.2, prec['光伏MAE(kw)'], width=0.4, label='光伏 MAE', color='tab:orange'
224 )
225 ax[1].bar(x + 0.2, prec['光伏RMSE(kw)'], width=0.4, label='光伏 RMSE', color='tab:red')
226 ax[1].set_xticks(x); ax[1].set_xticklabels(prec['预测器'], rotation=38, ha='right',
227     fontsize=8)
228 ax[1].set_ylabel('kw'); ax[1].legend(); ax[1].set_title('(b) 光伏预测器精度 (滚动评价)')
229 fig.tight_layout(); sv(fig, '图07_问题2_预测器精度.png')
230
231 cal = pd.read_csv(os.path.join(RES, '问题2_裕度标定.csv'))
232 sel = cal[(cal.alpha > 0) | (cal.beta >= 0)]
233 piv = sel.pivot_table(index='alpha', columns='beta', values='total_cost_jan', aggfunc='
234 min')
235 fig, ax = plt.subplots(1, 2, figsize=(11, 3.9))
236 im = ax[0].imshow(piv.values / 1e4, aspect='auto', origin='lower', cmap=CMAP)
237 ax[0].set_xticks(range(len(piv.columns))); ax[0].set_xticklabels(['%.2f' % b for b in
238     piv.columns])
239 ax[0].set_yticks(range(len(piv.index))); ax[0].set_yticklabels(['%.2f' % a for a in piv
240     .index])
241 ax[0].set_xlabel('光伏下调系数  $\beta$ '); ax[0].set_ylabel('负载上浮系数  $\alpha$ ')
242 ax[0].set_title('(a) 1 月标定: 总费用 (万元)'); fig.colorbar(im, ax=ax[0])
243 b_alpha = sel.groupby('alpha')['total_cost_jan'].min()
244 ax[1].plot(b_alpha.index, b_alpha.values / 1e4, 'o-', color='tab:blue')
245 ax[1].axhline(sel['total_cost_jan'].max() / 1e4, color='tab:red', ls='--', lw=1, label=
246     '无裕度')
247 ax[1].set_xlabel('负载上浮系数  $\alpha$ '); ax[1].set_ylabel('1 月总费用 (万元)')
248 ax[1].legend(); ax[1].set_title('(b) 最优裕度的内部极小点')
249 fig.tight_layout(); sv(fig, '图08_问题2_裕度标定.png')
250
251 m2 = pd.read_csv(os.path.join(RES, '问题2_月度费用汇总.csv'))
252 m2 = m2[m2['月份'] != '合计']
253 fig, ax = plt.subplots(1, 2, figsize=(11, 3.9))
254 ax[0].bar(m2['月份'], m2['计划购电费元'] / 1e4, label='计划购电费', color='tab:blue')
255 ax[0].bar(m2['月份'], m2['紧急购电费元'] / 1e4, bottom=m2['计划购电费元'] / 1e4,
256     label='紧急购电费', color='tab:red')
257 ax[0].set_xlabel('月份'); ax[0].set_ylabel('费用 (万元)'); ax[0].legend()
258 ax[0].set_title('(a) 月度费用构成 (问题2)')
259 EM = m2['EM']
260 mtx = np.zeros((12, 24))
261 for k, d in enumerate(rep):
262     mtx[d.month - 1] += EM[k].reshape(24, 6).sum(1)
263 im = ax[1].imshow(mtx, aspect='auto', origin='lower', cmap=CMAP)
264 ax[1].set_xlabel('时刻 (h)'); ax[1].set_ylabel('月份')
265 ax[1].set_xticks(range(0, 24, 3)); ax[1].set_xticklabels(range(0, 24, 3))
266 ax[1].set_title('(b) 紧急购电量的时段-月度分布 (kWh)'); fig.colorbar(im, ax=ax[1])
267 fig.tight_layout(); sv(fig, '图09_问题2_月度与紧急分布.png')
268
269 fig, ax = plt.subplots(1, 2, figsize=(11, 3.9))
270 labs = ['无裕度计划', '预测+裕度', '完美信息']
271 vals = [float(n2['cost0'].sum()) / 1e4, float((n2['plan_cost'].sum() + n2['emg_cost'].
272     sum())) / 1e4,
273     float(n2['costP'].sum()) / 1e4]

```

```

267 ax[0].bar(labs, vals, color=['tab:gray', 'tab:blue', 'tab:green'])
268 for i, v in enumerate(vals):
269     ax[0].text(i, v, '%.0f' % v, ha='center', va='bottom', fontsize=9)
270 ax[0].set_ylabel('全年总费用 (万元)'); ax[0].set_title('(a) 裕度价值与信息价值 (问题2)')
271
272 ax[1].hist(n2['EM'].sum(1), bins=30, color='tab:red', alpha=0.8)
273 ax[1].set_xlabel('当日紧急购电量 (kWh)'); ax[1].set_ylabel('天数')
274 ax[1].set_title('(b) 每日紧急购电量分布')
275 fig.tight_layout(); sv(fig, '图10_问题2_价值与分布.png')
276
277 # ===== 问题3 =====
278 n3 = np.load(os.path.join(RES, '问题3_结果.npz'))
279 s3 = pd.read_csv(os.path.join(RES, '问题3_策略对比.csv'))
280 fig, ax = plt.subplots(1, 2, figsize=(11, 4.1))
281 cols = ['tab:gray', 'tab:blue', 'tab:cyan', 'tab:green', 'tab:olive', 'tab:orange', 'tab:brown', 'tab:purple']
282 ax[0].bar(s3['策略'], s3['总费用'] / 1e4, color=cols[:len(s3)])
283 for i, v in enumerate(s3['总费用'] / 1e4):
284     ax[0].text(i, v, '%.0f' % v, ha='center', va='bottom', fontsize=8)
285 ax[0].set_ylabel('全年总费用 (万元)'); ax[0].set_title('(a) 各预报更新策略的费用')
286 ax[1].bar(s3['策略'], s3['违约费'] / 1e4, color='tab:orange', label='违约费')
287 ax[1].bar(s3['策略'], s3['紧急购电费'] / 1e4, bottom=s3['违约费'] / 1e4, color='tab:red', label='紧急购电费')
288 ax[1].set_ylabel('费用 (万元)'); ax[1].legend(); ax[1].set_title('(b) 费用构成对比')
289 fig.tight_layout(); sv(fig, '图11_问题3_策略对比.png')
290
291 rev = pd.read_csv(os.path.join(RES, '问题3_预报修订幅度.csv'))
292 fig, ax = plt.subplots(1, 2, figsize=(11, 3.8))
293 x = np.arange(len(rev))
294 ax[0].bar(x - 0.2, rev['MAE(kw)'], width=0.4, label='MAE', color='tab:blue')
295 ax[0].bar(x + 0.2, rev['RMSE(kw)'], width=0.4, label='RMSE', color='tab:cyan')
296 ax[0].set_xticks(x); ax[0].set_xticklabels(rev['修订'], fontsize=9)
297 ax[0].set_ylabel('kw'); ax[0].legend(); ax[0].set_title('(a) 预报修订幅度 (同一时钟小时)')
298
299 marg = pd.read_csv(os.path.join(RES, '问题3_更新边际价值.csv'))
300 info = marg[marg['项目'].str.contains('价值')]
301 names = [s.split('(')[0] for s in info['项目']]
302 ax[1].barh(range(len(info)), info['费用下降(元)'] / 1e4, color=['tab:orange', 'tab:blue', 'tab:green'])
303 ax[1].set_yticks(range(len(info))); ax[1].set_yticklabels(names, fontsize=9)
304 ax[1].set_xlabel('价值 (万元)'); ax[1].set_title('(b) 信息价值分解 (单阶段口径)')
305 fig.tight_layout(); sv(fig, '图12_问题3_修订与信息价值.png')
306
307 EM3, G_HAT, G3 = n3['EM3'], n3['G_HAT'], n3['G3']
308 fig, ax = plt.subplots(2, 1, figsize=(10.5, 6), sharex=True)
309 ks = [C.day_index(dates, d) for d in kd]
310 kk = rep.index(kd[3])
311 ax[0].plot(h, G_HAT[kk] / C.DT, label='0:00 计划购电', color='tab:blue')
312 ax[0].plot(h, G3[kk] / C.DT, label='最终调整购电', color='tab:green')
313 ax[0].plot(h, EM3[kk] / C.DT, label='紧急购电', color='tab:red', lw=1.0)
314 ax[0].plot(h, load[ks[3]], 'k--', lw=0.8, label='实际负载')
315 ax[0].set_ylabel('功率 (kW)'); ax[0].legend(fontsize=8)

```

```

314 ax[0].set_title('2025-12-21: 计划/调整/紧急购电与实际负载')
315 ax[1].plot(np.arange(0, C.T + 1) / 6.0, n3['E3'][kk], color='tab:blue')
316 ax[1].axhline(C.E_MAX, color='gray', ls='--', lw=1); ax[1].axhline(C.E_MIN, color='gray',
317     , ls=':', lw=1)
318 ax[1].set_ylabel('储电量 (kWh)'); ax[1].set_xlabel('时刻 (h)'); ax[1].set_xlim(0, 24)
319 fig.tight_layout(); sv(fig, '图13_问题3_最难日.png')
320
321 # ===== 问题4 =====
322 n4 = np.load(os.path.join(RES, '问题4_结果.npz'))
323 pdf4 = pd.read_csv(os.path.join(RES, '问题4_电价预测精度.csv'))
324 idx4 = n4['rep_idx']
325 pa = price4[idx4]
326 fig, ax = plt.subplots(1, 2, figsize=(11, 3.9))
327 x = np.arange(len(pdf4))
328 ax[0].bar(x, pdf4['MAE(元/kWh)'] * 1000, color='tab:purple')
329 ax[0].set_xticks(x); ax[0].set_xticklabels(pdf4['预测器'], rotation=38, ha='right',
330     fontsize=8)
331 ax[0].set_ylabel('MAE ( $\times 10^3$ ) 元/kWh'); ax[0].set_title('(a) 电价预测器精度')
332 d0 = C.day_index(dp, kd[1])
333 ax[1].plot(h, price4[d0], color='tab:red', label='实际电价')
334 ax[1].plot(h, C.pred(price4[:d0], dp[:d0], '7日均', kd[1]), color='tab:blue', ls='--',
335     label='7 日均预测')
336 ax[1].plot(h, C.pred(price4[:d0], dp[:d0], '昨日持续', kd[1]), color='tab:green', ls=':',
337     label='昨日持续')
338 ax[1].set_xlabel('时刻 (h)'); ax[1].set_ylabel('电价 (元/kWh)'); ax[1].legend(fontsize
339     =8)
340 ax[1].set_title('(b) 2025-06-21 电价预测与实际'); ax[1].set_xlim(0, 24)
341 fig.tight_layout(); sv(fig, '图14_问题4_电价预测.png')
342
343 m4 = pd.read_csv(os.path.join(RES, '问题4_月度费用对比.csv'))
344 fig, ax = plt.subplots(1, 2, figsize=(11, 3.9))
345 ax[0].plot(m4['月份'], m4['电价均值'], 'o-', color='tab:green')
346 ax[0].set_xlabel('月份'); ax[0].set_ylabel('月均电价 (元/kWh)')
347 ax[0].set_title('(a) 实时电价月度水平')
348 ax0b = ax[0].twinx()
349 ax0b.bar(m4['月份'], m4['4-2紧急量kWh'] / 1e3, alpha=0.3, color='tab:red')
350 ax0b.set_ylabel('紧急购电量 (MWh)', color='tab:red')
351 ax[1].bar(m4['月份'] - 0.2, m4['4-2总费用'] / 1e4, width=0.4, label='问题4-2 (重算问题2)',
352     color='tab:blue')
353 ax[1].bar(m4['月份'] + 0.2, m4['4-3总费用'] / 1e4, width=0.4, label='问题4-3 (重算问题3)',
354     color='tab:orange')
355 ax[1].set_xlabel('月份'); ax[1].set_ylabel('总费用 (万元)'); ax[1].legend(fontsize=8)
356 ax[1].set_title('(b) 波动电价下月度费用')
357 fig.tight_layout(); sv(fig, '图15_问题4_月度对比.png')
358
359 fig, ax = plt.subplots(1, 3, figsize=(12.5, 3.6))
360 ax[0].scatter(load[ridx].ravel(), pa.ravel(), s=1, alpha=0.15, color='tab:blue')
361 r = np.corrcoef(load[ridx].ravel(), pa.ravel())[0, 1]
362 ax[0].set_xlabel('负载 (kW)'); ax[0].set_ylabel('电价 (元/kWh)')
363 ax[0].set_title('(a) 电价与负载: $r=0.3f$' % r)
364 ax[1].scatter(pv[ridx].ravel(), pa.ravel(), s=1, alpha=0.15, color='tab:orange')
365 r2 = np.corrcoef(pv[ridx].ravel(), pa.ravel())[0, 1]

```

```

359 ax[1].set_xlabel('光伏 (kW)'); ax[1].set_title('(b) 电价与光伏: $r=%.3f$' % r2)
360 ax[2].boxplot([pa[np.array([d.month for d in rep]) == m].ravel() for m in range(2, 13)
361 ],
362               tick_labels=range(2, 13), showfliers=False)
363 ax[2].set_xlabel('月份'); ax[2].set_ylabel('电价 (元/kWh)'); ax[2].set_title('(c) 电价
364 月度箱线图')
365 fig.tight_layout(); sv(fig, '图16_问题4_电价相关性.png')
366
367 s43 = pd.read_csv(os.path.join(RES, '问题4_4-3策略对比.csv'))
368 fig, ax = plt.subplots(1, 2, figsize=(11, 3.9))
369 tot2 = float(n4['plan42'].sum() + n4['emg42'].sum())
370 s3p3 = float(s3.loc[s3.策略 == 'P3', '总费用'].values[0])
371 lab = ['问题2\n(固定电价)', '问题4-2\n(波动电价)', '问题3\n(固定电价)', '问题4-3\n(波动
372 电价)']
373 val = [16647984.89 / 1e4, tot2 / 1e4, s3p3 / 1e4, float(s43.loc[s43.策略 == 'P3', '总费
374 用'].values[0]) / 1e4]
375 ax[0].bar(lab, val, color=['tab:blue', 'tab:red', 'tab:cyan', 'tab:orange'])
376 for i, v in enumerate(val):
377     ax[0].text(i, v, '%.0f' % v, ha='center', va='bottom', fontsize=9)
378 ax[0].set_ylabel('全年总费用 (万元)'); ax[0].set_title('(a) 固定电价 vs 波动电价')
379 ax[1].bar(s43['策略'], s43['总费用'] / 1e4,
380           color=['tab:gray', 'tab:blue', 'tab:cyan', 'tab:green', 'tab:olive', 'tab:
381 purple'][:len(s43)])
382 ax[1].set_ylabel('全年总费用 (万元)'); ax[1].set_title('(b) 波动电价下的更新策略')
383 fig.tight_layout(); sv(fig, '图17_问题4_策略对比.png')
384
385 print('全部图表已输出到 %s' % OUT)
386
387 if __name__ == '__main__':
388     main()

```

B.8 sensitivity.py: 补充灵敏度实验

```

1 # -*- coding: utf-8 -*-
2 """Phase 3 补充灵敏度实验
3 (1) 问题2: 紧急购电费率倍数 K 的敏感性 (计划不变, 按实际负载/光伏重新结算)
4 (2) 问题3: 违约费率取对称 (1.0,1.0) (dev_pen=1.0) 时 P0--P3 的全年费用
5 输出: 论文/图/灵敏度_问题2_费率.csv、灵敏度_问题3_费率.csv
6 """
7 import os
8 import sys
9 import importlib.util
10 import numpy as np
11 import pandas as pd
12
13 def _find_dir(name, maxup=8):
14     """自脚本目录向上查找子目录 name, 使附录中的源码副本在任意布局下均可运行"""
15     cur = os.path.dirname(os.path.abspath(__file__))
16     for _ in range(maxup):
17         cand = os.path.join(cur, name)
18         if os.path.isdir(cand):

```

```

19         return cand
20     parent = os.path.dirname(cur)
21     if parent == cur:
22         break
23     cur = parent
24     return os.path.join(os.path.dirname(os.path.abspath(__file__)), name)
25
26
27 _att = _find_dir('附件')
28 ROOT = os.path.dirname(_att)
29 sys.path.insert(0, os.path.join(ROOT, '求解'))
30 import common as C
31
32 OUT = os.path.join(ROOT, '论文', '图')
33 RES = C.RES
34
35 d1 = C.load_att1()
36 price = d1['电价'].values.astype(float)
37 dates, load, pv = C.load_att2()
38 rep = C.report_dates()
39 ridx = [C.day_index(dates, d) for d in rep]
40 n2 = np.load(os.path.join(RES, '问题2_结果.npz'))
41 G, A, B = n2['G'], n2['A'], n2['B']
42 L, P = load[ridx], pv[ridx]
43
44 # ----- (1) 紧急费率倍数 K -----
45 e_all = np.maximum(0.0, L * C.DT - G - C.ETA_D * B - P * C.DT)
46 plan_fee = float((G * price).sum())
47 rows = []
48 for K in [3, 4, 5, 6, 8]:
49     emg = K * float((e_all * price).sum())
50     rows.append({'紧急费率倍数K': K, '计划购电费元': plan_fee, '紧急购电费元': emg,
51                 '总费用元': plan_fee + emg, '紧急费占比%': 100 * emg / (plan_fee + emg)})
52 df1 = pd.DataFrame(rows)
53 df1.to_csv(os.path.join(OUT, '灵敏度_问题2_费率.csv'), index=False, encoding='utf-8-sig')
54 print(' (1) 问题2 紧急费率倍数敏感性 (总费用为 334 天合计) ')
55 print(df1.to_string(index=False, float_format=lambda x: '%.0f' % x))
56
57 # ----- (2) 问题3 违约费率对称 (1.0,1.0) -----
58 spec = importlib.util.spec_from_file_location('p3', os.path.join(ROOT, '求解', '问题3', '
    问题3_求解.py'))
59 p3 = importlib.util.module_from_spec(spec)
60 spec.loader.exec_module(p3)
61 fc = C.load_att3()
62 L_PRED = C.best_load_predictor()
63 ALPHA = 0.05
64
65 for dev_pen, tag in [(0.5, '基准(0.5,1.5)'), (1.0, '对称(1.0,1.0)')]:
66     p3.DEV = dev_pen
67     tot = {p: 0.0 for p in ['P0', 'P1', 'P2', 'P3']}
68     for d, i in zip(rep, ridx):
69         Lhat = C.pred(load[:i], dates[:i], L_PRED, d)

```

```

70     out = p3.run_day(fc, d, i, load, pv, price, Lhat, ALPHA, hourly=False)
71     for p in tot:
72         tot[p] += out[p]['total']
73     rows = [{'违约费率口径': tag, '策略': p, '总费用元': tot[p]} for p in ['P0', 'P1', 'P2',
74         , 'P3']]
75     df2 = pd.DataFrame(rows)
76     if dev_pen == 0.5:
77         df_all = df2
78     else:
79         df_all = pd.concat([df_all, df2], ignore_index=True)
80     print(' (2) %s: P0=%.0f P1=%.0f P2=%.0f P3=%.0f' % (tag, tot['P0'], tot['P1'], tot['P2'],
81         tot['P3']))
82 df_all.to_csv(os.path.join(OUT, '灵敏度_问题3_费率.csv'), index=False, encoding='utf-8-sig')
83 print('完成')

```

B.9 explore_data.py: 数据探索

```

1  # -*- coding: utf-8 -*-
2  """CUMCM2026 C题 数据探索脚本（分析阶段）
3  输出关键统计量，供《框架设计.md》引用。
4  运行方式：在工作区根目录执行 python 求解/数据探索.py
5  """
6  import os
7  import datetime as dt
8  import numpy as np
9  import pandas as pd
10
11  ROOT = os.getcwd()
12  def P(*a):
13      return os.path.join(ROOT, *a)
14
15  def hour_label(k):
16      """小时 k (1..24) 对应的附件2时间列标签（区间末时刻）。"""
17      return '0:00+1' if k == 24 else dt.time(k, 0)
18
19  print("=" * 78)
20  print("附件1（某天：电价 / 小区负载 / 光伏预测功率，10分钟粒度）")
21  print("=" * 78)
22  df1 = pd.read_excel(P("附件", "附件1.xlsx"))
23  df1.columns = ["时间", "电价", "小区负载", "光伏发电预测功率"]
24  for c in df1.columns[1:]:
25      df1[c] = pd.to_numeric(df1[c], errors="coerce")
26  print("行数 =", len(df1), "| 首/末时间标签:", df1["时间"].iloc[0], "->", df1["时间"].iloc
27      [-1])
28  print("电价: min %.4f / max %.4f / mean %.4f 元/kWh" % (df1["电价"].min(), df1["电价"].max
29      (), df1["电价"].mean()))
30  print("负载: min %.0f / max %.0f / mean %.0f kW | 日电量 %.0f kWh" %
31      (df1["小区负载"].min(), df1["小区负载"].max(), df1["小区负载"].mean(),
32      df1["小区负载"].sum() * 10 / 60))
33  print("光伏预测: max %.0f kW | 非零时段 %d 个(%.1f h) | 日电量 %.0f kWh" %
34      (df1["光伏发电预测功率"].max(), (df1["光伏发电预测功率"] > 0).sum(),

```

```

33     (df1["光伏发电预测功率"] > 0).sum() * 10 / 60,
34     df1["光伏发电预测功率"].sum() * 10 / 60))
35 print("净缺口(负载-光伏)日电量 %.0f kWh | 光伏日电量占负载 %.1f%%" %
36       ((df1["小区负载"] - df1["光伏发电预测功率"]).sum() * 10 / 60,
37        100 * df1["光伏发电预测功率"].sum() / df1["小区负载"].sum()))
38 print("电价日内小时均值曲线(元/kWh):")
39 hp = df1["电价"].values.reshape(24, 6).mean(1)
40 print(" " + " ".join("%2d时%.3f" % (h, hp[h]) for h in range(24)))
41
42 print()
43 print("=" * 78)
44 print("附件2 (2025.1.1-12.31 每天 小区负载 / 光伏实际功率, 10分钟粒度)")
45 print("=" * 78)
46 load_df = pd.read_excel(P("附件", "附件2.xlsx"), sheet_name="小区负载")
47 pv_df = pd.read_excel(P("附件", "附件2.xlsx"), sheet_name="光伏发电实际功率")
48 load_v = load_df.iloc[:, 1:].astype(float).values # (365, 144)
49 pv_v = pv_df.iloc[:, 1:].astype(float).values
50 times = list(load_df.columns[1:])
51 dates = pd.to_datetime(load_df.iloc[:, 0].astype(str).str[:10])
52 print("形状:", load_v.shape, "| 列标签类型:", type(times[0]).__name__, "| 末列:", times
53       [-1])
54 print("负载: 全年 min %.0f / max %.0f / mean %.0f kW | 日电量 mean %.0f / max %.0f kWh" %
55       (load_v.min(), load_v.max(), load_v.mean(),
56        (load_v.sum(1) * 10 / 60).mean(), (load_v.sum(1) * 10 / 60).max()))
57 print("光伏实际: max %.0f kW | 日电量 min %.0f / mean %.0f / max %.0f kWh" %
58       (pv_v.max(), (pv_v.sum(1) * 10 / 60).min(),
59        (pv_v.sum(1) * 10 / 60).mean(), (pv_v.sum(1) * 10 / 60).max()))
60 print("月度统计 (日均负载 kW | 日均光伏电量 kWh | 日均缺口电量 kWh) :")
61 month = dates.dt.month.values
62 for m in range(1, 13):
63     mask = month == m
64     print(" %2d月: %6.0f | %6.0f | %6.0f" %
65           (m, load_v[mask].mean(), (pv_v[mask].sum(1) * 10 / 60).mean(),
66            ((load_v - pv_v)[mask].sum(1) * 10 / 60).mean()))
67 print("日内典型曲线 (全年小时均值, kW) :")
68 for k in range(24):
69     a, b = k * 6, min(k * 6 + 6, 144)
70     print(" %2d:00 %6.0f %6.0f" % (k, load_v[:, a:b].mean(), pv_v[:, a:b].mean()))
71 print("附件1 vs 附件2全年均值: 负载相关 %.4f | 光伏相关 %.4f" %
72       (np.corrcoef(df1["小区负载"].values, load_v.mean(0))[0, 1],
73        np.corrcoef(df1["光伏发电预测功率"].values, pv_v.mean(0))[0, 1]))
74
75 print()
76 print("=" * 78)
77 print("预测难度评估 (为问题2的0:00决策提供依据)")
78 print("=" * 78)
79 # 昨日持续法: 用昨天同时刻值预测今天
80 persist_load = load_v[:-1] # 预测 = 昨日实际
81 persist_pv = pv_v[:-1]
82 err_l = load_v[1:] - persist_load
83 err_p = pv_v[1:] - persist_pv
84 print("昨日持续法误差(同时刻, kW): 负载 MAE %.0f RMSE %.0f | 光伏 MAE %.0f RMSE %.0f" %

```

```

84     (np.abs(err_l).mean(), np.sqrt((err_l ** 2).mean()),
85     np.abs(err_p).mean(), np.sqrt((err_p ** 2).mean()))
86 # 7天滑动平均法
87 w = np.ones(7) / 7
88 smooth_l = np.apply_along_axis(lambda c: np.convolve(c, w, 'valid'), 0, load_v)
89 smooth_p = np.apply_along_axis(lambda c: np.convolve(c, w, 'valid'), 0, pv_v)
90 err_l7 = load_v[6:] - smooth_l
91 err_p7 = pv_v[6:] - smooth_p
92 print("7天滑动平均法误差(同时刻, kW): 负载 MAE %.0f RMSE %.0f | 光伏 MAE %.0f RMSE %.0f" %
93       (np.abs(err_l7).mean(), np.sqrt((err_l7 ** 2).mean()),
94       np.abs(err_p7).mean(), np.sqrt((err_p7 ** 2).mean())))
95 # 缺口电量误差 (预测-实际, 全天): 昨日持续法的缺口能量误差分布
96 gap_act = (load_v - pv_v).sum(1) * 10 / 60
97 gap_fc = (load_v[:-1] - pv_v[:-1]).sum(1) * 10 / 60
98 e_gap = gap_act[1:] - gap_fc
99 print("全天缺口电量 昨日持续法误差: mean %+.0f kWh | MAE %.0f | std %.0f | P5/P95 %+.0f
    /%+.0f" %
100      (e_gap.mean(), np.abs(e_gap).mean(), e_gap.std(),
101      np.percentile(e_gap, 5), np.percentile(e_gap, 95)))
102
103 print()
104 print("=" * 78)
105 print("附件3 (4个预报时刻 x 未来24小时整点光伏预报)")
106 print("=" * 78)
107 fc = pd.read_excel(P("附件", "附件3.xlsx"))
108 fc.columns = ["日期", "预报时刻"] + ["预报%d小时" % i for i in range(1, 25)]
109 fc["日期"] = fc["日期"].ffill()
110 fc["日期"] = pd.to_datetime(fc["日期"])
111 print("行数:", len(fc), "| 每天预报次数:", set(fc.groupby("日期").size().values),
112       "| 预报时刻:", sorted(fc["预报时刻"].unique()))
113 # 实际小时光伏 (区间末时刻列)
114 actual_h = {k: pv_v[:, times.index(hour_label(k))] for k in range(1, 25)}
115 print("0:00预报按提前期的误差 (365天, kW | MAE / RMSE / bias):")
116 zero_fc = fc[fc["预报时刻"] == "0:00"]
117 for lead in [1, 3, 6, 9, 12, 15, 18, 21, 24]:
118     fv = zero_fc["预报%d小时" % lead].astype(float).values
119     av = actual_h[lead]
120     err = fv - av
121     print("提前%2dh: MAE %6.1f | RMSE %6.1f | bias %6.1f" %
122          (lead, np.abs(err).mean(), np.sqrt((err ** 2).mean()), err.mean()))
123 print("0:00预报全天电量误差: |误差|日电量 mean %.0f kWh (实际日均 %.0f, 相对 %.1f%%)" %
124      (np.abs(zero_fc.iloc[:, 2:].astype(float).values -
125            np.column_stack([actual_h[k] for k in range(1, 25)]).sum(1).mean(),
126            np.column_stack([actual_h[k] for k in range(1, 25)]).sum(1).mean(),
127            100 * np.abs(zero_fc.iloc[:, 2:].astype(float).values.sum(1) -
128                        np.column_stack([actual_h[k] for k in range(1, 25)]).sum(1)).mean() /
129            np.column_stack([actual_h[k] for k in range(1, 25)]).sum(1).mean()))
130 # 6:00预报对当天剩余时段(7:00-24:00) 是否优于0:00预报
131 z7_24 = zero_fc["预报%d小时" % k for k in range(7, 25)].astype(float).values
132 six_fc = fc[fc["预报时刻"] == "6:00"][["预报%d小时" % k for k in range(1, 19)]].astype(
133     float).values
134 act7_24 = np.column_stack([actual_h[k] for k in range(7, 25)])

```

```

134 print("当天7:00-24:00光伏 MAE: 0:00预报 %.0f kW vs 6:00预报 %.0f kW (改善 %.0f%%)" %
135       (np.abs(z7_24 - act7_24).mean(), np.abs(six_fc - act7_24).mean(),
136        100 * (1 - np.abs(six_fc - act7_24).mean() / np.abs(z7_24 - act7_24).mean())))
137
138 print()
139 print("=" * 78)
140 print("附件4 (2025全年 10分钟电价)")
141 print("=" * 78)
142 pr_df = pd.read_excel(P("附件", "附件4.xlsx"))
143 pr_v = pr_df.iloc[:, 1:].astype(float).values
144 print("电价: 全年 min %.4f / max %.4f / mean %.4f 元/kWh" %
145       (pr_v.min(), pr_v.max(), pr_v.mean()))
146 print("日内价差(日max-min): mean %.4f / max %.4f 元/kWh" %
147       ((pr_v.max(1) - pr_v.min(1)).mean(), (pr_v.max(1) - pr_v.min(1)).max()))
148 print("同一时刻跨日标准差: mean %.4f 元/kWh | 相邻日同刻差 std %.4f" %
149       (pr_v.std(0).mean(), np.diff(pr_v, axis=0).std()))
150 print("电价与负载(同时刻)相关系数 %.3f | 电价与光伏相关系数 %.3f" %
151       (np.corrcoef(pr_v.ravel(), load_v.ravel())[0, 1],
152        np.corrcoef(pr_v.ravel(), pv_v.ravel())[0, 1]))
153 print("电价日内典型曲线(小时均值, 元/kWh):")
154 hourly = pr_v.reshape(365, 24, 6).mean(2).mean(0)
155 print(" " + " ".join("%2d时%.3f" % (h, hourly[h]) for h in range(24)))
156 print("附件1电价曲线 vs 附件4年均日内曲线 相关系数: %.3f" %
157       np.corrcoef(df1["电价"].values, hourly.repeat(6))[0, 1])
158 print("月度均价:")
159 for m in range(1, 13):
160     print("%2d月: %.4f" % (m, pr_v[month == m].mean()))
161 print("电价昨日持续法误差(元/kWh): MAE %.4f RMSE %.4f" %
162       (np.abs(np.diff(pr_v, axis=0)).mean(), np.sqrt((np.diff(pr_v, axis=0) ** 2).mean())))
163
164 print()
165 print("=" * 78)
166 print("表3 指定4天的概况(负载/光伏/电价)")
167 print("=" * 78)
168 for ds in ["2025-03-20", "2025-06-21", "2025-09-23", "2025-12-21"]:
169     d = pd.Timestamp(ds)
170     i = np.where(dates == d)[0]
171     if len(i) == 0:
172         print(ds, "未找到"); continue
173     i = i[0]
174     print("%s: 负载max %.0f kW | 日负载电量 %.0f | 光伏电量 %.0f kWh | 缺口 %.0f kWh | 电价
175           mean %.4f/max %.4f" %
176           (ds, load_v[i].max(), load_v[i].sum() * 10 / 60, pv_v[i].sum() * 10 / 60,
177            (load_v[i] - pv_v[i]).sum() * 10 / 60, pr_v[i].mean(), pr_v[i].max()))
178
179 print()
180 print("=" * 78)
181 print("附件5 模板核对")
182 print("=" * 78)
183 r1 = pd.read_excel(P("附件", "附件5", "result1.xlsx"), sheet_name=None)
184 for name, sh in r1.items():
185     print("result1[%s] %s | 首行: %s | 末行: %s" % (name, sh.shape, list(sh.iloc[0]), list(

```

```
        sh.iloc[-1]))
185 r2 = pd.read_excel(P("附件", "附件5", "result2.xlsx"), sheet_name=None)
186 for name, sh in r2.items():
187     print("result2[%s] %s | 首末列: %s ... %s" %
188           (name, sh.shape, list(sh.columns[:2]), list(sh.columns[-3:])))
```